

Anexo III

Código completo

FairNight: un sistema de recomendación de precios para anuncios de Airbnb

Trabajo Fin de Máster, Modalidad Semipresencial
Yago Coll Crespo, Septiembre 2026

Código completo: aplicación FairNight y pipeline de Barcelona

Este anexo recoge el código fuente completo de dos partes del proyecto: la aplicación FairNight (app/) en su totalidad, y el pipeline de modelización de **Barcelona** (notebooks/barcelona/, solo celdas de código; el razonamiento y los resultados de cada celda, en markdown, tablas y gráficas, ya están recogidos en la memoria y en el Anexo II). Barcelona se toma como ciudad de referencia por ser la más documentada; las otras 6 ciudades ejecutan el **mismo pipeline** sobre sus propios datos (Sección 2.4 y 4.5 de la memoria, Anexo I Sección L y Anexo II Sección M detallan en qué difiere cada una), así que no se repite aquí: su código está disponible en el repositorio público de GitHub, github.com/yagocoll/Airbnb.

Estructura del repositorio

```
Airbnb/  
  app/ # aplicacion Streamlit FairNight  
    app.py # interfaz y orquestacion (formulario, prediccion, comparables,  
      mapa)  
    features.py # reconstruye el vector de features que espera cada price_model.pkl  
    comparables.py # seleccion del conjunto de anuncios reales comparables  
    requirements.txt # dependencias exactas de la app  
    assets/<ciudad>/ # geojson de distritos, importancia de variables, fotos, logo  
  notebooks/<ciudad>/ # 01_eda, 02_feature_engineering, 03_model_baseline,  
    # 04_model_training, 05_model_evaluation (una carpeta por ciudad)  
  models/<ciudad>/ # price_model.pkl + model_metadata.json (features, hiperparametros)  
  data/  
    raw/<ciudad>/ # listings.csv.gz de Inside Airbnb (no versionado, pesa demasiado)  
    processed/<ciudad>/ # csv limpios y con features, generados por los notebooks  
  tfm/ # memoria.html/pdf y los tres anexos (variables, EDA, codigo)
```

Dependencias de la aplicación

Versiones exactas fijadas en app/requirements.txt:

Paquete	Versión
streamlit	1.50.0
pandas	2.3.3
numpy	2.0.2
scikit-learn	1.6.1
joblib	1.5.3
pydeck	0.9.3
shap	0.49.1
altair	5.5.0
folium	0.20.0

streamlit-folium	0.25.3
branca	0.8.2

Aplicación FairNight

app/app.py

Interfaz Streamlit completa: formulario de descripción del anuncio, predicción, explicación (SHAP), comparables y mapa.

```
import base64
import html
import json
import math
import threading
from pathlib import Path

import altair as alt
import branca.colormap as cm
import folium
import joblib
import numpy as np
import pandas as pd
import pydeck as pdk
import shap
import streamlit as st
from streamlit_folium import st_folium

import comparables
import features

st.set_page_config(page_title="Recomendador de precio Airbnb", layout="wide")

PROJECT_DIR = Path(__file__).resolve().parent.parent
APP_DIR = Path(__file__).resolve().parent

@st.cache_data(show_spinner=False)
def _count_listings(city):
    # usecols=["id"]: algunos campos del CSV (name, picture_url) llevan saltos de
    # línea dentro de comillas, así que contar líneas a mano infla el resultado.
    return len(pd.read_csv(PROJECT_DIR / "data" / "processed" / city / "listings_full_clean.csv",
        usecols=["id"]))

# Con True, cada card de zona disponible añade una línea con su MAE bajo el nº de
# anuncios ("Error típico ±X €/noche"). Con False (por defecto) esa línea no se
# pinta: un único punto de control en vez de comprobaciones repartidas por el código.
MOSTRAR_ERROR_EN_CARDS = False

ZONES_DIR = APP_DIR / "assets" / "zonas"
BRANDING_DIR = APP_DIR / "assets" / "branding"

@st.cache_data(show_spinner=False)
def _welcome_logo_base64():
    """Logotipo grande (icono + FairNight) para la pantalla de bienvenida, cacheado
    para no releer/recodificar el archivo del disco en cada rerun."""
    path = BRANDING_DIR / "logo_lockup.png"
    return base64.b64encode(path.read_bytes()).decode("ascii")

@st.cache_data(show_spinner=False)
def _zone_photo_base64(city):
    """WebP de la zona en base64, cacheado por ciudad para no releer/recodificar el
```

```

archivo del disco en cada rerun. None si la foto no existe todavía (la zona se
trata como 'próxima', ver load_zone_registry)."""
path = ZONES_DIR / f"{city}.webp"
if not path.exists():
    return None
return base64.b64encode(path.read_bytes()).decode("ascii")

@st.cache_data(show_spinner=False)
def load_zone_registry():
    """Registro de zonas para la pantalla de bienvenida: nombre y nº de anuncios se leen
    de CITY_LABELS y del CSV real de cada zona, nunca escritos a mano aquí. Una zona pasa
    a 'próxima' (foto en gris, botón desactivado) si le falta el modelo, los datos
    procesados o la foto – así una zona a medio desplegar nunca se muestra como si ya
    funcionara. Devuelve también qué le falta a cada zona incompleta, para poder avisar."""
    zones = []
    missing_report = []
    for city in features.CITIES:
        label = features.CITY_LABELS[city]
        model_path = PROJECT_DIR / "models" / city / "model_metadata.json"
        data_path = PROJECT_DIR / "data" / "processed" / city / "listings_full_clean.csv"
        photo_b64 = _zone_photo_base64(city)

        missing = []
        if not model_path.exists():
            missing.append("modelo")
        if not data_path.exists():
            missing.append("datos")
        if photo_b64 is None:
            missing.append("foto")
        available = not missing
        if missing:
            missing_report.append(f"{label}: falta {'', '.join(missing)}")

        n_listings, mae = None, None
        if available:
            n_listings = _count_listings(city)
            with open(model_path, encoding="utf-8") as f:
                mae = json.load(f)["test_metrics"]["MAE"]

        zones.append({
            "id": city, "label": label, "photo_b64": photo_b64,
            "available": available, "n_listings": n_listings, "mae": mae,
        })

    if missing_report:
        # "avisame de qué zona falta" (petición explícita): consola, no la propia UI –
        # es diagnóstico para quien despliega la app, no algo que deba ver el usuario final.
        print("[zonas] Zonas incompletas, tratadas como 'próxima':", flush=True)
        for line in missing_report:
            print(f" - {line}", flush=True)

    return zones, missing_report

def _inject_welcome_css():
    st.markdown(
        """<style>
        /* Todo lo de aquí abajo, más el h1/subtítulo, apretado para que las 8 cards quepan
        sin scroll en una pantalla normal (pedido explícito tras verlo cortado): menos
        padding arriba, título y subtítulo más pegados, foto menos alta (2.4:1 en vez de
        3:2) y menos aire dentro de cada card. */
        .block-container {padding-top: 1.2rem; padding-bottom: 0.5rem;}
        /* El PNG del logo trae su propio fondo casi blanco (no transparente) con una
        textura irregular tipo viñeteado: intentar recortarlo a transparente dejaba
        restos visibles del patrón. En vez de eso, la tarjeta usa ese mismo tono cálido
        (~#F8F7F5, la media de las esquinas del PNG) para que no haya costura visible
        entre la imagen y el fondo de la tarjeta. */
        .welcome-logo { text-align: center; margin: 48px 0 10px 0; }
        .welcome-logo .logo-card {
            display: inline-block; background: #F8F7F5; border-radius: 16px;
            padding: 14px 30px; box-shadow: 0 1px 3px rgba(0,0,0,0.06);
        }
        """
    )

```

```

.welcome-logo img { height: 64px; display: block; }
h1 { font-size: 2rem !important; margin-bottom: 2px !important; }
/* Única leyenda de esta pantalla (el subtítulo bajo el título): sin este margen
extra, sobraba un hueco de sobra antes de la primera fila de cards. Tamaño subido
(pedido explícito) respecto al resto de captions de la app. */
[data-testid="stCaptionContainer"] { margin-bottom: 4px; }
[data-testid="stCaptionContainer"] p { font-size: 1.05rem !important; }

.st-key-zone_grid [data-testid="stHorizontalBlock"] {
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr));
  gap: 12px;
}
/* Streamlit fija flex-basis/width por columna en línea; con display:grid en el
padre eso solo estorba (columnas descuadradas). Se anula para que el ancho lo
decida el grid-template-columns de arriba, igual en las 8 celdas. */
.st-key-zone_grid [data-testid="stHorizontalBlock"] > [data-testid="stColumn"] {
  width: 100% !important;
  min-width: 0 !important;
  flex: none !important;
  align-self: stretch !important;
}
.st-key-zone_grid [data-testid="stColumn"] > div,
.st-key-zone_grid [data-testid="stColumn"] [data-testid="stLayoutWrapper"],
.st-key-zone_grid [data-testid="stColumn"] [data-testid="stVerticalBlock"] {
  height: 100%;
}
@media (max-width: 1100px) {
  .st-key-zone_grid [data-testid="stHorizontalBlock"] { grid-template-columns:
    repeat(2, minmax(0, 1fr)); }
}
@media (max-width: 620px) {
  .st-key-zone_grid [data-testid="stHorizontalBlock"] { grid-template-columns: 1fr; }
}

.st-key-zone_grid [class*="st-key-welcome_card_"] {
  border: 1px solid #E3E0DA;
  border-radius: 14px;
  overflow: hidden;
  padding: 0 !important;
  height: 100%;
}
.st-key-zone_grid [class*="st-key-welcome_card_"] > div {
  gap: 0 !important;
  height: 100%;
}
.zc-foto { position: relative; aspect-ratio: 2.4 / 1; }
.zc-foto img { width: 100%; height: 100%; object-fit: cover; display: block; }
.zc-foto.zc-proxima img, .zc-foto.zc-proxima .zc-placeholder { filter: grayscale(1);
opacity: .55; }
.zc-placeholder { width: 100%; height: 100%; background: #EDEAE4; }
.zc-cubierta {
  position: absolute; left: 0; right: 0; bottom: 0;
  display: flex; align-items: flex-end;
  padding: 8px 12px;
  background: rgba(28,28,26,.62);
}
.zc-nombre { color: #fff; font-size: 17px; font-weight: 600; letter-spacing: -.01em; }
.zc-cuerpo-texto { padding: 8px 14px 0; }
.zc-badge {
  display: inline-block; margin: 0 0 4px; padding: 2px 8px;
  background: #EDEAE4; color: #6B6A66; border-radius: 6px; font-size: 12px;
  font-weight: 600;
}
.zc-dato { margin: 0; font-size: 13px; line-height: 1.4; color: #6B6A66; }
.zc-dato strong { color: #2C2C2A; font-weight: 600; }
.zc-dato-sec { margin: 4px 0 0; font-size: 12px; color: #8B8A85; }

.st-key-zone_grid [class*="st-key-welcome_card_"] [data-testid="stButton"] {
  padding: 8px 14px 10px;
  margin-top: auto;
}
.st-key-zone_grid [class*="st-key-welcome_card_"] [data-testid="stButton"] button {

```

```

        border-radius: 9px;
        font-size: 15px;
        font-weight: 600;
    }
    .st-key-zone_grid [class*="st-key-welcome_card_"] [data-testid="stButton"]
button:disabled {
    background: #E8E5DF;
    color: #A8A6A0;
}

    .st-key-zone_grid [class*="st-key-welcome_card_vacia"] {
        border: 1px dashed #D6D2CA;
        background: #FCFBF9;
        border-radius: 14px;
        padding: 16px 18px !important;
        display: flex; align-items: center; justify-content: center;
        height: 100%;
    }
    .zc-vacia-titulo { margin: 0 0 6px; font-size: 15px; font-weight: 600; color: #2C2C2A; }
    .zc-vacia-texto { margin: 0; font-size: 13px; color: #6B6A66; line-height: 1.55; }
</style>""",
    unsafe_allow_html=True,
)

def render_welcome_screen():
    """Pantalla de elección de zona, solo la primera vez (sin 'active_city' en
    session_state). No es una página de la nav: se pinta antes de cualquier otra
    cosa y corta la ejecución con st.stop(), así que el resto del script (selector
    de la sidebar, páginas, features.set_city...) no llega a ejecutarse hasta que
    el usuario elige."""
    _inject_welcome_css()
    st.markdown(
        f'<div class="welcome-logo"><div class="logo-card">'
        f'</div></div>',
        unsafe_allow_html=True,
    )
    st.title("Recomendador de precio · Airbnb")
    st.caption(
        "Descubre a qué precio publicar tu anuncio, comparándolo con miles de anuncios reales. "
        "Elige tu zona: cada una tiene su propio modelo, entrenado solo con datos de esa zona."
    )

    zones, _ = load_zone_registry()

    with st.container(key="zone_grid"):
        cols = st.columns(len(zones) + 1)
        for col, zone in zip(cols, zones):
            city = zone["id"]
            label_safe = html.escape(zone["label"])
            with col:
                with st.container(key=f"welcome_card_{city}"):
                    if zone["available"]:
                        st.markdown(
                            f'<div class="zc-foto"><img'
                            src="data:image/webp;base64,{zone["photo_b64"]}" '
                            f'alt="{label_safe}"><div class="zc-cubierta">'
                            f'<span class="zc-nombre">{label_safe}</span></div></div>',
                            unsafe_allow_html=True,
                        )
                        n_text = f"{zone['n_listings']:,}".replace(",", ".")
                        dato_sec = (
                            f'<p class="zc-dato-sec">Error típico ±{zone["mae"]:.0f} €/noche</p>'
                            if MOSTRAR_ERROR_EN_CARDS else ""
                        )
                        st.markdown(
                            f'<div class="zc-cuerpo-texto">'
                            f'<p class="zc-dato">Modelo entrenado con <strong>{n_text}</strong>'
                            anuncios reales</p>'
                            f'{dato_sec}</div>',
                            unsafe_allow_html=True,
                        )
                    if st.button("Empezar", key=f"welcome_btn_{city}", type="primary",

```

```

        width="stretch"):
            st.session_state["active_city"] = city
            st.rerun()
    else:
        photo_html = (
            f''
            if zone["photo_b64"] else '<div class="zc-placeholder"></div>'
        )
        st.markdown(
            f'<div class="zc-foto zc-proxima">{photo_html}<div
            class="zc-cubierta">'
            f'<span class="zc-nombre">{label_safe}</span></div></div>',
            unsafe_allow_html=True,
        )
        st.markdown(
            '<div class="zc-cuerpo-texto"><span
            class="zc-badge">Próximamente</span>'
            '<p class="zc-dato">Aún no hay modelo entrenado para esta
            zona.</p></div>',
            unsafe_allow_html=True,
        )
        st.button("Empezar", key=f"welcome_btn_{city}", disabled=True,
            width="stretch")

    with cols[-1]:
        with st.container(key="welcome_card_vacia"):
            st.markdown(
                '<div><p class="zc-vacia-titulo">¿No está tu zona?</p>'
                '<p class="zc-vacia-texto">Escríbeme y la añado al siguiente lote de
                modelos.</p></div>',
                unsafe_allow_html=True,
            )

if "active_city" not in st.session_state:
    render_welcome_screen()
    st.stop()

st.logo(
    str(APP_DIR / "assets" / "branding" / "logo_lockup.png"),
    size="large",
    icon_image=str(APP_DIR / "assets" / "branding" / "logo_icon.png"),
)

# Selector de ciudad: se coloca aquí, antes que cualquier otro contenido, para que
# Streamlit lo pinte arriba del todo en la barra lateral. El menú de navegación de
# páginas se construye con position="hidden" al final de este fichero y se pinta a
# mano con st.sidebar.page_link() – st.navigation() con position="sidebar" siempre
# se ancla arriba del todo de la sidebar, por delante de cualquier otro st.sidebar.*
# por muy pronto que se llame, así que es la única forma de dejar este selector antes.
_city_options = features.CITIES
_city_labels = [features.CITY_LABELS[c] for c in _city_options]
# El índice inicial se deriva de active_city (no de un session_state["city_selector"])
# preasignado a mano desde la pantalla de bienvenida: asignar directamente el valor de
# un widget en session_state ANTES de que exista funciona en el primer render, pero no
# sobrevive de forma fiable a la primera navegación entre páginas con st.page_link (el
# selector volvía a Barcelona, la primera opción, justo al navegar a la página siguiente
# tras elegir ciudad en la bienvenida). Con index=, Streamlit gestiona el valor inicial
# por su cuenta y sí es estable frente a la navegación.
_default_city = st.session_state.get("active_city", _city_options[0])
_default_city_index = _city_options.index(_default_city) if _default_city in _city_options else 0
with st.sidebar.container(border=True):
    _selected_label = st.selectbox(
        ":material/location-city: Zona", _city_labels, index=_default_city_index,
        key="city_selector",
    )
_selected_city = _city_options[_city_labels.index(_selected_label)]

if st.session_state.get("active_city") != _selected_city:
    # Cambio de ciudad: el formulario confirmado o en curso de la ciudad anterior
    # (barrio, coordenadas...) no tiene sentido en la nueva, así que se limpia en
    # vez de arrastrar un estado que puede no existir aquí (p.ej. un barrio de

```

```

# Barcelona no es una clave válida en NEIGHBOURHOOD_STATS de Madrid).
for _k in ["confirmed_inputs", "live_inputs", "coord_lat", "coord_lon",
"coord_neighbourhood"]:
    st.session_state.pop(_k, None)
# El simulador "¿Qué pasaría si...?" también guarda valores por ciudad
# (whatif_val_district="Eixample", whatif_val_neighbourhood...) que pueden
# no existir en la ciudad nueva, así que se limpian igual que el resto.
# Ídem para el estado propio de la pantalla Comparar (alcance elegido,
# panel de Ajustar abierto/cerrado, fila o punto fijado en fases futuras...).
for _k in [k for k in st.session_state if k.startswith("whatif_") or
k.startswith("comparar_")]:
    st.session_state.pop(_k, None)
st.session_state["active_city"] = _selected_city

features.set_city(_selected_city)

from features import (
    CITY,
    CITY_CENTER,
    DISTRICTS,
    FEATURE_COLS,
    HAS_NEIGHBOURHOOD_LEVEL,
    METADATA,
    NEIGHBOURHOODS_BY_DISTRICT,
    NEIGHBOURHOOD_STATS,
    PROPERTY_TYPES,
    ROOM_TYPES,
    build_features,
    confidence_flags,
    host_size_tier,
    pretty_feature_name,
)

CITY_LABEL = features.CITY_LABELS[CITY]

ROOM_TYPE_ICONS = {
    "Entire home/apt": ":material/home: Vivienda entera",
    "Private room": ":material/door_front: Habitación privada",
    "Shared room": ":material/group: Habitación compartida",
    "Hotel room": ":material/hotel: Habitación de hotel",
}
ROOM_TYPE_LABELS = {v: k for k, v in ROOM_TYPE_ICONS.items()}

REVIEW_DEFAULTS = {
    "number_of_reviews": 10,
    "number_of_reviews_ltm": 5,
    "reviews_per_month": 1.0,
    "review_scores_rating": 4.8,
    "listing_age_days": 365,
    "days_since_last_review": 30,
    "estimated_occupancy_l365d": 100,
}

WHATIF_LABELS = {
    "district": "Distrito",
    "neighbourhood": "Barrio",
    "room_type": "Tipo de alojamiento",
    "property_type": "Tipo de propiedad",
    "accommodates": "Huéspedes",
    "bedrooms": "Dormitorios",
    "beds": "Camas",
    "bathrooms": "Baños",
    "minimum_nights": "Estancia mínima (noches)",
    "maximum_nights": "Estancia máxima (noches)",
    "availability_365": "Días disponibles al año",
    "has_license": "Tiene licencia turística",
    "host_is_superhost": "Soy superhost",
    "host_has_profile_pic": "Tengo foto de perfil",
    "host_identity_verified": "Identidad verificada",
    "calculated_host_listings_count": "Anuncios que gestiono",
    "host_tenure_years": "Años como anfitrión",
    "host_user_tenure_years": "Años con cuenta en la plataforma",
    "is_new_listing": "Es un anuncio nuevo (sin reseñas)",

```

```

"number_of_reviews": "Número de reseñas",
"number_of_reviews_ltm": "Reseñas último año",
"reviews_per_month": "Reseñas al mes",
"review_scores_rating": "Valoración media",
"listing_age_days": "Días desde la publicación",
"days_since_last_review": "Días desde la última reseña",
"estimated_occupancy_l365d": "Noches ocupadas último año",
"has_ac": "Aire acondicionado",
"has_pool": "Piscina",
"has_dishwasher": "Lavavajillas",
"has_sea_view": "Vistas al mar",
"n_amenities": "Nº de amenities",
"n_nearby_150m": "Densidad de anuncios cercanos (150m)",
}
if not HAS_NEIGHBOURHOOD_LEVEL:
    # Sin nivel de barrio (ver features.HAS_NEIGHBOURHOOD_LEVEL): sería el mismo dato
    # que "Distrito" en el simulador, así que no se ofrece como variable aparte.
    WHATIF_LABELS = {k: v for k, v in WHATIF_LABELS.items() if k != "neighbourhood"}
# Igual que en PDP_GROUPS: las claves que no son columnas directas del modelo
# (district/room_type/property_type se traducen a sus one-hot; is_new_listing es
# un toggle de UI, no una feature) se mantienen siempre; el resto solo si esa
# columna existe en el modelo de la ciudad activa (p.ej. has_dishwasher no existe
# en Mallorca, has_sea_view solo existe en Mallorca).
_WHATIF_NON_FEATURE_KEYS = {"district", "neighbourhood", "room_type", "property_type",
"is_new_listing"}
WHATIF_LABELS = {
    k: v for k, v in WHATIF_LABELS.items()
    if k in _WHATIF_NON_FEATURE_KEYS or k in FEATURE_COLS
}

WHATIF_NUMERIC_VALUES = {
    "accommodates": [1, 2, 4, 6, 8, 10, 12, 14, 16],
    "bedrooms": [0, 1, 2, 3, 4, 6, 8, 10],
    "beds": [1, 2, 3, 4, 6, 8, 10],
    "bathrooms": [0.5, 1, 1.5, 2, 3, 4, 6, 8],
    "minimum_nights": [1, 2, 3, 7, 14, 30, 60, 90, 180, 365],
    "maximum_nights": [7, 30, 90, 180, 365, 730, 1125],
    "availability_365": [0, 30, 90, 180, 270, 365],
    "calculated_host_listings_count": [1, 2, 5, 10, 25, 50, 100, 250, 500],
    "host_tenure_years": [0, 0.5, 1, 2, 5, 10, 15, 20],
    "host_user_tenure_years": [0.5, 1, 2, 5, 10, 15, 20, 25],
    "number_of_reviews": [0, 1, 5, 10, 25, 50, 100, 250],
    "number_of_reviews_ltm": [0, 1, 5, 10, 25, 50],
    "reviews_per_month": [0, 0.5, 1, 2, 4, 8],
    "review_scores_rating": [3.0, 3.5, 4.0, 4.5, 4.8, 5.0],
    "listing_age_days": [30, 90, 180, 365, 730, 1460, 2920],
    "days_since_last_review": [0, 7, 30, 90, 180, 365],
    "estimated_occupancy_l365d": [0, 10, 30, 60, 100, 150, 250, 365],
    "neighbourhood_price_encoded": [80, 120, 160, 200, 250, 300, 350, 400, 450],
    "distance_to_center_km": [0, 0.5, 1, 2, 3, 4, 5, 6, 8],
    "host_entire_homes_ratio": [0, 0.25, 0.5, 0.75, 1.0],
    "n_amenities": [0, 10, 20, 30, 40, 50, 60, 84],
    "n_nearby_150m": [0, 25, 50, 75, 100, 125, 150, 165],
}
# Los 4 de arriba (precio, distancia, amenities, densidad) están calibrados a la escala
# de Barcelona; en Madrid varios se quedan cortos frente al rango real de los datos (ver
# _CATALOG_RANGES) y el gráfico de qué-pasa-si nunca llega a explorar esa parte del rango.
_WHATIF_CITY_OVERRIDES = {
    "barcelona": {
        "neighbourhood_price_encoded": [80, 120, 160, 200, 250, 300, 350, 400, 450],
        "distance_to_center_km": [0, 0.5, 1, 2, 3, 4, 5, 6, 8],
        "n_amenities": [0, 10, 20, 30, 40, 50, 60, 84],
        "n_nearby_150m": [0, 25, 50, 75, 100, 125, 150, 165],
    },
    "madrid": {
        "neighbourhood_price_encoded": [80, 100, 120, 140, 160, 180, 200, 225, 253],
        "distance_to_center_km": [0, 1, 2, 3, 4, 6, 8, 10, 12, 15],
        "n_amenities": [0, 10, 20, 30, 40, 55, 70, 85, 96],
        "n_nearby_150m": [0, 40, 80, 120, 160, 200, 250, 300, 320],
    },
    "valencia": {
        "neighbourhood_price_encoded": [100, 120, 140, 160, 180, 200, 220, 241],
        "distance_to_center_km": [0, 1, 2, 3, 4, 6, 8, 12, 18],
    },
}

```

```

        "n_amenities": [0, 10, 20, 30, 40, 50, 65, 80, 92],
        "n_nearby_150m": [0, 20, 40, 60, 80, 100, 120, 140, 163],
    },
    "malaga": {
        "neighbourhood_price_encoded": [150, 180, 210, 240, 270, 300, 330, 360, 375],
        "distance_to_center_km": [0, 1, 2, 3, 5, 7, 10, 14, 19],
        "n_amenities": [0, 10, 20, 30, 40, 50, 65, 80, 95],
        "n_nearby_150m": [0, 50, 100, 150, 200, 300, 400, 500, 533],
    },
    "sevilla": {
        "neighbourhood_price_encoded": [95, 115, 135, 155, 170, 185, 200, 220],
        "distance_to_center_km": [0, 0.5, 1, 1.5, 2, 3, 4, 6, 9.6],
        "n_amenities": [0, 10, 20, 30, 40, 50, 65, 80, 90],
        "n_nearby_150m": [0, 30, 60, 90, 120, 150, 190, 230, 266],
    },
    "mallorca": {
        "neighbourhood_price_encoded": [400, 450, 500, 550, 600, 650, 750, 900, 1090],
        "distance_to_center_km": [0, 5, 12, 20, 30, 40, 50, 60, 72],
        "n_amenities": [0, 10, 20, 30, 40, 50, 65, 80, 98],
        "n_nearby_150m": [0, 1, 2, 4, 7, 13, 23, 35, 53],
    },
    "euskadi": {
        "neighbourhood_price_encoded": [150, 180, 210, 240, 270, 300, 350, 450, 556],
        "distance_to_center_km": [0, 1, 2, 5, 10, 17, 25, 33, 42],
        "n_amenities": [0, 10, 20, 30, 40, 50, 60, 75, 93],
        "n_nearby_150m": [0, 2, 5, 10, 20, 35, 55, 80, 137],
    },
}
WHATIF_NUMERIC_VALUES.update(_WHATIF_CITY_OVERRIDES.get(CITY,
_WHATIF_CITY_OVERRIDES["barcelona"]))

PDP_GROUPS = {
    "Estancia mínima": {"kind": "numeric", "cols": ["minimum_nights"], "sample_key":
    "minimum_nights"},
    "Huéspedes": {"kind": "numeric", "cols": ["accommodates"], "sample_key": "accommodates"},
    "Tipo de alojamiento": {"kind": "onehot", "prefix": "room_type_", "options": ROOM_TYPES},
    "Baños": {"kind": "numeric", "cols": ["bathrooms", "bathrooms_log"], "sample_key":
    "bathrooms"},
    "Dormitorios": {"kind": "numeric", "cols": ["bedrooms", "bedrooms_log"], "sample_key":
    "bedrooms"},
    "Anuncios que gestiona": {"kind": "host_scale"},
    "Valoración media": {"kind": "numeric", "cols": ["review_scores_rating"], "sample_key":
    "review_scores_rating"},
    "Días disponibles al año": {"kind": "numeric", "cols": ["availability_365"], "sample_key":
    "availability_365"},
    "Nivel de precios del barrio": {"kind": "numeric", "cols": ["neighbourhood_price_encoded"],
    "sample_key": "neighbourhood_price_encoded"},
    "Tipo de propiedad": {"kind": "onehot", "prefix": "property_type_", "options":
    PROPERTY_TYPES},
    "Años como anfitrión": {"kind": "numeric", "cols": ["host_tenure_years"], "sample_key":
    "host_tenure_years"},
    "Años con cuenta en la plataforma": {"kind": "numeric", "cols": ["host_user_tenure_years"],
    "sample_key": "host_user_tenure_years"},
    "Días desde la última reseña": {"kind": "numeric", "cols": ["days_since_last_review"],
    "sample_key": "days_since_last_review"},
    "Estancia máxima": {"kind": "numeric", "cols": ["maximum_nights"], "sample_key":
    "maximum_nights"},
    "Licencia turística": {"kind": "boolean", "col": "has_license"},
    "Distancia al centro": {"kind": "numeric", "cols": ["distance_to_center_km"], "sample_key":
    "distance_to_center_km"},
    "Reseñas al mes": {"kind": "numeric", "cols": ["reviews_per_month", "reviews_per_month_log"],
    "sample_key": "reviews_per_month"},
    "Antigüedad del anuncio": {"kind": "numeric", "cols": ["listing_age_days"], "sample_key":
    "listing_age_days"},
    "Superhost": {"kind": "boolean", "col": "host_is_superhost"},
    "Número de reseñas": {"kind": "numeric", "cols": ["number_of_reviews",
    "number_of_reviews_log"], "sample_key": "number_of_reviews"},
    "Reseñas último año": {"kind": "numeric", "cols": ["number_of_reviews_ltm",
    "number_of_reviews_ltm_log"], "sample_key": "number_of_reviews_ltm"},
    "Noches ocupadas último año": {"kind": "numeric", "cols": ["estimated_occupancy_l365d"],
    "sample_key": "estimated_occupancy_l365d"},
    "Camas": {"kind": "numeric", "cols": ["beds", "beds_log"], "sample_key": "beds"},
    "Distrito": {"kind": "onehot", "prefix": "district_", "options": DISTRICTS},

```

```

"Identidad verificada": {"kind": "boolean", "col": "host_identity_verified"},
"Tiene reseñas": {"kind": "boolean", "col": "has_reviews"},
"Foto de perfil": {"kind": "boolean", "col": "host_has_profile_pic"},
"% de vivienda entera del anfitrión": {"kind": "numeric", "cols":
["host_entire_homes_ratio"], "sample_key": "host_entire_homes_ratio"},
"Aire acondicionado": {"kind": "boolean", "col": "has_ac"},
"Piscina": {"kind": "boolean", "col": "has_pool"},
"Lavavajillas": {"kind": "boolean", "col": "has_dishwasher"},
"Vistas al mar": {"kind": "boolean", "col": "has_sea_view"},
"Nº de amenities": {"kind": "numeric", "cols": ["n_amenities"], "sample_key": "n_amenities"},
"Densidad de anuncios cercanos": {"kind": "numeric", "cols": ["n_nearby_150m"], "sample_key":
"n_nearby_150m"},
}
# Las amenities concretas varían de una ciudad a otra (p.ej. Mallorca tiene
# "has_sea_view" en vez de "has_dishwasher", ver 02_feature_engineering.ipynb de
# Mallorca, sección 9): se descartan aquí las entradas numeric/boolean cuya
# columna no exista en el modelo de la ciudad activa, para no ofrecer una
# variable que no tendría ningún efecto sobre la predicción. Las agrupadas en
# one-hot (distrito, tipo...) ya se filtran solas al construirse desde FEATURE_COLS.
PDP_GROUPS = {
    label: spec for label, spec in PDP_GROUPS.items()
    if spec["kind"] == "onehot"
    or spec["kind"] == "host_scale"
    or (spec["kind"] == "numeric" and spec["sample_key"] in FEATURE_COLS)
    or (spec["kind"] == "boolean" and spec["col"] in FEATURE_COLS)
}

# Para el resumen SHAP: solo variables con un valor propio y continuo (numéricas y
# booleanas). Las agrupadas en one-hot (distrito, tipo...) no tienen un "valor" único
# que colorear y ya se ven en el ranking de importancia y en el PDP categórico.
SHAP_SUMMARY_GROUPS = {
    label: spec for label, spec in PDP_GROUPS.items() if spec["kind"] in ("numeric", "boolean")
}

# De cada una de las 66 columnas crudas del modelo a la variable "de negocio" a la que
# pertenece (p.ej. las 10 columnas district_* se agrupan todas bajo "Distrito"), para no
# mostrar nunca una categoría que el usuario no ha elegido (Distrito: Gràcia, etc.).
_HOST_SCALE_COLS = [
    "calculated_host_listings_count", "calculated_host_listings_count_log",
    "host_size_individual", "host_size_pequeno", "host_size_mediano", "host_size_grande",
]
FEATURE_TO_GROUP = {}
for _label, _spec in PDP_GROUPS.items():
    if _spec["kind"] == "numeric":
        for _c in _spec["cols"]:
            FEATURE_TO_GROUP[_c] = _label
    elif _spec["kind"] == "boolean":
        FEATURE_TO_GROUP[_spec["col"]] = _label
    elif _spec["kind"] == "onehot":
        for _c in FEATURE_COLS:
            if _c.startswith(_spec["prefix"]):
                FEATURE_TO_GROUP[_c] = _label
    elif _spec["kind"] == "host_scale":
        for _c in _HOST_SCALE_COLS:
            FEATURE_TO_GROUP[_c] = _label
for _c in FEATURE_COLS:
    FEATURE_TO_GROUP.setdefault(_c, pretty_feature_name(_c))

def pdp_apply(Xc, group_label, value):
    spec = PDP_GROUPS[group_label]
    kind = spec["kind"]
    if kind == "numeric":
        for col in spec["cols"]:
            Xc[col] = np.log1p(value) if col.endswith("_log") else value
    elif kind == "boolean":
        Xc[spec["col"]] = int(value)
    elif kind == "onehot":
        onehot_cols = [c for c in Xc.columns if c.startswith(spec["prefix"])]
        Xc[onehot_cols] = 0
        col = f"{spec['prefix']}{value}"
        if col in Xc.columns:

```

```

        Xc[col] = 1
    elif kind == "host_scale":
        Xc["calculated_host_listings_count"] = value
        Xc["calculated_host_listings_count_log"] = np.log1p(value)
        tier = host_size_tier(value)
        for t in ["individual", "pequeno", "mediano", "grande"]:
            Xc[f"host_size_{t}"] = 1 if t == tier else 0
    else:
        raise ValueError(f"kind desconocido: {kind}")
    return Xc

def pdp_axis_values(group_label):
    spec = PDP_GROUPS[group_label]
    kind = spec["kind"]
    if kind == "numeric":
        return WHATIF_NUMERIC_VALUES[spec["sample_key"]], "numeric"
    if kind == "host_scale":
        return WHATIF_NUMERIC_VALUES["calculated_host_listings_count"], "numeric"
    if kind == "boolean":
        return [False, True], "categorical"
    if kind == "onehot":
        return spec["options"], "categorical"
    raise ValueError(f"kind desconocido: {kind}")

def pdp_label(value):
    if value is True:
        return "Sí"
    if value is False:
        return "No"
    return str(value)

def pdp_real_values(X_base, group_label):
    """Valores reales (no los puntos curados de muestreo) de una variable numérica
    en el test set, para dibujar su histograma de soporte bajo el PDP."""
    spec = PDP_GROUPS[group_label]
    if spec["kind"] == "numeric":
        return X_base[spec["sample_key"]]
    if spec["kind"] == "host_scale":
        return X_base["calculated_host_listings_count"]
    raise ValueError(f"pdp_real_values no aplica a variables categóricas: {group_label}")

def pdp_correlation_column(group_label):
    """Columna cruda para calcular correlación real, si la variable tiene una sola
    (numérica, booleana o el conteo de anuncios del anfitrión). None para las
    agrupadas en one-hot (distrito, tipo...), que no tienen un único valor a correlacionar."""
    spec = PDP_GROUPS[group_label]
    if spec["kind"] == "numeric":
        return spec["sample_key"]
    if spec["kind"] == "boolean":
        return spec["col"]
    if spec["kind"] == "host_scale":
        return "calculated_host_listings_count"
    return None

def pdp_pair_correlation(X_base, label_a, label_b):
    col_a = pdp_correlation_column(label_a)
    col_b = pdp_correlation_column(label_b)
    if col_a is None or col_b is None:
        return np.nan
    return float(X_base[col_a].astype(float).corr(X_base[col_b].astype(float)))

def interaction_diagnosis(indice, corr):
    """Explicación en una frase de si conviene fiarse del índice de interacción de una
    pareja, cruzándolo con la correlación real entre ambas variables."""
    if indice < 0.10:
        return "Apenas interactúan: cada variable actúa casi de forma independiente."
    nivel = "fuerte" if indice > 0.25 else "moderada"

```

```

if pd.isna(corr):
    return f"Interacción {nivel}, pero no se puede comprobar si es artefacto: alguna es una
    variable agrupada."
if abs(corr) >= 0.5:
    return f"Ambigua: correlación real alta (r={corr:.2f}) entre ambas, gran parte del índice
    puede venir de combinaciones poco realistas."
if abs(corr) >= 0.25:
    return f"Parcialmente ambigua: hay algo de correlación real (r={corr:.2f}) que puede
    inflar el índice."
return f"Interacción {nivel} y probablemente genuina: poca correlación real entre ambas
(r={corr:.2f})."

# Hipótesis de negocio curadas a mano para el top-15 de interaction_scores.json: por qué
# tendría sentido de negocio que estas dos variables se combinen, más allá del índice y la
# correlación. Cuando no encuentro una historia convincente, lo digo explícitamente en vez
# de forzar una explicación (puede ser simplemente ruido del modelo en zonas con pocos datos).
INTERACTION_BUSINESS_NOTES = {
    frozenset({"Anuncios que gestiona", "Días disponibles al año"}):
        "Los anfitriones profesionales (muchos anuncios) suelen mantener sus pisos disponibles
        casi todo el "
        "año como negocio a tiempo completo; uno con un solo piso puede tenerlo disponible solo
        parte del año.",
    frozenset({"Años como anfitrión", "Años con cuenta en la plataforma"}):
        "No hay una historia de negocio real detrás: son casi la misma variable (quien lleva más
        tiempo como "
        "anfitrión también lleva más tiempo con cuenta), así que el índice alto es sobre todo
        redundancia.",
    frozenset({"Baños", "Reseñas último año"}):
        "No encuentro una razón de negocio clara. Con tan poca correlación entre ambas, y pocos
        anuncios reales "
        "con muchos baños y muchas reseñas recientes a la vez, esto puede ser ruido del modelo,
        no un patrón real.",
    frozenset({"Estancia máxima", "Antigüedad del anuncio"}):
        "Los anfitriones con anuncios más antiguos han tenido más tiempo para ajustar su política
        de estancia "
        "máxima a lo que realmente funciona; en anuncios nuevos ese valor suele ser más
        genérico.",
    frozenset({"Estancia máxima", "Reseñas al mes"}):
        "Una estancia máxima corta favorece más rotación de huéspedes y por tanto más reseñas por
        mes: el "
        "efecto de la estancia máxima sobre el precio podría depender de cuánta rotación real
        tiene el anuncio.",
    frozenset({"Tipo de alojamiento", "Baños"}):
        "Tiene sentido de negocio: más baños importa mucho en una vivienda entera (todos los
        huéspedes los "
        "disfrutan), pero apenas cambia la experiencia en una habitación privada o compartida.",
    frozenset({"Años con cuenta en la plataforma", "Distancia al centro"}):
        "No encuentro una historia de negocio convincente; con correlación casi nula, si hay algo
        real aquí "
        "probablemente sea ruido de combinaciones poco pobladas (cuentas muy antiguas lejos del
        centro).",
    frozenset({"Años con cuenta en la plataforma", "Estancia máxima"}):
        "Especulativo: podría reflejar que Airbnb ha cambiado sus políticas y valores por defecto
        de estancia "
        "máxima a lo largo de los años, y los anfitriones más veteranos configuraron el suyo en
        otro contexto.",
    frozenset({"Años con cuenta en la plataforma", "Superhost"}):
        "Tiene sentido: ser Superhost exige historial demostrado, así que muchos años en la
        plataforma más el "
        "sello de Superhost transmiten más confianza conjunta que cualquiera de las dos por
        separado.",
    frozenset({"Días disponibles al año", "Distancia al centro"}):
        "Plausible: en el centro, alta disponibilidad todo el año suele señalar un piso turístico
        profesional "
        "(más caro); en la periferia, alta disponibilidad puede indicar simplemente poca
        demanda.",
    frozenset({"Antigüedad del anuncio", "Distrito"}):
        "Los distritos turísticos más consolidados llevan más tiempo con oferta activa en Airbnb
        que zonas más "
        "residenciales o de incorporación más reciente al mercado.",
    frozenset({"Estancia mínima", "Huéspedes"}):
        "Coherente con lo visto en el mapa: los anuncios para muchos huéspedes suelen ser

```

alquiler de temporada "

"(estancias largas), mientras que los pequeños son turismo de estancia corta.",

frozenset({"Huéspedes", "Dormitorios"}):

"Poca sorpresa de negocio: más dormitorios casi siempre implica más capacidad, así que gran parte del "

"índice es solo esa correlación natural entre ambas, no una interacción genuina.",

frozenset({"Huéspedes", "Tipo de alojamiento"}):

"Tiene sentido: el número de huéspedes importa mucho en vivienda entera (cabén grupos grandes) pero "

"apenas varía en habitación privada o compartida, donde casi siempre son 1-2 personas.",

frozenset({"Antigüedad del anuncio", "Noches ocupadas último año"}):

"Plausible: un anuncio más antiguo ha tenido más tiempo para ganar visibilidad y reseñas, lo que puede "

"cambiar cómo se traduce la ocupación reciente en precio frente a uno recién publicado.",

Pares nuevos del top-15 de Valencia, sin equivalente exacto en Barcelona/Madrid.

frozenset({"Días disponibles al año", "Años con cuenta en la plataforma"}):

"Cuentas más veteranas en la plataforma (no necesariamente como anfitrión) pueden reflejar usuarios más "

"profesionalizados, que gestionan la disponibilidad de forma más constante durante todo el año.",

frozenset({"Huéspedes", "Baños"}):

"Poca sorpresa de negocio, igual que Huéspedes/Dormitorios: más huéspedes casi siempre implica más baños "

"necesarios, así que gran parte del índice es esa correlación natural de tamaño, no una interacción genuina.",

frozenset({"Noches ocupadas último año", "Lavavajillas"}):

"No encuentro una historia de negocio clara. Puede ser ruido del modelo en combinaciones poco pobladas "

"(anuncios con mucha ocupación real y lavavajillas a la vez), no un patrón real.",

frozenset({"Días disponibles al año", "Años como anfitrión"}):

"Plausible: los anfitriones con más años de experiencia gestionan la disponibilidad de forma más "

"constante y profesional a lo largo del año, frente a anfitriones nuevos que la ajustan de forma irregular.",

frozenset({"Anuncios que gestiona", "% de vivienda entera del anfitrión"}):

"Tiene sentido de negocio: los anfitriones que gestionan muchos anuncios suelen operar como negocio "

"profesional especializado en vivienda entera, así que ambas variables tienden a moverse juntas.",

frozenset({"Tipo de alojamiento", "Aire acondicionado"}):

"Tiene sentido, sobre todo en una ciudad de clima cálido como Valencia: el aire acondicionado importa más "

"en una vivienda entera (controlas el clima de toda la casa) que en una habitación privada o compartida.",

frozenset({"Años con cuenta en la plataforma", "Nº de amenities"}):

"Especulativo: las cuentas más veteranas han tenido más tiempo para ir añadiendo amenities al anuncio, "

"aunque también podría ser al revés (anfitriones nuevos que equipan de más para competir); no hay una "

"dirección claramente más plausible.",

frozenset({"Huéspedes", "Valoración media"}):

"No encuentro una razón de negocio clara: `review\_scores\_rating` apenas se relaciona con el precio en "

"general (ya visto en la EDA), así que esta combinación concreta es candidata a ruido antes que a una "

"interacción real.",

frozenset({"Licencia turística", "Reseñas al mes"}):

"Plausible: un anuncio con licencia turística suele operar de forma más estable y visible, lo que facilita "

"un flujo más constante de reseñas mensuales frente a uno sin licencia.",

frozenset({"Baños", "% de vivienda entera del anfitrión"}):

"Relacionado con Anuncios que gestiona + % de vivienda entera: los anfitriones especializados en vivienda "

"entera suelen gestionar propiedades más grandes, con más baños, el mismo perfil de anfitrión profesional.",

frozenset({"Días disponibles al año", "Nivel de precios del barrio"}):

"Coherente con el patrón de Días disponibles al año + Distancia al centro: en barrios caros, alta "

"disponibilidad todo el año suele señalar un piso turístico profesional; en los más baratos, puede "

"reflejar simplemente poca demanda.",

frozenset({"Licencia turística", "Antigüedad del anuncio"}):

"Plausible: un anuncio más antiguo ha tenido más tiempo para tramitar y obtener la

```

    licencia turística; "
    "uno recién publicado puede estar todavía en proceso o directamente sin ella.",
frozenset({"Estancia máxima", "Foto de perfil"}):
    "No encuentro una razón de negocio real: 'Foto de perfil' apenas varía (casi todos los
    anfitriones la "
    "tienen) y su importancia individual es prácticamente nula, así que un índice alto aquí
    es casi "
    "seguro ruido del modelo en la franja pequeña de anfitriones sin foto, no un patrón
    genuino.",
frozenset({"Anuncios que gestiona", "Nº de amenities"}):
    "Sentido de negocio razonable: los anfitriones profesionales (muchos anuncios) tienden a
    estandarizar "
    "el equipamiento entre sus pisos, sea hacia arriba (packs de comodidades uniformes) o
    hacia abajo "
    "(menos personalización que un piso gestionado por su propio dueño); el índice no dice en
    qué dirección.",
frozenset({"Estancia máxima", "Número de reseñas"}):
    "No encuentro una razón de negocio clara: 'Estancia máxima' es un campo que muchos
    anfitriones dejan "
    "en su valor por defecto sin pensarlo, así que la combinación con 'Número de reseñas'
    probablemente "
    "refleja coincidencias de segmentos (anuncios nuevos frente a antiguos) más que una
    interacción real.",
frozenset({"Foto de perfil", "Nº de amenities"}):
    "Igual que 'Estancia máxima' + 'Foto de perfil': esta variable varía tan poco que
    cualquier índice de "
    "interacción con ella hay que mirarlo con escepticismo, probablemente ruido en la franja
    pequeña de "
    "anuncios de anfitriones sin foto.",
frozenset({"Estancia máxima", "Distrito"}):
    "Con cierta lógica: los distritos más turísticos del centro pueden limitar la estancia
    máxima permitida "
    "de forma distinta a las villas de las afueras (Churriana, Campanillas), pensadas para
    estancias "
    "vacacionales más largas.",
frozenset({"Número de reseñas", "Distrito"}):
    "Encaja con un hallazgo muy concreto de 05_model_evaluation.ipynb: varios de los peores
    errores del "
    "modelo eran anuncios sin ninguna reseña concentrados en el distrito Este, así que esta
    combinación "
    "probablemente está capturando ese patrón real y localizado, no ruido genérico.",
frozenset({"Dormitorios", "Valoración media"}):
    "Podría reflejar que las viviendas más grandes (villas) suelen ser también las mejor
    valoradas, al ser "
    "alojamientos más cuidados y de gama alta – pero con pocos anuncios de muchos
    dormitorios, también "
    "podría ser casualidad de muestra pequeña.",
frozenset({"Reseñas al mes", "Nº de amenities"}):
    "Sentido de negocio razonable: más comodidades pueden traducirse en más reservas y por
    tanto más "
    "reseñas al mes, aunque también podría ser al revés (los anuncios más solicitados van
    añadiendo "
    "comodidades con el tiempo).",
frozenset({"Número de reseñas", "Nº de amenities"}):
    "Parecido a 'Reseñas al mes' + 'Nº de amenities': ambas variables tienden a crecer con el
    tiempo que "
    "lleva el anuncio activo, así que la combinación puede reflejar simplemente antigüedad
    compartida, no "
    "que una cause la otra.",
frozenset({"Aire acondicionado", "Nº de amenities"}):
    "En parte es casi definicional: el aire acondicionado es una de las comodidades que ya
    cuenta dentro "
    "de 'Nº de amenities', así que buena parte del índice es solape entre ambas variables, no
    una "
    "interacción genuina de negocio.",
frozenset({"Licencia turística", "Densidad de anuncios cercanos"}):
    "Podría reflejar que las zonas más saturadas de anuncios turísticos son también las más
    vigiladas por "
    "la administración a la hora de exigir licencia, aunque no hay forma de confirmarlo solo
    con estos datos.",
# Pares nuevos del top-15 de Sevilla, sin equivalente exacto en las otras cuatro ciudades.
frozenset({"Años con cuenta en la plataforma", "Densidad de anuncios cercanos"}):
    "Especulativo: las cuentas más veteranas podrían haberse concentrado antes en las zonas

```

```

de más densidad "
"turística de Sevilla (las primeras en atraer oferta de Airbnb), pero con tan poca
correlación real "
"esperable entre ambas variables, es difícil descartar que sea ruido del modelo en
combinaciones poco pobladas.",
frozenset({"Baños", "Dormitorios"}):
"Poca sorpresa de negocio, igual que Huéspedes + Dormitorios: más dormitorios casi
siempre implica más "
"baños, así que buena parte del índice es la misma correlación de tamaño ya vista en
otras parejas, no "
"una interacción genuina.",
frozenset({"Años con cuenta en la plataforma", "Reseñas último año"}):
"Especulativo: las cuentas más veteranas podrían mantener un flujo más estable de reseñas
recientes "
"gracias a su experiencia acumulada, aunque también podría ser al revés (los anuncios más
nuevos generan "
"más interés inicial); no hay una dirección claramente más plausible.",
frozenset({"Estancia mínima", "Días disponibles al año"}):
"Plausible: exigir una estancia mínima larga y mantener alta disponibilidad todo el año
son dos rasgos "
"que suelen ir juntos en el alquiler de temporada, frente al turístico de estancias
cortas y "
"disponibilidad más fragmentada por reservas puntuales.",
frozenset({"Anuncios que gestiona", "Valoración media"}):
"Podría reflejar que los anfitriones profesionales (muchos anuncios) mantienen un
estándar de calidad "
"más consistente entre sus propiedades, aunque también es plausible lo contrario (menos
atención "
"personal por anuncio); no hay una dirección claramente más probable.",
frozenset({"Años con cuenta en la plataforma", "Noches ocupadas último año"}):
"Especulativo: las cuentas más veteranas podrían tener una ocupación más estable y
predecible gracias a "
"su historial y visibilidad acumulada en la plataforma, frente a cuentas nuevas con
ocupación más errática.",
frozenset({"Noches ocupadas último año", "Aire acondicionado"}):
"Plausible en una ciudad de veranos muy calurosos como Sevilla: los anuncios con aire
acondicionado "
"pueden mantener ocupación durante los meses de más calor, mientras que los que no lo
tienen pierden "
"reservas en esa temporada.",
frozenset({"Anuncios que gestiona", "Reseñas al mes"}):
"Sentido de negocio razonable: los anfitriones profesionales (muchos anuncios) suelen
tener un flujo de "
"reservas más constante, lo que se traduce en un ritmo de reseñas mensuales más estable,
aunque gestionar "
"muchos anuncios a la vez también podría diluir la atención dedicada a cada uno.",
frozenset({"Anuncios que gestiona", "Densidad de anuncios cercanos"}):
"Plausible: los anfitriones profesionales suelen concentrar su cartera en las zonas más
turísticas y de "
"más oferta de Airbnb (más fáciles de gestionar y con más demanda), lo que explicaría que
ambas variables "
"se muevan juntas.",
frozenset({"Número de reseñas", "Densidad de anuncios cercanos"}):
"Plausible: las zonas con más densidad de anuncios suelen ser también las más
consolidadas "
"turísticamente, donde los anuncios llevan más tiempo activos y acumulan más reseñas; en
zonas con poca "
"oferta, los anuncios tienden a ser más recientes y con menos historial.",
frozenset({"Estancia máxima", "Reseñas último año"}):
"No encuentro una razón de negocio clara, en la misma línea que 'Estancia máxima' +
'Número de reseñas': "
"al ser un campo que muchos anfitriones dejan en su valor por defecto, esta combinación
probablemente "
"refleja coincidencias de segmentos más que una interacción real.",
# Pares nuevos del top-15 de Mallorca, sin equivalente exacto en las otras cinco ciudades.
frozenset({"Identidad verificada", "Densidad de anuncios cercanos"}):
"No encuentro una razón de negocio clara. Con la importancia individual de 'Identidad
verificada' "
"prácticamente nula, esta combinación probablemente es ruido del modelo en la franja
pequeña de "
"anfitriones sin verificar, no un patrón real.",
frozenset({"Años como anfitrión", "Densidad de anuncios cercanos"}):
"Especulativo: los anfitriones más veteranos podrían haberse establecido antes en las

```

zonas costeras más "
 "densas en anuncios (las primeras en atraer oferta turística de la isla), pero con tan poca correlación "
 "individual esperable, es difícil descartar que sea ruido en combinaciones poco pobladas.",
 frozenset({"Huéspedes", "Días disponibles al año"}):
 "Plausible en un mercado de villas: las propiedades grandes gestionadas por profesionales suelen "
 "mantenerse disponibles buena parte del año como negocio a tiempo completo, mientras que un alojamiento "
 "pequeño de un anfitrión particular puede tener una disponibilidad más irregular.",
 frozenset({"Baños", "Días disponibles al año"}):
 "Relacionado con Huéspedes + Días disponibles al año: las propiedades con más baños son también las más "
 "grandes y profesionalizadas, con una gestión de la disponibilidad más constante durante el año.",
 frozenset({"Días desde la última reseña", "Reseñas al mes"}):
 "En buena parte mecánico: un anuncio con un ritmo alto de reseñas al mes tiene, casi por definición, "
 "menos días transcurridos desde la última – no es tanto una interacción de negocio como la misma "
 "actividad medida de dos formas distintas.",
 frozenset({"Días desde la última reseña", "Antigüedad del anuncio"}):
 "Especulativo: un anuncio más antiguo ha tenido más ocasiones de acumular huecos largos sin reseñas "
 "nuevas (temporada baja, cambios de gestión...), mientras que uno reciente todavía no ha tenido tiempo "
 "de alejarse de su primera reseña.",
 frozenset({"Años como anfitrión", "Días desde la última reseña"}):
 "Especulativo: los anfitriones más veteranos podrían gestionar el calendario de forma más constante y "
 "recibir reseñas con más regularidad, frente a anfitriones nuevos con un ritmo más irregular; no hay una "
 "dirección claramente más plausible.",
 frozenset({"Años con cuenta en la plataforma", "Aire acondicionado"}):
 "Especulativo, en la línea de 'Años con cuenta en la plataforma' + 'Nº de amenities': las cuentas más "
 "veteranas han tenido más tiempo para instalar aire acondicionado en la propiedad, aunque también podría "
 "ser al revés (anfitriones nuevos que equipan de más para competir desde el principio).",
 frozenset({"% de vivienda entera del anfitrión", "Piscina"}):
 "Tiene sentido de negocio, coherente con la EDA: los anfitriones especializados en vivienda entera "
 "gestionan sobre todo villas y casas, el tipo de propiedad donde la piscina es mucho más frecuente que "
 "en un piso urbano.",
 frozenset({"Número de reseñas", "Noches ocupadas último año"}):
 "En buena parte mecánico: más noches ocupadas en el último año generan, con el tiempo, más reseñas "
 "acumuladas – ambas variables miden la misma actividad real del anuncio desde ángulos distintos, no dos "
 "efectos independientes sobre el precio.",
 frozenset({"Anuncios que gestiona", "Superhost"}):
 "Plausible en el mercado más profesionalizado de las seis ciudades: gestionar muchos anuncios facilita "
 "acumular el volumen de reseñas y la constancia operativa que exige el estatus de Superhost, aunque "
 "también podría diluir la atención dedicada a cada anuncio individual; no hay una dirección claramente "
 "más probable.",
 # Pares nuevos del top-15 de Euskadi, sin equivalente exacto en las otras cinco ciudades.
 frozenset({"Años como anfitrión", "Estancia máxima"}):
 "Especulativo, en la línea de 'Años con cuenta en la plataforma' + 'Estancia máxima': podría reflejar "
 "cambios en las políticas y valores por defecto de Airbnb a lo largo del tiempo, con anfitriones "
 "veteranos habiendo configurado el suyo en un contexto distinto al de uno recién llegado.",
 frozenset({"Días desde la última reseña", "Número de reseñas"}):
 "Plausible: los anuncios más populares (muchas reseñas acumuladas) suelen mantener un ritmo de reservas "
 "más constante, así que su última reseña tiende a ser más reciente; uno con pocas reseñas puede llevar "

```

"tiempo sin actividad real.",
frozenset({"Baños", "Nivel de precios del barrio"}):
    "Tiene sentido de negocio: en una zona cara, un baño adicional se paga más caro (el
    espacio en sí ya "
    "vale más), mientras que en una zona barata el mismo baño extra aporta menos diferencial
    de precio.",
frozenset({"Valoración media", "Distancia al centro"}):
    "Plausible: lejos de cualquier capital de provincia, la buena valoración puede pesar más
    para compensar "
    "una ubicación menos cómoda; cerca del centro, la propia ubicación ya justifica el precio
    con "
    "independencia de la nota.",
frozenset({"Nivel de precios del barrio", "Distancia al centro"}):
    "Coherente con un hallazgo propio de Euskadi (02_feature_engineering.ipynb, sección 8.1):
    la relación "
    "entre distancia y precio cambia de signo según la capital de provincia más cercana
    (negativa en "
    "Donostia, plana en Bilbao, invertida en Vitoria-Gasteiz), así que combinarla con el
    nivel de precios "
    "del barrio recoge parte de esa heterogeneidad geográfica.",
frozenset({"Días desde la última reseña", "Camas"}):
    "No encuentro una razón de negocio clara. Con poca correlación individual de por medio,
    es candidata a "
    "ruido en combinaciones poco pobladas (anuncios de muchas camas con reseñas muy recientes
    o muy "
    "antiguas a la vez), no un patrón real.",
frozenset({"Reseñas último año", "Piscina"}):
    "Especulativo: la piscina es un atractivo más estacional (uso de verano) que el resto de
    comodidades, "
    "así que podría relacionarse con picos de reservas y reseñas concentrados en determinadas
    épocas del "
    "año, aunque el dato no distingue estacionalidad de forma directa.",
frozenset({"Días desde la última reseña", "Densidad de anuncios cercanos"}):
    "No encuentro una razón de negocio clara. Ambas variables tienen poca relación directa
    con el precio "
    "por separado, así que es candidata a ruido antes que a una interacción real.",
frozenset({"Dormitorios", "Nivel de precios del barrio"}):
    "Tiene sentido de negocio: las zonas más caras (la costa de Donostia, sobre todo)
    concentran más "
    "viviendas grandes de gama alta, mientras que las más baratas tienden a pisos más
    pequeños – tamaño y "
    "ubicación se refuerzan mutuamente en el precio.",
}

def interaction_business_note(label_a, label_b):
    return INTERACTION_BUSINESS_NOTES.get(
        frozenset({label_a, label_b}),
        "Sin hipótesis de negocio guardada para esta pareja; interpreta el índice y la
        correlación de arriba.",
    )

def pdp_category_counts(X_base, group_label, raw_values):
    """Nº de anuncios reales del test set en cada categoría, para variables categóricas."""
    spec = PDP_GROUPS[group_label]
    if spec["kind"] == "onehot":
        return [int(X_base[f"{spec['prefix']}{v}"].sum()) for v in raw_values]
    if spec["kind"] == "boolean":
        col = X_base[spec["col"]]
        return [int((col == int(v)).sum()) for v in raw_values]
    raise ValueError(f"pdp_category_counts no aplica a variables numéricas: {group_label}")

def pdp_curve(model, X_base, group_label):
    values, kind = pdp_axis_values(group_label)
    results = [float(np.exp1(model.predict(pdp_apply(X_base.copy(), group_label, v))).mean())
    for v in values]
    labels = values if kind == "numeric" else [pdp_label(v) for v in values]
    return labels, results, kind

def pdp_surface(model, X_base, group_a, group_b):

```

```

values_a, kind_a = pdp_axis_values(group_a)
values_b, kind_b = pdp_axis_values(group_b)
rows = []
for va in values_a:
    Xa = pdp_apply(X_base.copy(), group_a, va)
    for vb in values_b:
        Xc = pdp_apply(Xa.copy(), group_b, vb)
        price = float(np.expml(model.predict(Xc)).mean())
        rows.append({"a": pdp_label(va), "b": pdp_label(vb), "a_raw": va, "b_raw": vb,
                    "price": price})
df = pd.DataFrame(rows)
order_a = df.drop_duplicates("a").sort_values("a_raw")["a"].tolist() if kind_a == "numeric"
else sorted(df["a"].unique())
order_b = df.drop_duplicates("b").sort_values("b_raw")["b"].tolist() if kind_b == "numeric"
else sorted(df["b"].unique())
return df, order_a, order_b

def interaction_strength(grid_df):
    """Compara la superficie real con la que habría si las dos variables actuaran de forma
    independiente (efecto de cada una por separado, sin combinarse). Cuanto más se aleje la
    superficie real de esa versión aditiva, más fuerte es la interacción."""
    pivot = grid_df.pivot(index="b", columns="a", values="price")
    grand_mean = pivot.values.mean()
    row_effect = pivot.mean(axis=1)
    col_effect = pivot.mean(axis=0)
    additive = pd.DataFrame(
        np.add.outer(row_effect.values, col_effect.values) - grand_mean,
        index=pivot.index, columns=pivot.columns,
    )
    residual_std = (pivot - additive).values.std()
    total_std = pivot.values.std()
    ratio = residual_std / total_std if total_std > 0 else 0.0
    return ratio

WHATIF_BOOL_KEYS = {"has_license", "host_is_superhost", "host_has_profile_pic",
                    "host_identity_verified", "is_new_listing"}

REVIEW_FIELD_KEYS = set(REVIEW_DEFAULTS.keys())

ROOM_TYPE_PLAIN_LABELS = [v.split(": ", 1)[1] for v in ROOM_TYPE_ICONS.values()]
PROPERTY_TYPE_PLAIN_LABELS = ["Otro (agrupa el resto)" if pt == "Other" else pt for pt in
PROPERTY_TYPES]

# Misma redacción que el help= de "Tipo de alojamiento" en el formulario (app.py, sección
# "Alojamiento"): una única fuente de verdad para no describir cada tipo de forma distinta
# en dos sitios de la app.
ROOM_TYPE_DESCRIPTIONS = {
    "Vivienda entera": "el huésped tiene todo el piso para él",
    "Habitación privada": "habitación propia dentro de una vivienda compartida",
    "Habitación compartida": "comparte también la habitación con otros huéspedes",
    "Habitación de hotel": "gestionado como un hotel u hostel",
}

PROPERTY_TYPE_DESCRIPTIONS = {
    "Entire rental unit": "piso de alquiler estándar, vivienda entera",
    "Private room in rental unit": "habitación privada dentro de un piso de alquiler estándar",
    "Entire serviced apartment": "apartamento entero con servicios tipo hotel",
    "Entire condo": "piso entero en un edificio con servicios comunitarios",
    "Room in hotel": "habitación gestionada como en un hotel",
    "Private room in hostel": "habitación privada dentro de un hostel",
    "Entire loft": "loft entero, espacio abierto tipo estudio",
    "Private room in home": "habitación privada dentro de una vivienda particular",
    "Room in boutique hotel": "habitación en un hotel pequeño y con encanto",
    "Entire home": "casa entera para el huésped",
    "Private room in condo": "habitación privada dentro de un piso con servicios comunitarios",
    "Shared room in hostel": "cama u habitación compartida en un hostel",
    "Private room in bed and breakfast": "habitación privada en un alojamiento con desayuno
incluido",
    "Otro (agrupa el resto)": "tipos de propiedad poco frecuentes en los datos, agrupados",
}
# Estas descripciones se escribieron a mano para las categorías de Barcelona: en otra

```

```

# ciudad, property_type_grouped puede traer categorías distintas (p.ej. "Private room"
# a secas, o "Room in aparthotel" en Madrid) que no tienen descripción propia. Se usan
# solo si cubren TODAS las categorías de la ciudad activa; si no, bullet_list cae a una
# lista simple sin descripción por elemento, en vez de romper con un KeyError.
if not all(pt in PROPERTY_TYPE_DESCRIPTIONS for pt in PROPERTY_TYPE_PLAIN_LABELS):
    PROPERTY_TYPE_DESCRIPTIONS = None

def bullet_list(values, descriptions=None):
    if descriptions:
        return "\n".join(f"- **{v}**: {descriptions[v]}" for v in values)
    return "\n".join(f"- **{v}**" for v in values)

# Rangos observados en 01_eda.ipynb/02_feature_engineering.ipynb de cada ciudad:
# varían bastante entre una y otra (Madrid tiene un radio mayor y más densidad de
# anuncios en el centro), así que no tiene sentido un único rango para las dos.
_CATALOG_RANGES = {
    "barcelona": {
        "distance_to_center": "0 - 8 km aprox.",
        "density": "0 - 165 anuncios aprox.",
        "price_level": "≈ 100€ - 380€",
        "amenities": "0 - 84",
    },
    "madrid": {
        "distance_to_center": "0 - 15 km aprox.",
        "density": "0 - 320 anuncios aprox.",
        "price_level": "≈ 90€ - 255€",
        "amenities": "0 - 96",
    },
    "valencia": {
        "distance_to_center": "0 - 23 km aprox.",
        "density": "0 - 163 anuncios aprox.",
        "price_level": "≈ 99€ - 241€",
        "amenities": "0 - 92",
    },
    "malaga": {
        "distance_to_center": "0 - 19 km aprox.",
        "density": "0 - 533 anuncios aprox.",
        "price_level": "≈ 150€ - 375€",
        "amenities": "0 - 95",
    },
    "sevilla": {
        "distance_to_center": "0 - 10 km aprox.",
        "density": "0 - 266 anuncios aprox.",
        "price_level": "≈ 92€ - 220€",
        "amenities": "0 - 90",
    },
    "mallorca": {
        "distance_to_center": "0 - 72,6 km aprox.",
        "density": "0 - 53 anuncios aprox.",
        "price_level": "≈ 398€ - 1.090€",
        "amenities": "0 - 98",
    },
    "euskadi": {
        "distance_to_center": "0 - 42,4 km aprox.",
        "density": "0 - 137 anuncios aprox.",
        "price_level": "≈ 151€ - 557€",
        "amenities": "0 - 93",
    },
}

_catalog_ranges = _CATALOG_RANGES.get(CITY, _CATALOG_RANGES["barcelona"])

_ubication_vars = [
    (
        "Distrito",
        f"Distrito de {CITY_LABEL} donde está el alojamiento. Una de las variables con más peso
        en el precio.\n\n",
        "Valores posibles:\n\n" + bullet_list(DISTRICTS),
        f"{len(DISTRICTS)} distritos",
    ),
]

if HAS_NEIGHBOURHOOD_LEVEL:

```

```

_ubicacion_vars.append((
    "Barrio",
    "Barrio concreto dentro del distrito. Se usa para calcular el nivel de precios de la zona.\n\n"
    "Valores posibles:\n\n" + bullet_list(sorted(NEIGHBOURHOOD_STATS.keys())),
    f"{len(NEIGHBOURHOOD_STATS)} barrios",
))
_ubicacion_vars.append(
    ("Nivel de precios", "***Nivel de precios del barrio**\n\nPrecio medio histórico del barrio. No se introduce a mano: se calcula automáticamente a partir del barrio elegido.",
    _catalog_ranges["price_level"])
)

FEATURE_CATALOG = [
    ("Ubicación", "location_on", _ubicacion_vars + [
        ("Dist. al centro", f"***Distancia al centro**\n\nDistancia en línea recta hasta {features.CITY_CENTER_NAMES[CITY]}. Se calcula automáticamente a partir de las coordenadas.", _catalog_ranges["distance_to_center"]),
        ("Densidad cercana", "***Densidad de anuncios cercanos**\n\nNúmero de otros anuncios de Airbnb a menos de 150 metros. Se calcula automáticamente a partir de las coordenadas, igual que la distancia al centro.", _catalog_ranges["density"]),
    ]),
    ("Alojamiento", "apartment", [
        (
            "Tipo alojamiento",
            "***Tipo de alojamiento**\n\nVivienda entera, habitación privada, compartida o de hotel.\n\n"
            "Valores posibles:\n\n" + bullet_list(ROOM_TYPE_PLAIN_LABELS, ROOM_TYPE_DESCRIPTIONS),
            "4 tipos",
        ),
        (
            "Tipo propiedad",
            "***Tipo de propiedad**\n\nTipo de inmueble concreto (piso, loft, apartamento con servicios, habitación de hostel...).\n\n"
            "Valores posibles:\n\n" + bullet_list(PROPERTY_TYPE_PLAIN_LABELS, PROPERTY_TYPE_DESCRIPTIONS),
            f"{len(PROPERTY_TYPES)} tipos",
        ),
        ("Huéspedes", "Número máximo de personas que pueden alojarse.", "1 - 16"),
        ("Dormitorios", "Número de dormitorios.", "0 - 20"),
        ("Camas", "Número de camas (puede diferir de los dormitorios).", "1 - 20"),
        ("Baños", "Número de baños, admite medios baños.", "0 - 10"),
        ("Noches mín.", "***Estancia mínima**\n\nNoches mínimas por reserva. La variable con más peso de todo el modelo.", "1 - 365 noches"),
        ("Noches máx.", "***Estancia máxima**\n\nNoches máximas que se pueden reservar de una vez.", "1 - 1125 noches"),
        ("Disponibilidad", "***Días disponibles al año**\n\nDisponibilidad del anuncio en el calendario.", "0 - 365 días"),
        ("Licencia", "***Licencia turística**\n\nSi el anuncio tiene número de licencia o registro turístico asociado (requisito legal en España para alquileres de corta duración).", "Sí / No"),
    ] + [
        # Las amenities concretas varían por ciudad (p.ej. Mallorca/Euskadi tienen
        # "Vistas al mar" pero Mallorca no tiene "Lavavajillas", ver PDP_GROUPS más
        # arriba: se documenta solo la que existe de verdad en el modelo activo, en
        # vez de una lista fija que listaría una variable inexistente en algunas
        # ciudades y omitiría una real en otras.
        (label, desc, "Sí / No") for col, label, desc in [
            ("has_ac", "Aire acondicionado", "Si el anuncio tiene aire acondicionado entre sus comodidades."),
            ("has_pool", "Piscina", "Si el anuncio tiene piscina (privada o comunitaria) entre sus comodidades."),
            ("has_dishwasher", "Lavavajillas", "Si el anuncio tiene lavavajillas entre sus comodidades."),
            ("has_sea_view", "Vistas al mar", "Si el anuncio tiene vistas al mar entre sus comodidades."),
        ] if col in FEATURE_COLS
    ] + [
        ("Nº amenities", "***Nº de amenities**\n\nCantidad total de comodidades que declara el anuncio (wifi, cocina, parking, calefacción...)", _catalog_ranges["amenities"]),
    ]),
    ("Sobre ti como anfitrión", "person", [

```

```

("Superhost", "Distinción de Airbnb por buenas valoraciones, respuesta rápida y pocas cancelaciones.", "Sí / No"),
("Foto perfil", "***Foto de perfil**\n\nSi el perfil del anfitrión tiene foto visible.", "Sí / No"),
("Id. verificada", "***Identidad verificada**\n\nSi el anfitrión ha verificado su identidad ante Airbnb.", "Sí / No"),
("Nº anuncios", "***Anuncios que gestiona**\n\nCuántos anuncios en total gestiona ese anfitrión, incluido este.", "1 - 500"),
("Años anfitrión", "***Años como anfitrión**\n\nTiempo publicando anuncios en Airbnb.", "0 - 20 años"),
("Años con cuenta", "***Años con cuenta en la plataforma**\n\nTiempo registrado en Airbnb, aunque no publicara desde el principio.", "0 - 25 años"),
("% viv. entera", "***% de sus anuncios que son vivienda entera**\n\nSe calcula automáticamente a partir de cuántos anuncios gestiona ese anfitrión y de qué tipo son.", "0 - 1"),
]),
("Historial del anuncio", "history", [
    ("Anuncio nuevo", "Si el anuncio todavía no tiene reseñas ni historial de reservas.", "Sí / No"),
    ("Tiene reseñas", "Se deriva automáticamente de si el anuncio es nuevo o no.", "Sí / No"),
    ("Nº reseñas", "***Número de reseñas**\n\nTotal de reseñas recibidas desde la publicación.", "0 - 250+"),
    ("Reseñas/año", "***Reseñas último año**\n\nReseñas recibidas en los últimos 12 meses.", "0 - 50+"),
    ("Reseñas/mes", "***Reseñas al mes**\n\nMedia de reseñas recibidas por mes.", "0 - 8+"),
    ("Valoración", "***Valoración media**\n\nPuntuación media de las reseñas.", "0 - 5"),
    ("Antigüedad", "***Antigüedad del anuncio**\n\nDías desde que se publicó el anuncio.", "0 - 2900+ días"),
    ("Últ. reseña", "***Días desde la última reseña**\n\nCuánto hace de la reseña más reciente.", "0 - 365+ días"),
    ("Ocupación/año", "***Noches ocupadas último año**\n\nEstimación de cuántas noches se ha reservado en el último año.", "0 - 365 noches"),
]),
]

def whatif_apply(base_inputs, key, value):
    inp = dict(base_inputs)
    if key == "district":
        candidates = NEIGHBOURHOODS_BY_DISTRICT[value]
        stats_by_n = {n: NEIGHBOURHOOD_STATS[n] for n in candidates}
        mean_price = np.mean([s["price_encoded"] for s in stats_by_n.values()])
        best_n = min(candidates, key=lambda n: abs(stats_by_n[n]["price_encoded"] - mean_price))
        inp["district"] = value
        inp["neighbourhood"] = best_n
        inp["latitude"] = stats_by_n[best_n]["lat"]
        inp["longitude"] = stats_by_n[best_n]["lon"]
    elif key == "neighbourhood":
        stats = NEIGHBOURHOOD_STATS[value]
        inp["neighbourhood"] = value
        inp["district"] = stats["district"]
        inp["latitude"] = stats["lat"]
        inp["longitude"] = stats["lon"]
    elif key in REVIEW_FIELD_KEYS:
        inp["is_new_listing"] = False
        for rk, rv in REVIEW_DEFAULTS.items():
            inp.setdefault(rk, rv)
        inp[key] = value
    elif key == "is_new_listing":
        inp["is_new_listing"] = value
        if not value:
            for rk, rv in REVIEW_DEFAULTS.items():
                inp.setdefault(rk, rv)
    else:
        inp[key] = value
    return inp

def whatif_predict(model, base_inputs, key, value):
    inp = whatif_apply(base_inputs, key, value)
    row = build_features(inp)
    Xrow = pd.DataFrame([row])[FEATURE_COLS]

```

```

for c in Xrow.select_dtypes(include="bool").columns:
    Xrow[c] = Xrow[c].astype(int)
return float(np.expml(model.predict(Xrow)[0]))

WF_HATCH_ID = "wf-hatch-antes" # id del <pattern> SVG de rayado, ver inject_global_css()
WF_HATCH_BG = "#D7D1C2" # fondo del patrón: gris cálido claro
WF_HATCH_LINE = "#8C8776" # rayas diagonales: gris cálido oscuro, mismo tono que WF_BEFORE

def inject_global_css():
    """Todo el sistema visual de la app en un único sitio: paleta, tarjetas, cabeceras
    de sección y escala del rojo. Se llama una sola vez, justo después de
    set_page_config, para que el estilo viva en un solo lugar."""
    st.markdown(
        """
        <style>
        /* ===== Base y paleta ===== */
        .stApp {background-color: #EDEAE4;}
        [data-testid="stHeader"] {background: transparent;}
        .block-container {padding-top: 1.6rem; padding-bottom: 1rem;}
        h2 {margin-top: 0.1rem; margin-bottom: 0.5rem;}
        h3 {margin-top: 0.1rem; margin-bottom: 0.4rem;}
        h4 {margin-top: 0.1rem; margin-bottom: 0.4rem;}
        [data-testid="stMetricValue"] {font-size: 1.9rem;}
        div[data-testid="stHorizontalBlock"] {align-items: stretch;}
        [data-testid="stWidgetLabel"] {
            display: flex !important;
            align-items: center !important;
            width: fit-content !important;
        }
        [data-testid="stWidgetLabel"] > div {
            flex: 0 0 auto !important;
            justify-content: flex-start !important;
            margin-left: 0.3rem !important;
        }

        /* ===== Barra lateral ===== */
        [data-testid="stSidebar"] {
            background-color: #FFFFFF;
            border-right: 0.5px solid #DAD7CE;
        }
        [data-testid="stSidebarUserContent"] {
            padding-top: 0.75rem !important;
        }
        [data-testid="stNavSectionHeader"] {
            text-transform: uppercase;
            letter-spacing: 0.07em;
            font-size: 0.72rem !important;
            color: #8A8778 !important;
        }
        [data-testid="stSidebarNavLink"] {
            gap: 0.85rem !important;
            padding-top: 0.6rem !important;
            padding-bottom: 0.6rem !important;
            border-radius: 8px !important;
        }
        [data-testid="stSidebarNavLink"] [data-testid="stIconMaterial"] {
            font-size: 1.5rem !important;
            width: 1.5rem !important;
            height: 1.5rem !important;
        }
        [data-testid="stSidebarNavLink"] span:not([data-testid="stIconMaterial"]) {
            font-size: 1.05rem !important;
        }

        /* Pestaña activa: rosa muy claro + texto rojo oscuro, sin barra lateral roja
        (el rojo pleno se reserva para la acción principal). */
        [data-testid="stSidebarNavLink"][aria-current="page"] {
            background-color: #FCE9ED !important;
        }
        [data-testid="stSidebarNavLink"][aria-current="page"] [data-testid="stIconMaterial"],
        [data-testid="stSidebarNavLink"][aria-current="page"]
        span:not([data-testid="stIconMaterial"]) {

```

```

        color: #B0203C !important;
    }

    /* ===== Tarjetas ===== */
    /* Blancas sobre el fondo cálido, sin borde gris: la separación la da la sombra. */
    [class*="st-key-section-"] {
        background-color: #FFFFFF !important;
        border: none !important;
        border-radius: 12px !important;
        box-shadow: 0 1px 3px rgba(0, 0, 0, 0.05);
    }
    [class*="st-key-section_form-"] [data-testid="stVerticalBlock"] {
        gap: 0.65rem !important;
    }
    [class*="st-key-section_form-"] {
        padding: 0.9rem 1.3rem !important;
    }
    [class*="st-key-section_catalog-"] {
        padding: 0.35rem 0.6rem !important;
        margin-bottom: 0.4rem !important;
    }
    }

    /* ===== Tooltips ===== */
    /* Tooltip propio en CSS puro en vez del atributo title del navegador: title tiene
    ~1s de retardo fijo (no configurable) antes de aparecer, y a veces no reaparece si
    el ratón vuelve a pasar por encima justo después de un rerun de Streamlit (el nodo
    se recrea y pierde el "hover" que el navegador llevaba internamente). Con opacity/
    transition el tooltip aparece casi al instante y de forma consistente. */
    .fn-tooltip {
        position: relative;
        border-bottom: 1px dotted #6B6862;
        cursor: help;
    }
    .fn-tooltip .fn-tooltip-box {
        visibility: hidden;
        opacity: 0;
        position: absolute;
        left: 0;
        top: 135%;
        background: #FFFFFF;
        color: #2C2C2A;
        padding: 0.5rem 0.65rem;
        border-radius: 6px;
        border: 1px solid #DAD7CE;
        font-size: 0.78rem;
        line-height: 1.4;
        width: 320px;
        z-index: 999;
        box-shadow: 0 2px 10px rgba(0, 0, 0, 0.18);
        transition: opacity 0.05s linear;
        transition-delay: 0.05s;
    }
    .fn-tooltip:hover .fn-tooltip-box {
        visibility: visible;
        opacity: 1;
    }
    }

    /* ===== Cabeceras de sección ===== */
    /* La cabecera va sobre la propia tarjeta blanca, separada del cuerpo por una
    línea inferior, y llega de borde a borde. Para estirla hay que aplicar los
    márgenes negativos al .stMarkdown interior, no al .stElementContainer: a ese
    Streamlit le fija el ancho en píxeles vía JS y reafirma el valor, así que
    cualquier ajuste de ancho ahí se pierde. El .stMarkdown es un div normal en
    flujo, así que con width:auto + margen negativo se expande sin pelea.
    Cada variante de tarjeta tiene su propio padding (15px por defecto, 9.6/16px en
    Formulario, 8/12px en el catálogo de Variables): se guardan en variables CSS
    para calcular el margen negativo exacto en vez de fijar valores a ciegas.
    Ojo con la especificidad: :not() suma, así que las variantes tienen que repetirlo
    o perderían contra la regla general aun yendo después. */
    [class*="st-key-section-"]:not([class*="st-key-section_home-"]) {
        --card-pad-y: 15px;
        --card-pad-x: 15px;
    }
    }

```

```

[class*="st-key-section_form_"]:not([class*="st-key-section_home_"]) {
  --card-pad-y: 9.6px;
  --card-pad-x: 16px;
}
[class*="st-key-section_catalog_"]:not([class*="st-key-section_home_"]) {
  --card-pad-y: 5.6px;
  --card-pad-x: 9.6px;
}
[class*="st-key-section_"]:not([class*="st-key-section_home_"]) >
.stElementContainer:first-child .stMarkdown {
  margin: calc(-1 * var(--card-pad-y)) calc(-1 * var(--card-pad-x)) 0 calc(-1 *
    var(--card-pad-x));
  padding: 0.55rem var(--card-pad-x) 0.5rem var(--card-pad-x);
  border-bottom: 1px solid #E4E1D9;
  border-radius: 12px 12px 0 0;
}
[class*="st-key-section_"]:not([class*="st-key-section_home_"]) >
.stElementContainer:first-child .stMarkdown :is(h1, h2, h3, h4, h5, p) {
  margin: 0 !important;
  color: #2C2C2A;
}

/* ===== Tarjeta héroe: el precio ===== */
/* Única cabecera con fondo propio: gris oscuro, para que el precio destaque
sobre el resto de secciones. El número va en negro, en el cuerpo blanco. */
.st-key-section_prediccion_resumen > .stElementContainer:first-child .stMarkdown {
  background-color: #2C2C2A !important;
  border-bottom: none !important;
  /* Franja fina, no un panel: solo el título cabe aquí, el precio va debajo
sobre fondo blanco. */
  padding-top: 0.22rem !important;
  padding-bottom: 0.2rem !important;
}
.st-key-section_prediccion_resumen > .stElementContainer:first-child .stMarkdown :is(h4,
p) {
  color: #F1EFE8 !important;
}
/* El precio (2rem, line-height 1.2) desborda hacia abajo la caja que Streamlit
calcula para su fila, así que el padding inferior real que queda visible es
mucho menor que el superior. Se compensa aquí para igualar ambos espacios. */
.st-key-section_prediccion_resumen {
  padding-bottom: 34px !important;
}

/* ===== Cajas interiores ===== */
/* Chips de variables y tarjetas de métricas: gris cálido propio, ni blanco (se
fundiría con la tarjeta) ni el gris del fondo de página. */
[data-testid="stMetric"] {
  background-color: #F1EFE8 !important;
  border: none !important;
  border-radius: 10px;
  padding: 0.5rem 0.75rem;
}
.st-key-variable_catalog [data-testid="stMetric"] {
  padding: 0.12rem 0.4rem !important;
  gap: 0 !important;
  /* Nombre en dos líneas (p.ej. "Densidad cercana") o valor en dos líneas
(p.ej. "0 - 2900+ días") hacen más alta esa tarjeta que sus vecinas de la
misma fila; se fija la altura del peor caso para que toda la fila quede pareja. */
  min-height: 120px;
}
.st-key-variable_catalog [data-testid="stMetricLabel"] {
  font-size: 0.75rem;
  font-weight: 400 !important;
  color: #6B6862 !important;
  margin: 0 !important;
}
.st-key-variable_catalog [data-testid="stMetricLabel"] p {font-weight: 400 !important;
color: #6B6862 !important;}
/* Streamlit trunca con ellipsis en un div interno (no en el <p> ni en el label),
tanto en la etiqueta como en el valor: hay que abrir el paso ahí también, si no
el texto se corta a medias aunque el <p> ya permita saltar de línea. */
.st-key-variable_catalog [data-testid="stMetricLabel"] > div,

```

```

.st-key-variable_catalog [data-testid="stMetricLabel"]
[data-testid="stMarkdownContainer"] {
  overflow: visible !important;
  white-space: normal !important;
  text-overflow: unset !important;
}
.st-key-variable_catalog [data-testid="stMetricLabel"] p {
  white-space: normal !important;
  overflow: visible !important;
  text-overflow: unset !important;
  overflow-wrap: normal !important;
  word-break: normal !important;
  line-height: 1.1;
  margin: 0 !important;
}
.st-key-variable_catalog [data-testid="stTooltipIcon"] {
  width: 0.9rem;
  height: 0.9rem;
}
.st-key-variable_catalog [data-testid="stMetricValue"],
.st-key-variable_catalog [data-testid="stMetricValue"] > div {
  font-size: 0.95rem;
  font-weight: 700 !important;
  line-height: 1.1;
  margin: 0 !important;
  white-space: normal !important;
  overflow: visible !important;
  text-overflow: unset !important;
}

.st-key-section_comparar_veredicto [data-testid="stMetric"] {
  padding: 0.3rem 0.6rem !important;
  gap: 0.05rem;
}
.st-key-section_comparar_veredicto [data-testid="stMetricValue"] {font-size: 1.3rem;
line-height: 1.2;}

/* ===== Botón flotante sobre el mapa de coordenadas =====
*/
/* El icono "ampliar mapa" vive dentro del mismo contenedor que el mapa pequeño,
posicionado en absoluto sobre su esquina superior derecha, para no añadir una fila
propia y no restar alto vertical al formulario. */
.st-key-coord_map_wrap {
  position: relative;
}
.st-key-coord_map_wrap .st-key-expand_map_btn {
  position: absolute;
  top: 6px;
  right: 6px;
  z-index: 1000;
}
.st-key-coord_map_wrap .st-key-expand_map_btn button {
  padding: 0.2rem 0.4rem;
  min-height: unset;
  border-radius: 6px;
  box-shadow: 0 1px 4px rgba(0,0,0,0.35);
}

/* ===== Escala del rojo ===== */
/* Toggles: rojo claro al estar activos, para no competir con el botón principal
(el único con rojo pleno). El track es el div directo dentro de
<label data-baseweb="checkbox"> que a su vez contiene otro div (el thumb): esa
forma distingue un toggle de un st.checkbox normal, cuyo primer hijo es un
<span> (icono de casilla). */
label[data-baseweb="checkbox"]:has(> input:checked) > div:first-child:has(> div) {
  background-color: #F7AEB3 !important;
  border-color: #F7AEB3 !important;
}
label[data-baseweb="checkbox"]:has(> input:not(:checked)) > div:first-child:has(> div) {
  background-color: #D3D1C7 !important;
  border-color: #D3D1C7 !important;
}
/* Checkbox marcado: mismo criterio que el toggle (su primer hijo es un <span>,

```

```

no un <div>, que es justo lo que los distingue). */
label[data-baseweb="checkbox"]:has(> input:checked) > span:first-child {
  background-color: #F7AEBC !important;
  border-color: #F7AEBC !important;
}
/* Pill seleccionada: mismo tratamiento que la pestaña activa de la nav. */
[data-testid="stBaseButton-pillsActive"] {
  background-color: #FCE9ED !important;
  border-color: #B0203C !important;
  color: #B0203C !important;
}

/* ===== Tarjetas de Inicio (enlaces) ===== */
[class*="st-key-section_home_"] {
  position: relative;
  transition: box-shadow 0.15s ease;
}
[class*="st-key-section_home_"]:hover {
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.10);
}
[class*="st-key-section_home_"]
.stElementContainer:has(a[data-testid="stPageLink-NavLink"]) {
  /* Streamlit pone position:relative aquí; si no se anula, ese contenedor (que se
  colapsa a 0 al sacar el enlace del flujo) pasa a ser el bloque contenedor del
  enlace absoluto, en vez de la tarjeta entera. */
  position: static !important;
  /* El enlace absoluto ya no aporta altura al flujo normal: se reserva a mano el
  hueco del título para que el texto de abajo no quede pegado/solapado. */
  min-height: 2rem;
}
[class*="st-key-section_home_"] a[data-testid="stPageLink-NavLink"] {
  /* inset con el mismo valor que el padding de la tarjeta (15px, el de
  st.container(border=True)): un enlace absoluto se posiciona respecto al borde
  de la caja de padding, así que con inset:0 el título quedaba pegado al borde,
  ignorando ese padding. Con width/height en auto (no 100%), se recalculan solos
  a partir de estos 4 huecos, dejando la misma separación que el resto del texto. */
  position: absolute !important;
  inset: 15px !important;
  width: auto !important;
  height: auto !important;
  z-index: 1;
  align-items: flex-start !important;
}

/* ===== Botón "Predecir precio" (Formulario) ===== */
/* Vive en col_left, justo debajo de Comodidades: al ser una columna independiente de
col_right, no se mueve aunque Historial del anuncio (en col_right) se expanda al
desplegar las reseñas. Sin tarjeta blanca (la clave no lleva prefijo "section_"),
solo el botón en sí, agrandado. */
.st-key-form_predict_wrap [data-testid="stBaseButton-primary"] {
  padding-top: 0.6rem;
  padding-bottom: 0.6rem;
  font-size: 1.05rem;
}

/* ===== Mapa por barrio (Explora el mercado) ===== */
/* La columna de "Ver por" + filtros tenía de sobra el gap por defecto (16px) entre
cada widget, lo que la alargaba muy por debajo del mapa (altura fija de 480px) y
obligaba a hacer scroll. Se aprieta aquí para que ambas columnas acaben a la misma
altura, sin tocar ni las etiquetas ni la lógica de los filtros. */
.st-key-section_mercado_mapa [data-testid="stHorizontalBlock"] > div:nth-child(2) >
[data-testid="stVerticalBlock"] {
  gap: 8px;
}
</style>
"""
unsafe_allow_html=True,
)
# Patrón SVG reutilizable para el relleno "rayado" del waterfall de Predicción (barra
# "antes" de simular): Vega-Lite no tiene relleno de patrón nativo, pero su renderer
# SVG traslada literalmente lo que se le pase como "fill" al atributo fill del <rect>,
# así que un fill="url(#id)" que apunte a un <pattern> definido en el propio DOM de la
# página SÍ funciona, aunque esté fuera del <svg> que genera Vega. Se define una única

```

```

# vez aquí (tamaño 0, no ocupa espacio) para que cualquier gráfico de la app pueda
# referenciarlo.
st.markdown(
    f"""
    <svg width="0" height="0" style="position:absolute" aria-hidden="true">
    <defs>
    <pattern id="{WF_HATCH_ID}" patternUnits="userSpaceOnUse" width="7" height="7"
    patternTransform="rotate(45)">
    <rect width="7" height="7" fill="{WF_HATCH_BG}"></rect>
    <line x1="0" y1="0" x2="0" y2="7" stroke="{WF_HATCH_LINE}"
    stroke-width="3.5"></line>
    </pattern>
    </defs>
    </svg>
    """,
    unsafe_allow_html=True,
)

inject_global_css()

@st.cache_resource
def load_model(city):
    return joblib.load(PROJECT_DIR / "models" / city / "price_model.pkl")

@st.cache_data
def load_comparables(city):
    cols = [
        "id", "host_id", "price", "room_type", "property_type", "neighbourhood_cleansed",
        "neighbourhood_group_cleansed", "accommodates", "bedrooms", "bathrooms",
        "minimum_nights", "latitude", "longitude", "name", "picture_url", "listing_url",
        "estimated_occupancy_l365d", "review_scores_rating", "has_license",
    ]
    csv_path = PROJECT_DIR / "data" / "processed" / city / "listings_full_clean.csv"
    # Málaga no tiene distrito separado de barrio (ver 02_feature_engineering.ipynb de
    # Málaga, sección 5.2): neighbourhood_group_cleansed no existe en su CSV, así que se
    # excluye de usecols en vez de fallar con un ValueError de columna no encontrada.
    header_cols = set(pd.read_csv(csv_path, nrows=0).columns)
    cols = [c for c in cols if c in header_cols]
    df = pd.read_csv(csv_path, usecols=cols)
    df["price_weight"] = np.log1p(df["price"])
    return df

@st.cache_data
def load_district_geojson(city):
    with open(APP_DIR / "assets" / city / "districts.geojson", encoding="utf-8") as f:
        return json.load(f)

@st.cache_data
def load_neighbourhood_geojson(city):
    with open(APP_DIR / "assets" / city / "neighbourhoods_colored.geojson", encoding="utf-8") as f:
        return json.load(f)

@st.cache_data
def load_feature_importance(city):
    with open(APP_DIR / "assets" / city / "feature_importance.json", encoding="utf-8") as f:
        return json.load(f)

@st.cache_data
def load_interaction_scores(city):
    with open(APP_DIR / "assets" / city / "interaction_scores.json", encoding="utf-8") as f:
        return json.load(f)

@st.cache_data
def load_test_matrix(city):

```

```

test_df = pd.read_csv(PROJECT_DIR / "data" / "processed" / city / "listings_test.csv")
X = test_df[FEATURE_COLS].copy()
for c in X.select_dtypes(include="bool").columns:
    X[c] = X[c].astype(int)
return X

# Único punto de control del nº de tramos de precio: antes estaba repetido a mano como
# literal "5" en tres sitios (pd.qcut de load_test_evaluation, la lista de labels de
# _PRICE_QUINTILE_ORDER y el quintile_order local de "Dónde es más y menos fiable"), más
# el propio pd.quantile de load_price_segment_errors con 6 cortes fijos. Cambiar este
# número ahora basta para regenerar etiquetas, cortes y la tira de "Cuánto acierta" igual.
N_PRICE_SEGMENTS = 5

def _price_segment_names(n=N_PRICE_SEGMENTS):
    names = [f"Q{i + 1}" for i in range(n)]
    names[0] += " (más baratos)"
    names[-1] += " (más caros)"
    return names

_PRICE_QUINTILE_ORDER = _price_segment_names()

@st.cache_data
def load_test_evaluation(city):
    """Sesgo del modelo (predicho - real) en el set de test, por distrito y por
    quintil de precio. Mismo análisis que 05_model_evaluation.ipynb, recalculado
    aquí para poder mostrarlo en la app."""
    test_df = pd.read_csv(PROJECT_DIR / "data" / "processed" / city / "listings_test.csv")
    X = test_df[FEATURE_COLS].copy()
    for c in X.select_dtypes(include="bool").columns:
        X[c] = X[c].astype(int)
    pred = np.expm1(model.predict(X))
    actual = test_df["price"].values
    residual = pred - actual

    district_cols = [c for c in FEATURE_COLS if c.startswith("district_")]
    district = X[district_cols].idxmax(axis=1).str.replace("district_", "", regex=False)

    quintile = pd.qcut(actual, N_PRICE_SEGMENTS, labels=_PRICE_QUINTILE_ORDER)

    return pd.DataFrame({
        "district": district.values,
        "quintile": quintile.astype(str),
        "residual": residual,
        "price": actual,
    })

@st.cache_data
def load_price_segment_errors(city):
    """MAE real por tramo de precio del anuncio, en vez del MAE plano de test_metrics
    (que promedia también la cola de anuncios de lujo, mucho más ancha en ciudades como
    Mallorca). Devuelve el MAE de cada tramo junto con sus bordes de precio, para poder
    decir "para un anuncio de tu rango de precio, el error típico es X€" en vez de aplicar
    el mismo margen a un anuncio barato y a una villa de miles de euros."""
    eval_df = load_test_evaluation(city)
    edges = np.quantile(eval_df["price"], np.linspace(0, 1, N_PRICE_SEGMENTS + 1))
    grouped = eval_df.groupby("quintile")
    segments = pd.DataFrame({
        "quintile": _PRICE_QUINTILE_ORDER,
        "mae": grouped["residual"].apply(lambda r:
            r.abs().mean()).reindex(_PRICE_QUINTILE_ORDER).values,
        "price_min": grouped["price"].min().reindex(_PRICE_QUINTILE_ORDER).values,
        "price_max": grouped["price"].max().reindex(_PRICE_QUINTILE_ORDER).values,
        "n": grouped["price"].size().reindex(_PRICE_QUINTILE_ORDER).values,
    })
    return segments, edges

def mae_for_price(city, price):

```

```

"""MAE del quintil de precio real de test al que pertenecería `price`. Los bordes
vienen del precio REAL de los anuncios de test (no del predicho): es la misma
referencia que ya se usa en el desglose de sesgo por quintil de 'Cómo funciona
el modelo', así que ambas vistas son consistentes entre sí."""
segments, edges = load_price_segment_errors(city)
idx = int(np.clip(np.searchsorted(edges, price, side="right") - 1, 0, len(segments) - 1))
return segments.iloc[idx]

def price_segment_index(edges, price):
    """Índice del tramo de load_price_segment_errors al que pertenece `price`, con los
    mismos bordes (edges) que ya usa mae_for_price. Función aparte porque aquí hace falta
    el índice en sí (para resaltar el tramo en la tira), no la fila de error."""
    return int(np.clip(np.searchsorted(edges, price, side="right") - 1, 0, len(edges) - 2))

def _format_price_segment_labels(edges):
    """Etiquetas de rango ('hasta X €', 'X - Y €', 'más de X €') a partir de los propios
    cortes de cuantil, no de min/max observados en cada tramo: así encajan siempre sin
    huecos ni solapes (edges[i] es a la vez el límite alto de un tramo y el bajo del
    siguiente), sin importar cuántos tramos haya (ver N_PRICE_SEGMENTS)."""
    n = len(edges) - 1
    labels = []
    for i in range(n):
        lo, hi = edges[i], edges[i + 1]
        if i == 0:
            labels.append(f"hasta {hi:,.0f} €")
        elif i == n - 1:
            labels.append(f"más de {lo:,.0f} €")
        else:
            labels.append(f"{lo:,.0f} - {hi:,.0f} €")
    return labels

def _inject_acierto_css():
    st.markdown(
        """<style>
        .st-key-section_modelo_acierto .acierto-tira {
            display: grid;
            grid-template-columns: repeat(var(--n-tramos), minmax(0, 1fr));
            background: #F5F3EE;
            border-radius: 11px;
            overflow: hidden;
        }
        .st-key-section_modelo_acierto .acierto-segmento {
            padding: 15px 14px 14px;
            border-left: 1px solid #E4E0D8;
        }
        .st-key-section_modelo_acierto .acierto-segmento:first-child { border-left: none; }
        .st-key-section_modelo_acierto .acierto-segmento--activo {
            background: #FFF1F4;
            box-shadow: inset 0 0 0 1.5px #FF385C;
        }
        .st-key-section_modelo_acierto .acierto-segmento--activo .acierto-rango,
        .st-key-section_modelo_acierto .acierto-segmento--activo .acierto-valor {
            color: #C42843;
        }
        .st-key-section_modelo_acierto .acierto-rango {
            font-size: 12.5px; color: #6B6A66; margin-bottom: 2px;
        }
        .st-key-section_modelo_acierto .acierto-valor {
            font-size: 25px; font-weight: 700; letter-spacing: -.02em; color: #2C2C2A;
        }
        .st-key-section_modelo_acierto .acierto-tu-prediccion {
            font-size: 11px; font-weight: 700; color: #C42843; margin-top: 4px;
        }
        @media (max-width: 760px) {
            .st-key-section_modelo_acierto .acierto-tira { grid-template-columns: 1fr; }
            .st-key-section_modelo_acierto .acierto-segmento { border-left: none; border-top: 1px
            solid #E4E0D8; }
            .st-key-section_modelo_acierto .acierto-segmento:first-child { border-top: none; }
        }
        .st-key-section_modelo_acierto [data-testid="stExpander"] {

```

```

        border: none !important;
        border-top: 1px solid #E3E0DA !important;
        border-radius: 0 !important;
    }
    .st-key-section_modelo_acierto [data-testid="stExpander"] summary {
        font-size: 14px !important;
        font-weight: 600 !important;
        color: #6B6A66 !important;
    }
</style>""",
    unsafe_allow_html=True,
)

def render_price_error_strip(segments, edges, highlight_idx, user_price):
    """Tira continua de tramos de precio (HTML propio: las columnas nativas de Streamlit
    meten hueco entre elementos y romperían la continuidad visual pedida). highlight_idx
    es el tramo de la predicción actual del usuario, o None si todavía no ha calculado
    ninguna."""
    labels = _format_price_segment_labels(edges)
    parts = [f'<div class="acierto-tira" style="--n-tramos:{len(segments)};">']
    for i, (label, row) in enumerate(zip(labels, segments.itertuples())):
        is_active = i == highlight_idx
        cls = "acierto-segmento acierto-segmento--activo" if is_active else "acierto-segmento"
        tu_prediccion = (
            f'<div class="acierto-tu-prediccion">tu predicción: {user_price:,.0f} €</div>' if
            is_active else ""
        )
        parts.append(
            f'<div class="{cls}">'
            f'<div class="acierto-rango">{html.escape(label)}</div>'
            f'<div class="acierto-valor">±{row.mae:,.0f} €</div>'
            f"{tu_prediccion}"
            f"</div>"
        )
    parts.append("</div>")
    return "".join(parts)

@st.cache_resource
def get_shap_explainer(_model, city):
    return shap.TreeExplainer(_model.named_steps["model"])

# El explainer de arriba es un @st.cache_resource: UNA sola instancia compartida por
# todas las sesiones/pestañas abiertas, no una por usuario. shap.TreeExplainer reutiliza
# buffers internos entre llamadas (por rendimiento) y no es thread-safe – si dos
# llamadas se solapan (típico al arrastrar el slider de "¿Qué pasaría si...?", que
# dispara varios reruns seguidos, o con dos usuarios a la vez), pueden pisarse los
# resultados y salir NaN/basura, y con eso el waterfall de "Por qué este precio" sale
# en blanco sin ningún error de Python (el NaN se propaga en silencio por los cálculos
# numéricos). Este lock serializa todas las llamadas al explainer, cueste lo que cueste
# en concurrencia, para eliminar esa condición de carrera.
_SHAP_EXPLAINER_LOCK = threading.Lock()

def explain(explainer, X):
    with _SHAP_EXPLAINER_LOCK:
        return explainer(X)

@st.cache_data
def load_shap_summary(city, sample_size=400):
    """Contribución SHAP de cada variable numérica/booleana en una muestra del test set,
    con el valor real de esa variable normalizado (0-1) para colorear cada punto."""
    X_test = load_test_matrix(city)
    n = min(sample_size, len(X_test))
    X_sample = X_test.sample(n=n, random_state=42).reset_index(drop=True)
    explainer = get_shap_explainer(model, city)
    shap_values = explain(explainer, X_sample).values
    col_index = {c: i for i, c in enumerate(X_sample.columns)}

    rng = np.random.default_rng(42)

```

```

frames = []
for label, spec in SHAP_SUMMARY_GROUPS.items():
    if spec["kind"] == "numeric":
        cols = spec["cols"]
        value_col = spec["sample_key"]
    else:
        cols = [spec["col"]]
        value_col = spec["col"]
    contrib = sum(shap_values[:, col_index[c]] for c in cols)
    raw_values = X_sample[value_col].to_numpy(dtype=float)
    vmin, vmax = raw_values.min(), raw_values.max()
    value_norm = (raw_values - vmin) / (vmax - vmin) if vmax > vmin else
    np.zeros_like(raw_values)
    frames.append(pd.DataFrame({
        "feature": label,
        "shap": contrib,
        "value_norm": value_norm,
        "jitter": rng.uniform(-0.4, 0.4, size=n),
    })))

df = pd.concat(frames, ignore_index=True)
order = (
    df.groupby("feature")["shap"].apply(lambda s:
        s.abs().mean()).sort_values(ascending=False).index.tolist()
)
return df, order

BLUE = "#3b6ea5"
RED = "#c0392b"
GREEN = "#1a7f37"
# What-if: dos intensidades de azul en vez de rojo, para dejar el rojo libre como
# "baja el precio" en el waterfall de al lado y que no se confundan.
WHATIF_CURRENT = "#185FA5" # "tu valor actual" (gráfico de línea, numéricas)
WHATIF_ALT = "#B5D4F4" # las alternativas (ambos gráficos what-if)
WHATIF_SIM = "#E8833A" # "valor simulado/tu valor" en toda la app (what-if de Predicción,
# veredicto de Comparar): naranja vivo, distinto del azul y del ámbar de los avisos
MARKET_COLOR = "#8e44ad" # codificación de datos para "mercado de temporada / larga estancia",
ver page_modelo
GRAY_MUTED = "#9aa0a6" # zona de transición 8-30 noches: poco frecuente, casi sin datos
WF_START = "#C4C0B6" # waterfall: punto de partida (gris neutro, no es un cambio de precio)
WF_TOTAL = "#3A3A37" # waterfall: precio final (gris oscuro; evita rojo, que ahí significa
"baja")
WF_BEFORE = "#A79E8C" # waterfall what-if: valor "antes" de simular (gris cálido
semitransparente,
# sin color de dirección) en la fila de la variable simulada, para no competir con verde/rojo

MARKET_CUTOFF = 30 # noches: <=30 turístico, >=31 temporada (frontera legal habitual para VUT en
España)

BEDROOMS_SLIDER_MAX = 6 # filtro "Tamaño" del Mapa del mercado: el tope del slider es "6+"
MIN_LISTINGS_PER_BARRIO = 5 # por debajo de esto, un barrio se pinta "sin datos": la media
# ya no es fiable calculada sobre 2-3 anuncios tras aplicar los filtros

def market_label(minimum_nights):
    if minimum_nights <= MARKET_CUTOFF:
        return "■■ Mercado turístico (corta estancia)"
    return "■ Mercado de temporada (larga estancia)"

@st.cache_data
def neighbourhood_market_agg(city, room_type, accommodates_range, bedrooms_range):
    """"Agregados por barrio para el Mapa del mercado. room_type='Todos' o uno de
    ROOM_TYPES, accommodates_range=(min, max) de huéspedes, bedrooms_range=(min, max)
    con max=BEDROOMS_SLIDER_MAX significando "sin tope superior" (el "6+" del slider):
    recalcula todos los agregados solo con los anuncios que cumplen los tres filtros,
    sin tocar comparables_all. avg_rating usa mean() con skipna, ignorando anuncios
    nuevos sin reseñas todavía en vez de arrastrar NaN al agregado del barrio. `city`
    solo sirve para que la caché distinga entre ciudades: comparables_all cambia de
    contenido al cambiar de ciudad aunque los demás argumentos sean iguales.""
    df = comparables_all
    if room_type != "Todos":

```

```

    df = df[df["room_type"] == room_type]
    acc_lo, acc_hi = accommodates_range
    df = df[df["accommodates"].between(acc_lo, acc_hi)]
    bed_lo, bed_hi = bedrooms_range
    if bed_hi >= BEDROOMS_SLIDER_MAX:
        df = df[df["bedrooms"] >= bed_lo]
    else:
        df = df[df["bedrooms"].between(bed_lo, bed_hi)]
    return (
        df.groupby("neighbourhood_cleansed")
        .agg(
            avg_price=("price", "mean"),
            n_listings=("price", "size"),
            pct_corta_estancia=("minimum_nights", lambda s: (s <= MARKET_CUTOFF).mean()),
            pct_entire_home=("room_type", lambda s: (s == "Entire home/apt").mean()),
            avg_occupancy=("estimated_occupancy_l365d", "mean"),
            avg_rating=("review_scores_rating", "mean"),
            pct_license=("has_license", "mean"),
        )
        .rename_axis("neighbourhood")
        .reset_index()
    )

def chart_market_bins(df):
    """Barras de nº de anuncios por tramo de minimum_nights, coloreadas por grupo de
    mercado (turístico / zona de transición casi sin datos / temporada), para enseñar
    de un vistazo el valle entre los dos mercados que conviven en el dataset."""
    order = df["tramo"].tolist()
    return (
        alt.Chart(df)
        .mark_bar(cornerRadiusEnd=3)
        .encode(
            x=alt.X("n:Q", title="Nº de anuncios"),
            y=alt.Y("tramo:N", sort=order, title=None),
            color=alt.Color(
                "grupo:N",
                scale=alt.Scale(
                    domain=["Turístico", "Zona de transición", "Temporada"],
                    range=[BLUE, GRAY_MUTED, MARKET_COLOR],
                ),
                legend=alt.Legend(title=None, orient="top"),
            ),
            tooltip=[
                alt.Tooltip("tramo:N", title=""),
                alt.Tooltip("n:Q", title="Nº de anuncios"),
                alt.Tooltip("mediana_precio:Q", title="Mediana €/noche", format=".0f"),
            ],
        )
        .properties(height=alt.Step(28))
    )

def chart_hbar(labels, values, x_title, color=BLUE):
    df = pd.DataFrame({"label": labels, "value": values})
    return (
        alt.Chart(df)
        .mark_bar(color=color, cornerRadiusEnd=3)
        .encode(
            x=alt.X("value:Q", title=x_title),
            # labelLimit=0: sin límite, para que no se trunquen con "..." ni las etiquetas
            # largas (variables agrupadas) ni las categorías con el conteo de anuncios al lado.
            y=alt.Y("label:N", sort="-x", title=None, axis=alt.Axis(labelLimit=0)),
            tooltip=[alt.Tooltip("label:N", title=""), alt.Tooltip("value:Q", title=x_title,
                format=".2f")],
        )
        .properties(height=alt.Step(27))
    )

def chart_hbar_highlight(labels, values, sim_label, real_label, x_title):
    """sim_label es el valor SIMULADO (el del desplegable "Simula un valor"). real_label
    es el guardado en el Formulario: si cae fuera de la ventana visible de barras no

```

```

aparece aquí (se muestra aparte, como texto). Si coinciden (no se está simulando
nada distinto), esa única barra es "Tu valor real": el naranja de "Valor simulado"
solo tiene sentido cuando de verdad hay un valor simulado distinto del real."""
df = pd.DataFrame({"label": labels, "value": values})
def estado_for(label):
    if label == sim_label and sim_label != real_label:
        return "Valor simulado"
    if label == real_label:
        return "Tu valor real"
    return "Alternativa"
df["estado"] = df["label"].apply(estado_for)
return (
    alt.Chart(df)
    .mark_bar(cornerRadiusEnd=3)
    .encode(
        # stack=None: cada fila es una barra independiente en su propia posición Y, no
        # un segmento a sumar con las demás – sin esto, Vega-Lite apila por defecto en
        # cuanto hay un canal de color en un mark_bar, y en ciertas combinaciones de
        # categorías (comprobado cambiando de variable en "¿Qué pasaría si...?" el
        # cálculo de la pila se rompía del todo ("Infinite extent for field
        # value_start/value_end" en la consola) y el gráfico salía en blanco.
        x=alt.X("value:Q", title=x_title, stack=None),
        y=alt.Y("label:N", sort="-x", title=None, axis=alt.Axis(labelLimit=250)),
        color=alt.Color(
            "estado:N",
            scale=alt.Scale(
                domain=["Tu valor real", "Valor simulado", "Alternativa"],
                range=[WHATIF_CURRENT, WHATIF_SIM, WHATIF_ALT],
            ),
            legend=alt.Legend(title=None, orient="top"),
        ),
        tooltip=[alt.Tooltip("label:N", title=""), alt.Tooltip("value:Q", title=x_title,
            format=".0f")],
    )
    .properties(height=alt.Step(20))
)

def chart_bias_hbar(labels, values, x_title, sort="x"):
df = pd.DataFrame({"label": labels, "value": values})
df["sesgo"] = df["value"].apply(lambda v: "Sobreestima" if v >= 0 else "Subestima")
return (
    alt.Chart(df)
    .mark_bar(cornerRadiusEnd=3)
    .encode(
        # stack=None: mismo motivo que en chart_hbar_highlight (cada fila es su propia
        # barra, no un segmento a apilar); el color aquí solo indica dirección del
        # sesgo, no participa en una suma.
        x=alt.X("value:Q", title=x_title, stack=None),
        y=alt.Y("label:N", sort=sort, title=None, axis=alt.Axis(labelLimit=200)),
        color=alt.Color(
            "sesgo:N",
            scale=alt.Scale(domain=["Sobreestima", "Subestima"], range=[BLUE, RED]),
            legend=alt.Legend(title=None, orient="top"),
        ),
        tooltip=[alt.Tooltip("label:N", title=""), alt.Tooltip("value:Q", title=x_title,
            format=".1f")],
    )
    .properties(height=alt.Step(24))
)

def chart_waterfall(df, x_title):
    """Tres significados de color: gris = punto de partida, verde/rojo = el paso sube o
    baja el precio, gris oscuro = total. Las etiquetas de valor van siempre por fuera del
    extremo derecho de cada barra, así nunca se solapan aunque la barra sea muy corta."""
    df = df.copy()
    # Ancla de la etiqueta principal: normalmente el final de la barra sólida ("end"), pero
    # si el rayado "antes" asoma más allá de esta (misma dirección, valor anterior mayor en
    # magnitud) el número quedaría escrito encima de la textura rayada y cuesta leerlo. Se
    # ancla entonces al extremo que quede más lejos de los dos, para caer siempre en blanco.
    df["etiqueta_x"] = df["end"] if "before_end" not in df.columns else df[["end",
    "before_end"]].max(axis=1)

```

```

order = df["label"].tolist()
# La leyenda de Vega-Lite va DESACTIVADA aquí (legend=None): "antes (valor anterior)"
# no puede entrar en ella (fusionarla como 5ª categoría entre las 3 capas de barras
# provocaba un bug real de Vega, categorías que desaparecían sin avisar), así que en
# vez de tener dos leyendas sueltas (la de Vega + la de "antes" aparte) se pinta una
# única leyenda propia en HTML/CSS, con las 5 categorías juntas y en el mismo orden,
# justo encima del gráfico (ver el st.markdown antes de "st.altair_chart").
color = alt.Color(
    "kind:N",
    scale=alt.Scale(
        domain=["punto de partida", "sube", "baja", "total"],
        range=[WF_START, GREEN, RED, WF_TOTAL],
    ),
    legend=None,
)
y = alt.Y("label:N", sort=order, title=None, axis=alt.Axis(labelLimit=250))
# Margen a la derecha para que quepan las etiquetas fuera de la barra más larga; el
# contorno "antes" (what-if) puede llegar más lejos que cualquier "end", así que
# también cuenta para el máximo de la escala.
x_max = float(df["end"].max())
if "before_end" in df.columns and df["before_end"].notna().any():
    x_max = max(x_max, float(df["before_end"].max()))
# Con etiqueta "antes" (variable simulada) el texto tras la barra es más largo (la
# etiqueta principal más "antes ±N €" pegada al lado): el margen de siempre (15%) se
# queda corto y la recorta contra el borde derecho, así que aquí se amplía.
has_antes_label = "etiqueta_antes" in df.columns and df["etiqueta_antes"].notna().any()
margin = 1.7 if has_antes_label else 1.15
x_scale = alt.Scale(domain=[0, x_max * margin], nice=False)
x = alt.X("start:Q", title=x_title, scale=x_scale)

pasos = df[df["kind"].isin(["sube", "baja"])]
extremos = df[df["kind"].isin(["punto de partida", "total"])]

# Dos capas de barras solo por el tooltip: los pasos tienen salto acumulado que
# enseñar y los extremos no, y en Vega el tooltip se define por capa.
# title=" " en vez de "": con título vacío Vega cae de vuelta al nombre del campo
# ("tip1") y lo pinta como etiqueta de la fila del tooltip.
# Vega-Lite/vega-tooltip escapan cualquier HTML puesto en el valor de un tooltip
# (comprobado: "<b>...</b>" sale literal, como texto, no en negrita) y
# st.altair_chart no expone la opción sanitize:false para desactivarlo – así que
# aquí no hay negrita ni color, solo texto plano en 3 líneas.
bars_pasos = alt.Chart(pasos).mark_bar(cornerRadiusEnd=3).encode(
    x=x, x2="end:Q", y=y, color=color,
    tooltip=[
        alt.Tooltip("tip1:N", title=" "),
        alt.Tooltip("tip2:N", title=" "),
        alt.Tooltip("tip3:N", title=" "),
    ],
)
# "Precio final" no tiene salto que enseñar: con un único campo (no una lista) Vega
# pinta solo el valor, sin tabla clave/valor. "Punto de partida" sí lleva una segunda
# línea explicando qué es ese precio medio de referencia, así que necesita su propia
# capa con tooltip de dos campos (una lista, como bars_pasos).
partida = extremos[extremos["kind"] == "punto de partida"]
total_extremo = extremos[extremos["kind"] == "total"]
bars_partida = alt.Chart(partida).mark_bar(cornerRadiusEnd=3).encode(
    x=x, x2="end:Q", y=y, color=color,
    tooltip=[
        alt.Tooltip("tip1:N", title=" "),
        alt.Tooltip("tip2:N", title=" "),
    ],
)
bars_total = alt.Chart(total_extremo).mark_bar(cornerRadiusEnd=3).encode(
    x=x, x2="end:Q", y=y, color=color,
    tooltip=alt.Tooltip("tip1:N"),
)
# Las etiquetas llevan color propio (scale=None pasa el hex tal cual y no añade una
# segunda leyenda): el gris de la barra de partida sería ilegible como texto.
# tooltip=alt.value(None): sin esto, Vega-Lite genera un tooltip por defecto con TODOS
# los canales codificados de la marca (x, y, color, text), mostrando de más cosas como
# "color_etiqueta #3A3A37" que no aportan nada al usuario.
# Las filas con formato flecha (ver "etiqueta_ahora" más abajo) vacían "etiqueta" a
# propósito para no duplicar el número: se filtran aquí para no pintar un texto vacío.

```

```

etiquetas = alt.Chart(df[df["etiqueta"] != ""]).mark_text(align="left", baseline="middle",
dx=5, fontSize=11).encode(
    x=alt.X("etiqueta_x:Q", title=x_title, scale=x_scale),
    y=y, text="etiqueta:N",
    color=alt.Color("color_etiqueta:N", scale=None, legend=None),
    tooltip=alt.value(None),
)

layers = []

# What-if: barra "antes" con relleno RAYADO (patrón SVG definido en inject_global_css,
# ver WF_HATCH_ID), tanto en la fila de la variable simulada como en la del total. Va
# DEBAJO de la barra sólida "ahora" en la fila de la variable (misma capa, antes en la
# lista = más al fondo): si ambas van en la misma dirección se solapan y solo asoma el
# rayado donde una es más larga que la otra; si van en direcciones opuestas no se
# tocan y se ven las dos completas. La fila del total no tiene una barra sólida en el
# mismo tramo (el total es una sola barra), así que ahí el rayado simplemente marca
# dónde llegaba el precio final antes de simular. Usa su propia columna "before_start"
# en vez de "start": el "start" de la barra sólida está normalizado con min/max para
# dibujarse bien en cualquier dirección y no siempre coincide con el punto de partida
# real de este paso.
# El fill se pasa como CONSTANTE del mark (no como encoding "color" data-driven):
# probado también como 5ª categoría de la escala/leyenda de "kind" compartida con las
# demás barras, para que Vega-Lite fusionara su entrada en la misma leyenda – pero al
# fusionar la leyenda entre 3 capas (bars_pasos, bars_extremos y esta) Vega tiraba
# categorías del dominio sin avisar ("baja" o "total" desaparecían según el layout),
# con el aviso de consola "Conflicting legend property 'disable'". Como constante no
# participa en ninguna escala/leyenda de Vega, así que la muestra rayada de "antes" se
# añade aparte, en HTML/CSS, justo debajo del gráfico (ver el st.markdown tras
# st.altair_chart en page_prediccion).
if "before_end" in df.columns:
    antes_df = df[df["before_end"].notna()]
    if len(antes_df):
        antes = alt.Chart(antes_df).mark_bar(fill=f"url({WF_HATCH_ID})",
cornerRadiusEnd=3).encode(
            x=alt.X("before_start:Q", scale=x_scale), x2="before_end:Q", y=y,
            tooltip=alt.Tooltip("before_tip:N", title=" "),
        )
        layers.append(antes)

layers += [bars_partida, bars_total, bars_pasos, etiquetas]

# Etiqueta en formato flecha "antes → ahora" para las filas que cambiaron al simular
# (la variable simulada y el total): dos textos pegados uno al otro, "antes → " en
# gris neutro (WF_BEFORE, sin colorear por signo: es solo referencia) seguido del
# valor actual con su color de dirección de siempre (color_etiqueta). Mismo anclaje
# "etiqueta_x" para ambos, cada uno con su propio dx (precalculado en Python a partir
# de la longitud del texto que lo precede) – como Vega-Lite no admite "dx" como canal
# codificado, cada fila necesita su propia capa con su propio dx constante.
if "etiqueta_antes" in df.columns:
    antes_label_df = df[df["etiqueta_antes"].notna()]
    for _, fila in antes_label_df.iterrows():
        fila_df = pd.DataFrame([fila])
        layers.append(
            alt.Chart(fila_df).mark_text(
                align="left", baseline="middle", fontSize=11, color=WF_BEFORE,
                dx=float(fila["dx_antes"]),
            ).encode(
                x=alt.X("etiqueta_x:Q", scale=x_scale),
                y=y, text="etiqueta_antes:N",
                tooltip=alt.value(None),
            )
        )
        layers.append(
            alt.Chart(fila_df).mark_text(
                align="left", baseline="middle", fontSize=11, dx=float(fila["dx_ahora"]),
            ).encode(
                x=alt.X("etiqueta_x:Q", scale=x_scale),
                y=y, text="etiqueta_ahora:N",
                color=alt.Color("color_etiqueta:N", scale=None, legend=None),
                tooltip=alt.value(None),
            )
        )

```

```

return alt.layer(*layers).properties(height=alt.Step(20))

def chart_shap_summary(df, order):
    return (
        alt.Chart(df)
        .mark_circle(size=22, opacity=0.55)
        .encode(
            x=alt.X("shap:Q", title="Impacto en el precio predicho (€)"),
            y=alt.Y(
                "feature:N", sort=order, title=None,
                axis=alt.Axis(labelExpr="datum.label == 'Estancia mínima' ? datum.label + ' *' : datum.label"),
            ),
            yOffset=alt.YOffset("jitter:Q"),
            color=alt.Color(
                "value_norm:Q",
                title="Valor de la variable",
                scale=alt.Scale(domain=[0, 1], range=[BLUE, RED]),
                legend=alt.Legend(orient="top", gradientLength=140, labelExpr="datum.value == 0 ? 'Bajo' : datum.value == 1 ? 'Alto' : ''"),
            ),
            tooltip=[
                alt.Tooltip("feature:N", title="Variable"),
                alt.Tooltip("shap:Q", title="Impacto (€)", format=".1f"),
            ],
        )
        .properties(height=alt.Step(26))
    )

def chart_line_highlight(x_values, y_values, current_x, sim_x, x_title, y_title="Precio predicho (€)"):
    """current_x es el valor guardado en el Formulario, sim_x el del desplegable "Simula un valor": si coinciden (no se simula nada distinto), solo se marca el real."""
    df = pd.DataFrame({"x": x_values, "y": y_values})
    # La curva entera son las alternativas (azul claro); los puntos destacados (real y simulado) van en su propia capa, coloreados igual que en el gráfico de barras del
    # what-if categórico, para que el mismo código de color signifique lo mismo en los dos.
    line = (
        alt.Chart(df)
        .mark_line(point=alt.OverlayMarkDef(color=WHATIF_ALT, size=50), color=WHATIF_ALT)
        .encode(
            x=alt.X("x:Q", title=x_title),
            y=alt.Y("y:Q", title=y_title),
            tooltip=[alt.Tooltip("x:Q", title=x_title), alt.Tooltip("y:Q", title=y_title, format=".0f")],
        )
    )
    highlight_states = [{"x": current_x, "estado": "Tu valor real"}]
    if sim_x != current_x:
        highlight_states.append({"x": sim_x, "estado": "Valor simulado"})
    highlight_df = df.merge(pd.DataFrame(highlight_states), on="x", how="inner")
    point = (
        alt.Chart(highlight_df)
        .mark_point(size=140, filled=True)
        .encode(
            x="x:Q", y="y:Q",
            color=alt.Color(
                "estado:N",
                scale=alt.Scale(domain=["Tu valor real", "Valor simulado"], range=[WHATIF_CURRENT, WHATIF_SIM]),
                legend=alt.Legend(title=None, orient="top"),
            ),
            tooltip=[alt.Tooltip("estado:N", title=""), alt.Tooltip("y:Q", title=y_title, format=".0f")],
        )
    )
    return (line + point).properties(height=220)

model = load_model(CITY)

```

```

comparables_all = load_comparables(CITY)

_default_district = features.DEFAULT_DISTRICT.get(CITY, DISTRICTS[0])
_default_neighbourhood = NEIGHBOURHOODS_BY_DISTRICT[_default_district][0]

DEFAULT_INPUTS = {
    "district": _default_district,
    "neighbourhood": _default_neighbourhood,
    "room_type": ROOM_TYPES[0],
    "property_type": PROPERTY_TYPES[0],
    "accommodates": 4,
    "bedrooms": 1,
    "beds": 2,
    "bathrooms": 1.0,
    "minimum_nights": 2,
    "maximum_nights": 365,
    "availability_365": 300,
    "has_license": True,
    "host_is_superhost": False,
    "host_has_profile_pic": True,
    "host_identity_verified": True,
    "calculated_host_listings_count": 1,
    "host_tenure_years": 0.0,
    "host_user_tenure_years": 0.5,
    "host_entire_homes_ratio": 1.0,
    "is_new_listing": True,
    "latitude": NEIGHBOURHOOD_STATS[_default_neighbourhood]["lat"],
    "longitude": NEIGHBOURHOOD_STATS[_default_neighbourhood]["lon"],
    "has_ac": CITY != "euskadi",
    "has_pool": False,
    "has_dishwasher": False,
    "has_sea_view": False,
    "n_amenities": 30,
}

N_LISTINGS_TEXT = f"{len(comparables_all):,}".replace(",", ".")

_MESES_ES = ["ene", "feb", "mar", "abr", "may", "jun", "jul", "ago", "sep", "oct", "nov", "dic"]

def _fecha_datos_es(iso_date):
    """'2026-09-08' -> 'sep 2026'. None si el modelo no trae fecha en sus metadatos."""
    if not iso_date:
        return None
    year, month = iso_date.split("-")[:2]
    return f"{_MESES_ES[int(month) - 1]} {year}"

# Grupos de color de las burbujas de icono de la home: un único punto de definición
# para que las 5 cards clicables (que comparten estructura y comportamiento) no
# repitan colores sueltos por todo el código.
HOME_GROUP_COLORS = {
    "tu_anuncio": ("FDF0F2", "A83049"), # los 3 pasos
    "datos": ("EDF3F8", "2F5E80"), # Explora el mercado
    "modelo": ("F1F0F8", "4E4A82"), # Cómo funciona el modelo
}

def _home_card_html(icon, group, eyebrow, title, desc):
    bg, fg = HOME_GROUP_COLORS[group]
    eyebrow_html = f'<p class="hc-eyebrow" style="color:{fg};">{html.escape(eyebrow)}</p>' if
    eyebrow else ""
    return (
        f'<div class="hc-row">'
        f'<div class="hc-bubble" style="background:{bg};color:{fg};">'
        f'<span data-testid="stIconMaterial" class="hc-icon">{icon}</span></div>'
        f'<div class="hc-text">{eyebrow_html}'
        f'<p class="hc-titulo">{html.escape(title)}</p>'
        f'<p class="hc-desc">{html.escape(desc)}</p></div></div>'
    )

def _inject_home_css(n_datos_cells, photo_b64):

```

```

bg_photo = f', url("data:image/webp;base64,{photo_b64}")' if photo_b64 else ""
st.markdown(
    f"""<style>
/* Sin título ni subtítulo en esta página (pedido explícito para que las 5 cards
quepan sin scroll): el hueco superior por defecto de .block-container ya no
tiene nada que separar del borde, así que se recorta aquí igual que en la
pantalla de bienvenida. */
.block-container {{ padding-top: 0.8rem; }}
/* Streamlit deja 16px de gap por defecto entre cada bloque de nivel superior de
la página (9 huecos en esta pantalla = 144px solo en separadores). Al dar más
aire a las cards y al bloque de datos (pedido explícito), hacía falta recortar
aquí para seguir cabiendo sin scroll: gap más ajustado solo entre las SECCIONES
de la página, sin tocar el padding/gap interno de cada card. */
.block-container > [data-testid="stVerticalBlock"] {{ gap: 8px !important; }}

/* ---- héroe con la foto de la zona ---- */
.st-key-home_hero {{
    position: relative;
    border-radius: 16px;
    overflow: hidden;
    margin-bottom: 16px;
    height: 260px;
    /* Nada de flex aquí (mismo motivo que en las cards, ver más abajo): con
display:flex, Streamlit calculaba mal el alto del bloque de texto cuando el
párrafo envuelve en varias líneas (max-width:46ch), y el texto invadía el gap
hacia el botón dejándolo muy pegado. Con block, cada hijo mide su alto real y
el espaciado se controla con el margin-top del CTA de abajo. */
    padding: 32px 34px;
    background-size: cover;
    background-position: center;
    background-image: linear-gradient(to right, rgba(20,20,18,.82) 0%, rgba(20,20,18,.55)
52%, rgba(20,20,18,.15) 100%){bg_photo};
}}
.st-key-home_cta {{ margin-top: 20px; }}
.ih-zona {{ text-transform: uppercase; font-size: 12.5px; font-weight: 600; color:
#E4E0D8; margin: 0; }}
.ih-titulo {{ font-size: 30px; font-weight: 700; color: #fff; margin: 0; letter-spacing:
-.01em; }}
.ih-texto {{ font-size: 15px; color: #DAD6CF; max-width: 46ch; margin: 0 0 6px;
line-height: 1.5; }}
.st-key-home_cta a[data-testid="stPageLink-NavLink"] {{
    display: inline-flex; align-items: center; gap: 6px; width: fit-content;
    background: #FF385C; padding: 12px 24px; border-radius: 10px;
    text-decoration: none !important; transition: background .15s ease;
}}
.st-key-home_cta a[data-testid="stPageLink-NavLink"] p {{ color: #fff !important;
font-size: 15.5px; font-weight: 600; margin: 0; }}
.st-key-home_cta a[data-testid="stPageLink-NavLink"]:hover {{ background: #E82E51; }}
/* st.page_link hereda el icono configurado en el st.Page de destino (aquí,
Formulario) cuando no se pasa uno explícito: se oculta porque la maqueta lleva
el botón sin icono, solo la flecha del texto. Se oculta el span que ENVUELVE al
icono, no solo el icono: ese span reserva su propio ancho fijo aunque su
contenido esté en display:none, y el "gap" del enlace seguía sumando espacio
antes del texto – el resultado era el texto visiblemente descentrado dentro
del botón (más hueco a la izquierda que a la derecha). */
.st-key-home_cta a[data-testid="stPageLink-NavLink"] >
span:has([data-testid="stIconMaterial"]) {{
    display: none;
}}

/* ---- rótulos de sección ---- */
.hc-rotulo-seccion {{ font-size: 12px; color: #8B8A85; margin: 0 0 10px; }}

/* ---- grids de cards (pasos y accesos directos) ---- */
.st-key-home_steps_grid [data-testid="stHorizontalBlock"],
.st-key-home_directo_grid [data-testid="stHorizontalBlock"] {{ display: grid; gap: 12px;
}}
.st-key-home_steps_grid [data-testid="stHorizontalBlock"] {{ grid-template-columns:
repeat(3, minmax(0, 1fr)); }}
.st-key-home_directo_grid [data-testid="stHorizontalBlock"] {{ grid-template-columns:
repeat(2, minmax(0, 1fr)); }}
.st-key-home_steps_grid [data-testid="stHorizontalBlock"] > [data-testid="stColumn"],
.st-key-home_directo_grid [data-testid="stHorizontalBlock"] > [data-testid="stColumn"] {{

```

```

width: 100% !important; min-width: 0 !important; flex: none !important;
}}
@media (max-width: 820px) {{
  .st-key-home_steps_grid [data-testid="stHorizontalBlock"],
  .st-key-home_directo_grid [data-testid="stHorizontalBlock"] {{ grid-template-columns:
    1fr; }}
}}

/* ---- las 5 cards clicables: mismo aspecto y comportamiento en todas ---- */
[class*="st-key-home_card_"] {{
  position: relative;
  background: #fff;
  border: 1px solid #E3E0DA;
  border-radius: 12px;
  padding: 26px 24px;
  transition: box-shadow .18s ease, transform .18s ease, border-color .18s ease;
  /* Streamlit hace de la card un flex column (para su propio "gap" entre
  elementos). Con el enlace invisible como segundo hijo, ese modo flex acababa
  calculando mal el alto disponible del primer hijo (el HTML visible se
  quedaba corto y el texto se recortaba contra el padding inferior). No hace
  falta flex aquí (solo hay contenido apilado): con block, el alto es
  simplemente la suma de los hijos, sin ese cálculo raro de por medio. */
  display: block !important;
}}
[class*="st-key-home_card_"]:hover {{
  box-shadow: 0 4px 14px rgba(44,44,42,.10);
  transform: translateY(-2px);
  border-color: #CFCBC2;
}}
[class*="st-key-home_card_"]:active {{ transform: translateY(0); }}
@media (prefers-reduced-motion: reduce) {{
  [class*="st-key-home_card_"] {{ transition: none; }}
  [class*="st-key-home_card_"]:hover {{ transform: none; }}
}}
/* Streamlit pone un margin negativo (-6px arriba y abajo) en sus
.stElementContainer, pensado para el interlineado por defecto entre widgets:
aquí se come parte del padding inferior de la card (el segundo elemento es el
page_link invisible, así que ese margen negativo actúa sobre el propio borde de
la card). Se anula dentro de las cards para que el padding declarado sea el
real. */
[class*="st-key-home_card_"] .stElementContainer {{ margin: 0 !important; }}
.hc-row {{ display: grid; grid-template-columns: 40px 1fr; gap: 18px; align-items: start;
}}
.hc-bubble {{ width: 40px; height: 40px; border-radius: 50%; display: flex; align-items:
center; justify-content: center; }}
.hc-icon {{ font-family: 'Material Symbols Rounded'; font-size: 21px; line-height: 1; }}
.hc-eyebrow {{ font-size: 11px; font-weight: 700; margin: 0 0 6px; }}
.hc-titulo {{ font-size: 15px; font-weight: 600; color: #2C2C2A; margin: 0 0 7px; }}
.hc-desc {{ font-size: 13px; color: #6B6A66; line-height: 1.5; margin: 0; }}
/* Streamlit pone position:relative en los envoltorios del page_link; si no se
anula, alguno de esos envoltorios (que se colapsan a un tamaño mínimo al sacar
el enlace del flujo) pasa a ser el bloque contenedor del enlace absoluto en vez
de la tarjeta entera, y el enlace no llega a cubrirla (visto con computed
width/height casi 0 pese a "100%": un alto en % necesita que el contenedor
tenga una altura explícita, y esos envoltorios no la tienen). */
[class*="st-key-home_card_"] [data-testid="stPageLink"],
[class*="st-key-home_card_"] [data-testid="stPageLink"] > div,
[class*="st-key-home_card_"] .stElementContainer:has(a[data-testid="stPageLink-NavLink"])
{{
  position: static !important;
}}
[class*="st-key-home_card_"] [data-testid="stPageLink-NavLink"] {{
  position: absolute !important; inset: 0; z-index: 2; margin: 0;
  width: 100% !important; height: 100% !important;
  font-size: 0; line-height: 0; color: transparent; text-decoration: none;
}}
/* Igual que en el CTA: el icono heredado del st.Page de destino se ve por
encima del propio icono de la card si no se oculta (el resto del enlace ya
queda invisible por font-size/color, pero el icono trae su propio tamaño). */
[class*="st-key-home_card_"] [data-testid="stPageLink-NavLink"] >
span:has([data-testid="stIconMaterial"]) {{
  display: none;
}}

```

```

[class*="st-key-home_card_"] [data-testid="stPageLink-NavLink"]:focus-visible {{
  outline: 2px solid #FF385C; outline-offset: 2px;
}}

/* ---- datos de la zona: bloque plano de celdas, sin acción ---- */
.st-key-home_datos_grid [data-testid="stHorizontalBlock"] {{
  display: grid;
  grid-template-columns: repeat({n_datos_cells}, minmax(0, 1fr));
  gap: 1px;
  /* Línea divisoria más suave que el gris de los bordes de card: mismo color
  de base pero a menos opacidad, para que separe sin leerse como una tabla. */
  background: rgba(227, 224, 218, .55);
  border-radius: 12px;
  overflow: hidden;
}}
.st-key-home_datos_grid [data-testid="stHorizontalBlock"] > [data-testid="stColumn"] {{
  width: 100% !important; min-width: 0 !important; flex: none !important;
  background: #F5F3EE;
}}
@media (max-width: 820px) {{
  .st-key-home_datos_grid [data-testid="stHorizontalBlock"] {{ grid-template-columns:
  repeat(2, minmax(0, 1fr)); }}
}}
.hd-celda {{ padding: 22px 24px; }}
.hd-rotulo {{ font-size: 11px; color: #6B6A66; margin: 0 0 8px; }}
.hd-valor {{ font-size: 28px; font-weight: 700; letter-spacing: -.01em; color: #2C2C2A;
margin: 0; }}
</style>""",
unsafe_allow_html=True,
)

def page_home():
    geo_unit = features.geo_unit_word(CITY)
    price_mediano = comparables_all["price"].median() if len(comparables_all) else None
    n_geo_units = len(NEIGHBOURHOOD_STATS) if NEIGHBOURHOOD_STATS else None
    fecha_datos = _fecha_datos_es(METADATA.get("date"))

    datos_cells = [
        ("Anuncios analizados", N_LISTINGS_TEXT),
        ("Precio mediano en la zona", f"{price_mediano:.0f} €".replace(",", ".")) if
        price_mediano is not None else None),
        (f"{geo_unit}s cubiertos", f"{n_geo_units:}".replace(",", ".")) if n_geo_units is not
        None else None),
        ("Datos actualizados a", fecha_datos),
    ]
    datos_cells = [(label, value) for label, value in datos_cells if value is not None]

    _inject_home_css(len(datos_cells) or 1, _zone_photo_base64(CITY))

    with st.container(key="home_hero"):
        st.markdown(
            f'<p class="ih-zona">{html.escape(CITY_LABEL.upper())}</p>'
            '<p class="ih-titulo">Pon precio a tu anuncio</p>'
            '<p class="ih-texto">Describe tu alojamiento y te digo a cuánto lo publicaría el
            mercado, '
            f'comparado con {N_LISTINGS_TEXT} anuncios reales de la zona.</p>',
            unsafe_allow_html=True,
        )
    with st.container(key="home_cta"):
        st.page_link(page_formulario_p, label="Empezar →")

    st.markdown('<p class="hc-rotulo-seccion">Cómo funciona</p>', unsafe_allow_html=True)
    with st.container(key="home_steps_grid"):
        steps = [
            ("formulario", page_formulario_p, "edit_note", "PASO 1", "Describe tu anuncio",
            f"{geo_unit}, tamaño, capacidad y condiciones de reserva."),
            ("prediccion", page_prediccion_p, "payments", "PASO 2", "Recibe un precio",
            "Con el margen de error que le corresponde."),
            ("comparar", page_comparar_p, "compare", "PASO 3", "Compáralo",
            "Con anuncios reales parecidos al tuyo."),
        ]
    cols = st.columns(3)

```

```

for col, (slug, target, icon, eyebrow, title, desc) in zip(cols, steps):
    with col:
        with st.container(key=f"home_card_{slug}"):
            st.markdown(_home_card_html(icon, "tu_anuncio", eyebrow, title, desc),
                        unsafe_allow_html=True)
            st.page_link(target, label=title)

if datos_cells:
    st.markdown('<p class="hc-rotulo-seccion">Datos de la zona</p>', unsafe_allow_html=True)
    with st.container(key="home_datos_grid"):
        cols = st.columns(len(datos_cells))
        for col, (label, value) in zip(cols, datos_cells):
            with col:
                st.markdown(
                    f'<div class="hd-celda"><p class="hd-rotulo">{html.escape(label)}</p>'
                    f'<p class="hd-valor">{html.escape(value)}</p></div>',
                    unsafe_allow_html=True,
                )

st.markdown('<p class="hc-rotulo-seccion">0 entra directo</p>', unsafe_allow_html=True)
with st.container(key="home_directo_grid"):
    directos = [
        ("mercado", page_mercado_p, "map", "datos", "Explora el mercado",
         f"Mapa de precios de {CITY_LABEL} por {geo_unit.lower()} y tipo de alojamiento, sin rellenar nada."),
        ("modelo", page_modelo_p, "account_tree", "modelo", "Cómo funciona el modelo",
         f"Qué variables pesan, cuánto acierta y dónde falla el modelo de {CITY_LABEL}."),
    ]
    cols = st.columns(2)
    for col, (slug, target, icon, group, title, desc) in zip(cols, directos):
        with col:
            with st.container(key=f"home_card_{slug}"):
                st.markdown(_home_card_html(icon, group, None, title, desc),
                            unsafe_allow_html=True)
                st.page_link(target, label=title)

def page_modelo():
    st.header("Cómo funciona el modelo")

    with st.container(border=True, key="section_modelo_que_es"):
        st.markdown("#### :material/account_tree: Qué es el modelo")
        st.markdown(
            "El precio que ves no sale de una fórmula fija ni de una media simple: lo calcula un modelo "
            f"entrenado con más de {N_LISTINGS_TEXT} anuncios reales de Airbnb en {CITY_LABEL}, que ha aprendido "
            "qué combinaciones de ubicación, tamaño, condiciones de reserva e historial del anfitrión se han "
            "pagado realmente en la ciudad."
        )
        st.caption(f"En detalle técnico, es un {METADATA['model']} (una familia de modelos de árboles de decisión).")

    with st.container(border=True, key="section_modelo_acierto"):
        _inject_acierto_css()
        st.markdown("#### :material/track_changes: Cuánto acierta")

        r2_pct = METADATA["test_metrics"]["R2_log"] * 100
        st.markdown(
            f'<div style="background:#2C2C2A;border-radius:12px;padding:24px 26px;margin-bottom:30px;">'
            f'display:grid;grid-template-columns:auto 1fr;gap:26px;align-items:center;">'
            f'<div style="font-size:54px;font-weight:700;color:#fff;letter-spacing:-.035em;line-height:1;">'
            f'{r2_pct:.0f}&nbsp;%</div>'
            f'<div style="font-size:15px;line-height:1.6;color:#CFCCC5;max-width:46ch;">'
            f'El modelo <strong style="color:#fff;font-weight:600;">explica un {r2_pct:.0f}&nbsp;%</strong> de '
            f'las diferencias de precio entre anuncios. El resto depende de factores que no están en los datos.'
            f'</div></div>',
            unsafe_allow_html=True,

```

```

)

segments, edges = load_price_segment_errors(CITY)
# Si a la zona le faltan datos de algún tramo (test set demasiado pequeño para que
# qcut complete N_PRICE_SEGMENTS tramos con contenido), se omite la tira entera en
# vez de enseñar un hueco o un 0 inventado: solo queda el bloque del R2 de arriba.
if segments["mae"].isna().any() or len(segments) < N_PRICE_SEGMENTS:
    st.caption(
        "No hay datos de test suficientes en esta zona para desglosar el margen de error
        por tramo de "
        "precio."
    )
else:
    st.markdown(
        "##### Y este es el margen de error del precio predicho por el modelo",
    )
    st.markdown(
        '<p style="font-size:12px;color:#8B8A85;margin:0 0 8px;">Según el precio del
        anuncio</p>',
        unsafe_allow_html=True,
    )

    has_prediction = "confirmed_inputs" in st.session_state
    highlight_idx = price_segment_index(edges, pred) if has_prediction else None
    st.markdown(
        render_price_error_stripe(segments, edges, highlight_idx, pred if has_prediction
        else None),
        unsafe_allow_html=True,
    )

    if not has_prediction:
        st.markdown(
            '<p style="font-size:13px;color:#8B8A85;margin-top:8px;">Cuando calcules el
            precio de tu '
            'anuncio, se marcará aquí el tramo que le corresponde.</p>',
            unsafe_allow_html=True,
        )

    elif highlight_idx == len(segments) - 1:
        st.markdown(
            '<div style="border:1px solid #E8D6D4;background:#FCF6F5;border-radius:11px;'
            'padding:15px
            18px;margin-top:14px;font-size:13.5px;line-height:1.6;color:#7A3F3B;">'
            'Tu anuncio cae en el tramo más alto de precio. Ahí el margen de error es
            mucho más amplio '
            'porque ese tramo reúne casos muy distintos entre sí (desde pisos algo más
            caros hasta '
            'villas de lujo), y el modelo tiene menos anuncios parecidos en los que
            apoyarse. Toma esta '
            'recomendación como orientativa, no como un precio cerrado.</div>',
            unsafe_allow_html=True,
        )

    with st.expander("Cómo se calcula"):
        st.markdown(
            '<div style="font-size:13.5px;line-height:1.6;color:#6B6A66;">'
            "Medido sobre anuncios de test que el modelo no vio al entrenar, agrupados
            por su precio "
            "real. El modelo aprende a predecir el precio en escala logarítmica: el % de
            arriba se mide "
            "en esa misma escala, y el margen en euros de cada tramo se calcula después
            sobre el precio "
            "real. Por eso mismo, ese % tampoco es comparable entre zonas – cada mercado
            tiene su propia "
            "dispersión de precios, así que un mismo porcentaje en dos zonas no
            representa el mismo "
            "margen en euros.</div>",
            unsafe_allow_html=True,
        )

    with st.container(border=True, key="section_modelo_fiabilidad"):
        st.markdown("#### :material/balance: Dónde es más y menos fiable")
        st.markdown(
            "Ningún modelo acierta siempre, y prefiere decírtelo claro a llevarte una sorpresa:"

```

```

)
_low_sample_text = " o ".join(features.LOW_SAMPLE_DISTRICTS.get(CITY, ())[2])
st.markdown(
    "- **Alojamientos muy grandes** (10 o más huéspedes): hay muy pocos en los datos de
    entrenamiento, "
    "así que el modelo tiene menos referencias para acertar en este tipo de anuncios.\n"
    f"- **Distritos con pocos anuncios**, como {_low_sample_text}: al haber menos
    ejemplos, "
    "la predicción es menos precisa que en distritos con más oferta."
)
st.caption(
    "La app te avisa automáticamente de estos casos en la página de Predicción, con una
    señal de confianza."
)
st.caption(
    "Sesgo medio del modelo en anuncios reales de test que no usó para entrenar (precio
    predicho menos "
    "precio real). Un valor negativo grande significa que el modelo tiende a quedarse
    corto en ese grupo, "
    "no que tu anuncio concreto vaya a fallar igual."
)
eval_df = load_test_evaluation(CITY)
col_dist, col_quint = st.columns(2)
with col_dist:
    by_district = eval_df.groupby("district")["residual"].agg(["mean",
    "count"]).reset_index()
    district_labels = [f"{row.district} ({row.count})" for row in
    by_district.iterrows()]
    chart = chart_bias_hbar(district_labels, by_district["mean"], x_title="Sesgo medio
    (€)")
    st.altair_chart(chart, use_container_width=True)
    st.caption("Entre paréntesis, nº de anuncios de test en ese distrito.")
with col_quint:
    quintile_order = _PRICE_QUINTILE_ORDER
    by_quintile = eval_df.groupby("quintile", sort=False).agg(
        sesgo=("residual", "mean"), precio_min=("price", "min"), precio_max=("price",
        "max")
    ).reindex(quintile_order)
    quintile_labels = [
        f"{q} · {int(row.precio_min)}-{int(row.precio_max)}€" for q, row in
        by_quintile.iterrows()
    ]
    chart = chart_bias_hbar(
        quintile_labels, by_quintile["sesgo"], x_title="Sesgo medio (€)",
        sort=list(reversed(quintile_labels)),
    )
    st.altair_chart(chart, use_container_width=True)
    st.caption("Quintiles de precio real: Q1 = 20% más baratos, Q5 = 20% más caros.")

mn_all = comparables_all["minimum_nights"]
price_all = comparables_all["price"]
bins = [0, 7, 30, 90, 10_000]
tramo_labels = ["1-7 noches", "8-30 noches", "31-90 noches", "90+ noches"]
grupo_by_tramo = {
    "1-7 noches": "Turístico", "8-30 noches": "Zona de transición",
    "31-90 noches": "Temporada", "90+ noches": "Temporada",
}
tramo = pd.cut(mn_all, bins=bins, labels=tramo_labels)
bins_df = pd.DataFrame({"tramo": tramo, "price": price_all})
bins_summary = bins_df.groupby("tramo", observed=True).agg(
    n=("price", "size"), mediana_precio=("price", "median")
).reindex(tramo_labels).reset_index()
bins_summary["grupo"] = bins_summary["tramo"].map(grupo_by_tramo)

pct_turistico = float((mn_all <= 30).mean() * 100)
pct_temporada = 100 - pct_turistico
mediana_turistico = float(price_all[mn_all <= 30].median())
mediana_temporada = float(price_all[mn_all >= 31].median())
ratio_precio = mediana_turistico / mediana_temporada
n_temporada = int(bins_summary.loc[bins_summary["grupo"] == "Temporada", "n"].sum())
importancia_mn = next(
    f["importance"] for f in load_feature_importance(CITY) if f["feature"] == "Estancia
    mínima"

```

```

)

# La estructura de esta sección es distinta por ciudad a propósito, no solo los
# números: en Barcelona los datos muestran de verdad dos mercados de tamaño
# comparable con un salto de precio brusco justo en la frontera legal de 31
# noches (63%/36%, precio 3.2 veces mayor por debajo del corte, sin apenas
# anuncios en la banda intermedia de 8 a 30 noches). En Madrid esa frontera no
# existe: el 95% del catálogo es turístico y el precio baja de forma progresiva
# según crece la estancia mínima exigida, sin ningún salto en las 30 noches, así
# que forzar la misma etiqueta de "dos mercados" contradiría los propios datos
# de Madrid. Se comprobó también la pista de las licencias como posible frontera
# legal equivalente para Madrid y se descartó: el grupo de temporada tiene más
# licencia (81%), no menos, que el turístico (72%), justo lo contrario de lo que
# esa explicación necesitaría.
if CITY == "barcelona":
    with st.container(border=True, key="section_modelo_mercados"):
        st.markdown("#### :material/call_split: Dos mercados en un mismo modelo")
        st.markdown(
            "Hay una razón de fondo por la que el modelo se apoya tanto en la estancia
            mínima. Dentro de "
            "estos anuncios conviven dos negocios distintos, aunque el dato en bruto no los
            separe. El "
            "turístico, con estancia mínima de hasta 30 noches, compite con hoteles y tiene
            precios por "
            "noche altos. El de temporada, de 31 noches en adelante, compite con el alquiler
            residencial "
            "normal y tiene precios por noche mucho más bajos. La frontera no es casualidad:
            en Barcelona, "
            "un alquiler de 31 noches o más deja de considerarse vivienda de uso turístico a
            efectos "
            "legales, y con la retirada de las licencias turísticas prevista para 2028 esta
            distinción va "
            "a pesar todavía más."
        )
    st.altair_chart(chart_market_bins(bins_summary), use_container_width=True)
    st.caption(
        f"Dos mercados de tamaño comparable ({pct_turistico:.0f}% turístico,
        {pct_temporada:.0f}% "
        f"temporada) con una diferencia de precio muy marcada ({mediana_turistico:.0f}€
        frente a "
        f"{mediana_temporada:.0f}€, {ratio_precio:.1f} veces más caro). Por eso la
        estancia mínima es, "
        f"con diferencia, la variable más importante del modelo
        ({importancia_mn*100:.0f}% de la "
        "importancia total). Un modelo separado por mercado no mejora la precisión porque
        el de "
        "árboles ya distingue el mercado internamente."
    )
elif CITY == "malaga":
    with st.container(border=True, key="section_modelo_mercados"):
        st.markdown("#### :material/trending_down: Un mercado turístico casi absoluto, sin
        gradiente limpio")
        st.markdown(
            "Aquí el turístico no es solo dominante, es casi todo el catálogo: el 98% de los
            anuncios tiene "
            "estancia mínima de hasta 30 noches, dejando la temporada como una franja
            marginal – todavía más "
            "extremo que en Madrid. Pero, a diferencia de Madrid, dentro de ese 98% el precio
            no baja de "
            "forma progresiva según crece la estancia exigida: se mantiene alto y estable
            entre 1 y 7 noches "
            "(en torno a 170€), y solo cae con fuerza al cruzar esa semana, sin una pendiente
            suave noche a "
            "noche. El salto grande de precio está en la frontera con la temporada, no dentro
            del propio "
            "mercado turístico."
        )
    st.altair_chart(chart_market_bins(bins_summary), use_container_width=True)
    st.caption(
        f"El mercado turístico domina casi todo ({pct_turistico:.0f}% frente a
        {pct_temporada:.0f}% de "
        f"temporada), con una diferencia de precio muy marcada entre ambos
        ({mediana_turistico:.0f}€ "

```

```

f"frente a {mediana_temporada:.0f}€, {ratio_precio:.1f} veces) – más pronunciada
incluso que en "
f"Barcelona. La estancia mínima conserva el primer puesto en importancia
({importancia_mn*100:.0f}% "
"del total), coherente con ese salto tan marcado. "
f"Tampoco compensa un modelo separado, porque la muestra de temporada
({n_temporada} anuncios) "
"es demasiado pequeña para entrenar uno propio."
)
elif CITY == "sevilla":
    with st.container(border=True, key="section_modelo_mercados"):
        st.markdown("#### :material/trending_down: Un mercado turístico casi absoluto, con un
        precio estable hasta la semana")
        st.markdown(
            "Aquí el turístico es prácticamente todo el catálogo: el 98% de los anuncios
            tiene estancia "
            "mínima de hasta 30 noches, un nivel de concentración muy similar al de Málaga y
            muy por encima "
            "de Madrid/Valencia. Dentro de ese 98%, el precio se mantiene relativamente
            estable entre 1 y 7 "
            "noches (con algo de ruido en 5-7 noches por el tamaño de muestra, ya pequeño
            ahí), y solo cae "
            "con fuerza en cuanto se exige más de una semana. El salto grande de precio está,
            de nuevo, en "
            "la frontera con la temporada, no en una pendiente progresiva dentro del propio
            turístico."
        )
        st.altair_chart(chart_market_bins(bins_summary), use_container_width=True)
        st.caption(
            f"El mercado turístico domina casi todo ({pct_turistico:.0f}% frente a
            {pct_temporada:.0f}% de "
            f"temporada), con la diferencia de precio más marcada de las cinco ciudades entre
            ambos "
            f"({mediana_turistico:.0f}€ frente a {mediana_temporada:.0f}€, {ratio_precio:.1f}
            veces). La "
            f"estancia mínima conserva el primer puesto en importancia
            ({importancia_mn*100:.0f}% del "
            "total), coherente con ese salto tan marcado. "
            f"Tampoco compensa un modelo separado, porque la muestra de temporada
            ({n_temporada} anuncios) "
            "es demasiado pequeña para entrenar uno propio."
        )
    )
elif CITY == "mallorca":
    with st.container(border=True, key="section_modelo_mercados"):
        st.markdown("#### :material/trending_down: Un mercado turístico casi absoluto, con la
        frontera real en la semana, no en el mes")
        st.markdown(
            "El turístico domina casi todo el catálogo (el 98% de los anuncios tiene estancia
            mínima de "
            "hasta 30 noches, un nivel similar a Málaga/Sevilla), pero ese 98% esconde una
            frontera propia "
            "que no coincide con el límite legal de 30 noches: el precio se mantiene alto
            mientras basta con "
            "reservar de 1 a 7 noches (villas y apartamentos turísticos, en torno a 400€), y
            cae en picado en "
            "cuanto el anuncio exige entre 8 y 30 noches, hasta un nivel casi idéntico al de
            la verdadera "
            "temporada (31+ noches). En la práctica, Mallorca tiene su propia frontera de
            negocio en la "
            "semana, no en el mes: por encima de 7 noches exigidas, el anuncio ya se comporta
            como alquiler "
            "de temporada aunque legalmente siga contando como turístico."
        )
        st.altair_chart(chart_market_bins(bins_summary), use_container_width=True)
        st.caption(
            f"El mercado turístico domina casi todo ({pct_turistico:.0f}% frente a
            {pct_temporada:.0f}% de "
            f"temporada), con una diferencia de precio muy marcada entre ambos
            ({mediana_turistico:.0f}€ "
            f"frente a {mediana_temporada:.0f}€, {ratio_precio:.1f} veces) – pero esa
            diferencia ya aparece "
            "dentro del propio 98% turístico, entre 1-7 noches y 8-30 noches, no solo en la
            frontera legal. "
        )
    )

```

```

        f"La estancia mínima conserva el primer puesto en importancia
        ({importancia_mn*100:.0f}% del "
        "total). "
        f"Tampoco compensa un modelo separado, porque la muestra de temporada
        ({n_temporada} anuncios) "
        "es demasiado pequeña para entrenar uno propio."
    )
elif CITY == "euskadi":
    with st.container(border=True, key="section_modelo_mercados"):
        st.markdown("#### :material/trending_down: Un mercado turístico casi absoluto, con un
        escalón ya dentro del propio turístico")
        st.markdown(
            "El turístico domina casi todo el catálogo (el 97% de los anuncios tiene estancia
            mínima de "
            "hasta 30 noches, un nivel similar a Málaga/Sevilla). Pero, como en Mallorca, ese
            97% no es "
            "uniforme: el precio se mantiene alto mientras basta con reservar de 1 a 7 noches
            (en torno a "
            "200€), y cae con fuerza en la banda de 8 a 30 noches (en torno a 122€) – un
            nivel ya mucho más "
            "cercano al de la verdadera temporada (96€, 31+ noches) que al del turístico de
            estancia corta. "
            "El escalón es menos brusco que en Mallorca, pero apunta en la misma dirección:
            la semana pesa "
            "más que el límite legal de 30 noches a la hora de fijar el precio."
        )
        st.altair_chart(chart_market_bins(bins_summary), use_container_width=True)
        st.caption(
            f"El mercado turístico domina casi todo ({pct_turistico:.0f}% frente a
            {pct_temporada:.0f}% de "
            f"temporada), con una diferencia de precio notable entre ambos
            ({mediana_turistico:.0f}€ frente "
            f"a {mediana_temporada:.0f}€, {ratio_precio:.1f} veces) – pero buena parte de esa
            caída ya "
            "ocurre dentro del propio turístico, entre 1-7 noches y 8-30 noches. "
            f"La estancia mínima conserva el primer puesto en importancia
            ({importancia_mn*100:.0f}% del "
            "total). "
            f"Tampoco compensa un modelo separado, porque la muestra de temporada
            ({n_temporada} anuncios) "
            "es demasiado pequeña para entrenar uno propio."
        )
else:
    with st.container(border=True, key="section_modelo_mercados"):
        st.markdown("#### :material/trending_down: Un mercado dominante, con un gradiente de
        precio")
        st.markdown(
            "Aquí no hay dos mercados comparables. El 95% de los anuncios tiene una estancia
            mínima de "
            "hasta 30 noches, así que casi todo el catálogo es turístico y el alquiler de
            temporada es una "
            "cola marginal. Sin embargo, dentro de ese 95% el precio no es plano: baja de
            forma progresiva "
            "según crece la estancia mínima exigida, desde unos 135€ por noche cuando basta
            con reservar "
            "una sola noche hasta unos 80€ cuando ya hace falta comprometerse a dos o tres
            semanas. Ese "
            "gradiente, y no una frontera legal entre dos negocios como en Barcelona, es lo
            que explica "
            "que la estancia mínima siga siendo la variable más importante del modelo, aunque
            con bastante "
            "menos peso que en Barcelona."
        )
        st.altair_chart(chart_market_bins(bins_summary), use_container_width=True)
        st.caption(
            f"El mercado turístico domina casi todo ({pct_turistico:.0f}% frente a
            {pct_temporada:.0f}% de "
            f"temporada) y la diferencia de precio entre ambos es mucho más suave
            ({mediana_turistico:.0f}€ "
            f"frente a {mediana_temporada:.0f}€, {ratio_precio:.1f} veces). La estancia
            mínima conserva el "
            f"primer puesto en importancia, pero con un {importancia_mn*100:.0f}% frente al
            63% de "

```

```

        "Barcelona, y aquí el tipo de alojamiento y el número de huéspedes pesan
        proporcionalmente "
        "mucho más. "
        f"Tampoco compensa un modelo separado, porque la muestra de temporada
        ({n_temporada} anuncios) "
        "es demasiado pequeña para entrenar uno propio."
    )

def page_variabales():
    st.header("Variables del modelo")

    with st.container(key="variable_catalog"):
        n_cols = 3
        col_left, col_right = st.columns(2)
        target_col = {0: col_left, 1: col_left, 2: col_right, 3: col_right}
        for idx, (category, icon, variables) in enumerate(FEATURE_CATALOG):
            with target_col[idx]:
                with st.container(border=True, key=f"section_catalog_{idx}"):
                    st.markdown(f"#### :material/{icon}: {category}")
                    rows = [variables[i:i + n_cols] for i in range(0, len(variables), n_cols)]
                    for row in rows:
                        cols = st.columns(n_cols)
                        for col, (name, desc, rng) in zip(cols, row):
                            with col:
                                st.metric(name, rng, help=desc, border=True)

def page_analisis():
    st.header("Análisis del modelo")

    col_imp, col_pdp = st.columns(2)
    with col_imp:
        # Con las 12 variables por defecto el contenido cabe de sobra en 580px: fijar esa
        # altura ahí solo metía una barra de scroll interna sin motivo. El scroll solo tiene
        # sentido cuando se despliegan las variables restantes y el contenido ya no cabe.
        imp_height = 580 if st.session_state.get("show_all_importance") else "content"
        with st.container(border=True, key="section_analisis_importancia", height=imp_height):
            st.markdown("#### :material/bar_chart: Qué variables pesan más")
            importance = load_feature_importance(CITY)
            top_n_imp = 12
            st.caption(
                f"Top {top_n_imp} de las {len(importance)} variables (agrupa, por ejemplo,
                distrito en una sola "
                "barra). Foto general del modelo, no de tu anuncio."
            )
            show_all_imp = st.checkbox(f"Ver las {len(importance)} variables",
                key="show_all_importance")
            shown_importance = importance if show_all_imp else importance[:top_n_imp]
            chart = chart_hbar(
                [row["feature"] for row in shown_importance],
                [row["importance"] for row in shown_importance],
                x_title="Importancia (caída de R²)",
            )
            st.altair_chart(chart, use_container_width=True)

    with col_pdp:
        with st.container(border=True, key="section_analisis_pdp", height=580):
            st.markdown("#### :material/show_chart: Cómo afecta cada variable")
            st.caption(
                "Precio medio predicho si todos los anuncios tuvieran ese valor. Abajo, cuántos
                anuncios reales "
                "respaldan cada tramo: pocos anuncios = tramo poco fiable."
            )
            pdp_choice = st.selectbox("Variable a explorar", list(PDP_GROUPS.keys()),
                key="pdp_choice")
            X_test_matrix = load_test_matrix(CITY)
            pdp_values, pdp_results, pdp_kind = pdp_curve(model, X_test_matrix, pdp_choice)
            raw_values, _ = pdp_axis_values(pdp_choice)

            if pdp_kind == "numeric":
                real_values = pdp_real_values(X_test_matrix, pdp_choice)
                # La curva PDP se muestrea en puntos fijos (WHATIF_NUMERIC_VALUES) que llegan

```

```

# muy lejos a propósito, para el simulador de Predicción. Aquí, para no gastar
# medio ancho del gráfico en una cola sin apenas anuncios reales, se recorta el
# eje X al rango donde de verdad hay datos (percentil 97 de los valores reales),
# sin tocar el cálculo de la curva en sí: los puntos por encima simplemente no
# se dibujan, igual que ya se documenta en la nota de "poco fiable" de arriba.
x_max = float(real_values.quantile(0.97))
x_min = float(real_values.min())
if x_max > x_min:
    pdp_values_plot = [v for v in pdp_values if v <= x_max]
    pdp_results_plot = [r for v, r in zip(pdp_values, pdp_results) if v <= x_max]
    real_values_plot = real_values[real_values <= x_max]
else:
    pdp_values_plot, pdp_results_plot, real_values_plot = pdp_values,
    pdp_results, real_values

line_chart = (
    alt.Chart(pd.DataFrame({"x": pdp_values_plot, "y": pdp_results_plot}))
    .mark_line(point=alt.OverlayMarkDef(color=BLUE, size=50), color=BLUE)
    .encode(
        x=alt.X("x:Q", title=None, axis=alt.Axis(labels=False, ticks=False)),
        y=alt.Y("y:Q", title="Precio medio predicho (€)",
            tooltip=[alt.Tooltip("x:Q", title=pdp_choice), alt.Tooltip("y:Q",
                title="Precio medio", format=".0f")],
        )
    .properties(height=140)
)

# Variables de conteo (Dormitorios, Huéspedes...) son enteras de verdad: sin
esto,
# el bineado automático de Vega puede etiquetar el eje con decimales ("1.0",
"3.0"... )
# que no existen en la variable. Las genuinamente continuas (Baños...) sí los
conservan.
is_integer_valued = (real_values.dropna() % 1 == 0).all()
hist_axis = alt.Axis(format="d") if is_integer_valued else alt.Axis()
hist_chart = (
    alt.Chart(pd.DataFrame({"x": real_values_plot}))
    .mark_bar(color=BLUE, opacity=0.45)
    .encode(
        x=alt.X("x:Q", title=pdp_choice, bin=alt.Bin(maxbins=25),
            axis=hist_axis),
        y=alt.Y("count():Q", title="Nº anuncios",
            tooltip=[alt.Tooltip("count():Q", title="Nº anuncios")],
        )
    .properties(height=95)
)

pdp_chart = alt.vconcat(line_chart, hist_chart,
    spacing=4).resolve_scale(x="shared")
else:
    counts = pdp_category_counts(X_test_matrix, pdp_choice, raw_values)
    labels_with_counts = [f"{lbl} ({c})" for lbl, c in zip(pdp_values, counts)]
    pdp_chart = chart_hbar(labels_with_counts, pdp_results, x_title="Precio medio
    predicho (€)")
st.altair_chart(pdp_chart, use_container_width=True)
if pdp_choice == "Estancia mínima":
    if CITY == "malaga":
        st.caption(
            "Casi todo el histograma se concentra en 1-3 noches, con un pico
            secundario pequeño en "
            "7 (el corte semanal habitual) y una cola larga y fina hacia estancias
            más exigentes – "
            "no dos mercados de tamaño comparable como en Barcelona. Más detalle en
            **Modelo**."
        )
    elif CITY == "sevilla":
        st.caption(
            "Casi todo el histograma se concentra en 1-2 noches, con una cola larga y
            fina hacia "
            "estancias más exigentes donde el precio cae con fuerza – no dos mercados
            de tamaño "
            "comparable como en Barcelona. Más detalle en **Modelo**."
        )
    elif CITY == "mallorca":
        st.caption(

```

```

        "Casi dos tercios del histograma se concentra en 1-3 noches (con picos
        también en 5 y 7, "
        "cortes semanales habituales), y la curva se desploma justo al cruzar la
        semana – la "
        "frontera real está en 7 noches, no en el límite legal de 30. Más detalle
        en **Modelo**."
    )
    elif CITY == "euskadi":
        st.caption(
            "La gran mayoría del histograma se concentra en 1-3 noches, y la curva ya
            baja de forma "
            "notable entre 7 y 8 noches – un escalón más suave que en Mallorca, pero
            en la misma "
            "dirección: la semana pesa más que el límite legal de 30 noches. Más
            detalle en **Modelo**."
        )
    else:
        st.caption(
            "El desplome de la curva y los dos picos del histograma reflejan los dos
            mercados "
            "(turístico vs temporada). Más detalle en **Modelo**."
        )

with st.container(border=True, key="section_analisis_shap"):
    st.markdown("#### :material/scatter_plot: Impacto y dirección de cada variable")
    shap_df, shap_order = load_shap_summary(CITY)
    top_n_shap = 12
    st.caption(
        f"Top {top_n_shap} de las {len(shap_order)} variables numéricas o Sí/No con más
        impacto. Cada punto es "
        "un anuncio de test: su posición indica cuánto sube o baja el precio predicho, y el
        color si el valor "
        "de la variable era bajo (azul) o alto (rojo). Las agrupadas (distrito, tipo...) ya
        están en el "
        "ranking de importancia de la izquierda."
    )
    show_all_shap = st.checkbox(f"Ver las {len(shap_order)} variables", key="show_all_shap")
    shown_labels = shap_order if show_all_shap else shap_order[:top_n_shap]
    shap_chart = chart_shap_summary(shap_df[shap_df["feature"].isin(shown_labels)],
    shown_labels)
    st.altair_chart(shap_chart, use_container_width=True)
    if "Estancia mínima" in shown_labels:
        if CITY == "malaga":
            st.caption(
                "\\* Su gran dispersión refleja el salto de precio entre el mercado turístico
                (1-7 noches) y "
                "todo lo demás, no dos mercados de tamaño comparable. Más detalle en
                **Modelo**."
            )
        elif CITY == "sevilla":
            st.caption(
                "\\* Su gran dispersión refleja el salto de precio entre el mercado turístico
                (1-7 noches) y "
                "todo lo demás, no dos mercados de tamaño comparable. Más detalle en
                **Modelo**."
            )
        elif CITY == "mallorca":
            st.caption(
                "\\* Su gran dispersión refleja el salto de precio entre 1-7 noches y 8+
                noches, la frontera "
                "real del mercado turístico en Mallorca – no el límite legal de 30 noches.
                Más detalle en **Modelo**."
            )
        elif CITY == "euskadi":
            st.caption(
                "\\* Su gran dispersión refleja el salto de precio entre 1-7 noches y 8+
                noches, un escalón "
                "ya dentro del propio mercado turístico. Más detalle en **Modelo**."
            )
        else:
            st.caption(
                "\\* Su gran dispersión refleja los dos mercados (turístico vs temporada).
                Más detalle en **Modelo**."
            )

```

```

    )

with st.container(border=True, key="section_analisis_interaccion"):
    st.markdown("#### :material/grid_on: Interacción entre dos variables")
    st.caption(
        "Precio medio predicho para cada combinación de valores de las dos variables  

        elegidas. Si el color "  

        "cambia igual de forma vertical en todas las columnas, las variables no interactúan:  

        el efecto de una "  

        "no depende de la otra. Si el patrón cambia de una columna a otra, sí hay  

        interacción."
    )

    group_names = list(PDP_GROUPS.keys())
    col_chart, col_sel = st.columns([2, 1.1])
    with col_sel:
        inter_a = st.selectbox("Variable en eje X", group_names, index=0, key="inter_a")
        options_b = [g for g in group_names if g != inter_a]
        inter_b = st.selectbox("Variable en eje Y", options_b, index=0, key="inter_b")

    grid_df, order_a, order_b = pdp_surface(model, X_test_matrix, inter_a, inter_b)
    heatmap = (
        alt.Chart(grid_df)
        .mark_rect()
        .encode(
            x=alt.X("a:0", title=inter_a, sort=order_a, axis=alt.Axis(labelAngle=0)),
            y=alt.Y("b:0", title=inter_b, sort=order_b),
            color=alt.Color(
                "price:Q", title="Precio medio (€)",
                scale=alt.Scale(scheme="viridis"),
            ),
            tooltip=[
                alt.Tooltip("a:N", title=inter_a),
                alt.Tooltip("b:N", title=inter_b),
                alt.Tooltip("price:Q", title="Precio medio", format=".0f"),
            ],
        )
        .properties(height=380)
    )
    with col_chart:
        st.altair_chart(heatmap, use_container_width=True)

    ratio = interaction_strength(grid_df)
    if ratio > 0.25:
        st.warning(
            f"***Interacción fuerte** entre estas dos variables (índice {ratio:.2f}): el  

            efecto de una "  

            f"cambia claramente según el valor de la otra. El patrón de la curva individual  

            de arriba no "  

            f"cuenta toda la historia aquí."
        )
    elif ratio > 0.1:
        st.info(f"***Interacción moderada** (índice {ratio:.2f}): hay algo de dependencia  

        entre ambas, pero no domina el resultado.")
    else:
        st.success(f"***Poca o ninguna interacción** (índice {ratio:.2f}): cada variable actúa  

        de forma bastante independiente de la otra.")
    st.caption(
        "Índice: cuánto se aleja la superficie real de la que habría si cada variable sumara  

        su efecto por "  

        "separado, sin combinarse (0 = totalmente independientes, más alto = más  

        interacción). Es una "  

        "heurística orientativa, no un test estadístico formal."
    )

def page_formulario():
    st.header("Características del anuncio")

    col_a, col_b = st.columns(2)

    with col_a:
        with st.container(border=True, key="section_form_ubicacion"):

```

```

st.markdown("#### :material/location_on: Ubicación")
if CITY == "barcelona":
    _district_help = (
        "Distrito de Barcelona donde está el alojamiento. Es una de las variables con
        más peso "
        "en el precio:\n\n"
        "- **Ciutat Vella**: casco antiguo, muy turístico\n"
        "- **Eixample**: zona comercial planificada, muy demandada\n"
        "- **Gràcia**: barrio bohemio, cerca del centro\n"
        "- **Sants-Montjuïc**: mixto, cerca de Montjuïc\n"
        "- **Les Corts**: residencial, zona universitaria\n"
        "- **Sarrià-Sant Gervasi**: residencial de nivel alto\n"
        "- **Sant Martí**: playa y Poblenou, en expansión\n"
        "- **Sant Andreu**: residencial, tranquilo\n"
        "- **Horta-Guinardó**: residencial, pocos anuncios\n"
        "- **Nou Barris**: residencial, el más económico"
    )
else:
    _district_help = (
        f"Distrito de {CITY_LABEL} donde está el alojamiento. Es una de las variables
        con más "
        "peso en el precio.\n\nValores posibles:\n\n" + bullet_list(DISTRICTS)
    )
if HAS_NEIGHBOURHOOD_LEVEL:
    dist_col, neigh_col = st.columns(2)
    with dist_col:
        district = st.selectbox(
            "Distrito", DISTRICTS,
            index=DISTRICTS.index(features.DEFAULT_DISTRICT.get(CITY, DISTRICTS[0])),
            help=_district_help,
        )
    with neigh_col:
        neighbourhood = st.selectbox(
            "Barrio", NEIGHBOURHOODS_BY_DISTRICT[district],
            help="Barrio dentro del distrito elegido. El modelo usa el precio medio
            histórico de cada barrio como variable.",
        )
else:
    # Sin nivel de barrio (ver features.HAS_NEIGHBOURHOOD_LEVEL): un único
    # desplegable de distrito, sin duplicar el mismo dato en dos campos.
    district = st.selectbox(
        "Distrito", DISTRICTS,
        index=DISTRICTS.index(features.DEFAULT_DISTRICT.get(CITY, DISTRICTS[0])),
        help=_district_help,
    )
    neighbourhood = NEIGHBOURHOODS_BY_DISTRICT[district][0]
st.caption(":material/pin_drop: Ajustar coordenadas exactas")
stats = NEIGHBOURHOOD_STATS[neighbourhood]
if (
    st.session_state.get("coord_neighbourhood") != neighbourhood
    or "coord_lat" not in st.session_state
    or "coord_lon" not in st.session_state
):
    # "coord_lat"/"coord_lon" son claves de widgets (los number_input de abajo):
    # Streamlit las elimina de session_state en cuanto sales de esta página, así
    # que hay que volver a poblarlas al entrar, no solo cuando cambia el barrio.
    # Redondeadas a 5 decimales: ~1m de precisión, de sobra para un anuncio, y
    # así el valor cabe en el campo sin cortarse (6 decimales no cabían).
    st.session_state["coord_neighbourhood"] = neighbourhood
    st.session_state["coord_lat"] = round(stats["lat"], 5)
    st.session_state["coord_lon"] = round(stats["lon"], 5)

@st.dialog("Ampliar mapa", width="large")
def _expand_map_dialog():
    st.caption("Haz clic en el mapa para fijar la ubicación exacta del anuncio.")
    big_map = folium.Map(
        location=[st.session_state["coord_lat"], st.session_state["coord_lon"]],
        zoom_start=16,
        tiles="OpenStreetMap",
    )
    folium.Marker(
        [st.session_state["coord_lat"], st.session_state["coord_lon"]],
        icon=folium.Icon(color="red", icon="home"),
    )

```

```

).add_to(big_map)
big_click = st_folium(
    big_map,
    height=520,
    use_container_width=True,
    key="coord_map_big",
    returned_objects=["last_clicked"],
)
if big_click and big_click.get("last_clicked"):
    new_lat = round(big_click["last_clicked"]["lat"], 5)
    new_lon = round(big_click["last_clicked"]["lng"], 5)
    if (new_lat, new_lon) != (
        round(st.session_state["coord_lat"], 5),
        round(st.session_state["coord_lon"], 5),
    ):
        st.session_state["coord_lat"] = new_lat
        st.session_state["coord_lon"] = new_lon
        st.rerun()

map_col, coord_col = st.columns([2, 1])
with map_col:
    with st.container(key="coord_map_wrap"):
        coord_map = folium.Map(
            location=[st.session_state["coord_lat"], st.session_state["coord_lon"]],
            zoom_start=15,
            tiles="OpenStreetMap",
        )
        folium.Marker(
            [st.session_state["coord_lat"], st.session_state["coord_lon"]],
            icon=folium.Icon(color="red", icon="home"),
        ).add_to(coord_map)
        map_click = st_folium(
            coord_map,
            height=140,
            use_container_width=True,
            key="coord_map",
            returned_objects=["last_clicked"],
        )
        if st.button(":material/zoom_in_map:", key="expand_map_btn", help="Ampliar
        mapa"):
            _expand_map_dialog()

if map_click and map_click.get("last_clicked"):
    new_lat = round(map_click["last_clicked"]["lat"], 5)
    new_lon = round(map_click["last_clicked"]["lng"], 5)
    if (new_lat, new_lon) != (
        round(st.session_state["coord_lat"], 5),
        round(st.session_state["coord_lon"], 5),
    ):
        st.session_state["coord_lat"] = new_lat
        st.session_state["coord_lon"] = new_lon
        st.rerun()

with coord_col:
    latitude = st.number_input(
        "Latitud", key="coord_lat", format="%.5f",
        help="Por defecto, el centro del barrio elegido. Ajústala escribiendo o
        haciendo clic en el mapa.",
    )
    longitude = st.number_input(
        "Longitud", key="coord_lon", format="%.5f",
        help="Por defecto, el centro del barrio elegido. Ajústala escribiendo o
        haciendo clic en el mapa.",
    )

with col_b:
    with st.container(border=True, key="section_form_alojamiento"):
        st.markdown("#### :material/apartment: Alojamiento")
        room_type = st.selectbox(
            "Tipo de alojamiento", ROOM_TYPES,
            format_func=lambda rt: ROOM_TYPE_ICONS[rt].split(": ", 1)[1],
            help=(
                "Cómo se comparte el espacio con otros huéspedes:\n\n"

```

```

        "- **Vivienda entera**: el huésped tiene todo el piso para él\n"
        "- **Habitación privada**: habitación propia dentro de una vivienda compartida\n"
        "- **Habitación compartida**: comparte también la habitación con otros huéspedes\n"
        "- **Habitación de hotel**: gestionado como un hotel u hostel"
    ),
)
property_type = st.selectbox(
    "Tipo de propiedad", PROPERTY_TYPES,
    help=(
        "Tipo de inmueble:\n\n"
        "- **Entire rental unit**: piso de alquiler estándar\n"
        "- **Entire loft / Entire home / Entire condo**: loft, casa o apartamento en un edificio con servicios\n"
        "- **Entire serviced apartment**: apartamento con servicios tipo hotel\n"
        "- **Room in hotel / Room in boutique hotel**: habitación gestionada profesionalmente\n"
        "- **Private room in...**: habitación privada dentro de ese tipo de vivienda\n"
        "- **Shared room in hostel**: cama u habitación compartida en un hostel\n"
        "- **Other**: categorías poco frecuentes, agrupadas"
    ),
)
c1, c2 = st.columns(2)
with c1:
    accommodates = st.number_input(
        "Huéspedes", min_value=1, max_value=16, value=4,
        help="Número máximo de personas que pueden alojarse. Una de las variables con más peso en el precio.",
    )
    bedrooms = st.number_input(
        "Dormitorios", min_value=0, max_value=20, value=1,
        help="Número de dormitorios del alojamiento.",
    )
    beds = st.number_input(
        "Camas", min_value=1, max_value=20, value=2,
        help="Número de camas disponibles (puede diferir de los dormitorios, por ejemplo con literas o sofás cama).",
    )
    bathrooms = st.number_input(
        "Baños", min_value=0.0, max_value=10.0, value=1.0, step=0.5,
        help="Número de baños. Se admiten medios baños (0.5) para aseos sin ducha ni bañera.",
    )
with c2:
    minimum_nights = st.number_input(
        "Estancia mínima (noches)", min_value=1, max_value=365, value=2,
        help="Noches mínimas por reserva. Es la variable que más influye en el precio: por encima de "
        "30 noches el anuncio deja de considerarse turístico, y compite con el alquiler residencial "
        "normal en vez de con hoteles.",
    )
    st.caption(market_label(minimum_nights))
    maximum_nights = st.number_input(
        "Estancia máxima (noches)", min_value=1, max_value=1125, value=365,
        help="Número máximo de noches que se puede reservar de una vez.",
    )
    availability_365 = st.number_input(
        "Días disponibles al año", min_value=0, max_value=365, value=300,
        help="Días del año que el anuncio está disponible para reservar. Menor disponibilidad puede indicar un uso más ocasional.",
    )
    has_license = st.toggle(
        "Licencia turística", value=True,
        help="Indica si el anuncio tiene número de licencia o registro turístico asociado, requisito legal en España para alquileres de corta duración.",
    )

col_c, col_d = st.columns(2)

with col_c:

```

```

with st.container(border=True, key="section_form_sobreti"):
    st.markdown("#### :material/person: Sobre ti como anfitrión")
    r1c1, r1c2 = st.columns(2, vertical_alignment="center")
    with r1c1:
        host_is_superhost = st.toggle(
            "Soy superhost", value=False,
            help="Distinción de Airbnb para anfitriones con muy buenas valoraciones, alta
            tasa de respuesta y pocas cancelaciones.",
        )
    with r1c2:
        calculated_host_listings_count = st.number_input(
            "Anuncios gestionados", min_value=1, max_value=500, value=1,
            help="Número total de anuncios que gestionas en Airbnb, incluido este. Los
            anfitriones con muchos anuncios suelen ser gestores profesionales.",
        )
    r2c1, r2c2 = st.columns(2, vertical_alignment="center")
    with r2c1:
        host_has_profile_pic = st.toggle(
            "Foto de perfil", value=True,
            help="Si tu perfil de anfitrión muestra una foto visible. Genera más
            confianza en los huéspedes.",
        )
    with r2c2:
        host_tenure_years = st.number_input(
            "Años como anfitrión", min_value=0.0, max_value=20.0, value=0.0, step=0.5,
            help="Tiempo que llevas publicando anuncios en Airbnb.",
        )
    r3c1, r3c2 = st.columns(2, vertical_alignment="center")
    with r3c1:
        host_identity_verified = st.toggle(
            "Identidad verificada", value=True,
            help="Si has verificado tu identidad ante Airbnb (documento oficial,
            teléfono, email...).",
        )
    with r3c2:
        host_user_tenure_years = st.number_input(
            "Años con cuenta", min_value=0.0, max_value=25.0,
            value=max(host_tenure_years, 0.5), step=0.5,
            help="Años con cuenta en la plataforma: tiempo que llevas registrado en
            Airbnb, aunque sea sin publicar anuncios.",
        )

with col_d:
    with st.container(border=True, key="section_form_comodidades"):
        st.markdown("#### :material/checklist: Comodidades")
        has_ac = st.toggle(
            "Aire acondicionado", value=CITY != "euskadi",
            help="Poco común aquí (en torno al 17% de los anuncios), esperable en un clima
            atlántico."
        )
        if CITY == "euskadi" else
        f"Muy común en {CITY_LABEL} (más del 80% de los anuncios lo tiene).",
    )
    ac2, ac3 = st.columns(2)
    with ac2:
        has_pool = st.toggle(
            "Piscina", value=False,
            help=(
                "Muy frecuente en Mallorca (cerca del 70% de los anuncios), reflejo del
                peso de "
                "las villas frente a los pisos urbanos."
                if CITY == "mallorca" else
                "Poco frecuente (en torno al 6% de los anuncios), pero un factor de lujo
                cuando está."
                if CITY == "euskadi" else
                "Poco frecuente (menos del 5% de los anuncios), pero un factor de lujo
                cuando está."
            ),
        )
    with ac3:
        if "has_dishwasher" in FEATURE_COLS:
            has_dishwasher = st.toggle(
                "Lavavajillas", value=False,
                help="Presente en casi la mitad de los anuncios."
            )

```

```

        if CITY == "euskadi" else
            "Presente en algo menos de la mitad de los anuncios.",
        )
    else:
        has_dishwasher = False
    if "has_sea_view" in FEATURE_COLS:
        has_sea_view = st.toggle(
            "Vistas al mar", value=False,
            help="En torno al 12% de los anuncios, una variable propia de un mercado insular."
        )
        if CITY == "mallorca" else
            "En torno al 5% de los anuncios, ligado al litoral de Donostia a Bilbao.",
        )
    else:
        has_sea_view = False
    # El máximo real de amenities varía por ciudad (84 en Barcelona, hasta 98 en
    # Mallorca, ver _CATALOG_RANGES): un max_value fijo en 84 bloqueaba silenciosamente
    # valores válidos en las otras seis ciudades. Se reutiliza el máximo ya calibrado
    # por ciudad en WHATIF_NUMERIC_VALUES en vez de duplicarlo con otro número suelto.
    n_amenities = st.number_input(
        "Nº total de comodidades del anuncio", min_value=0,
        max_value=WHATIF_NUMERIC_VALUES["n_amenities"][-1], value=30,
        help="Recuento total de comodidades marcadas en el anuncio (wifi, cocina, calefacción...), "
        "no solo las tres de arriba. La media real es de unas 30.",
    )

with col_a:
    with st.container(border=True, key="section_form_historial"):
        st.markdown("#### :material/history: Historial del anuncio")
        is_new_listing = st.toggle(
            "Es un anuncio nuevo, sin reseñas", value=True,
            help="Actívalo si el anuncio es nuevo y todavía no tiene reseñas ni historial de reservas.",
        )
        review_inputs = {}
        if not is_new_listing:
            c1, c2 = st.columns(2)
            with c1:
                review_inputs["number_of_reviews"] = st.number_input(
                    "Número de reseñas", min_value=0, value=10,
                    help="Total de reseñas recibidas desde la publicación del anuncio.",
                )
                review_inputs["reviews_per_month"] = st.number_input(
                    "Reseñas al mes", min_value=0.0, value=1.0, step=0.1,
                    help="Media de reseñas recibidas por mes, una forma indirecta de estimar la frecuencia de reservas.",
                )
                review_inputs["listing_age_days"] = st.number_input(
                    "Días desde la publicación", min_value=0, value=365,
                    help="Antigüedad del anuncio en días.",
                )
            with c2:
                review_inputs["number_of_reviews_ltm"] = st.number_input(
                    "Reseñas último año", min_value=0, value=5,
                    help="Reseñas recibidas en los últimos 12 meses.",
                )
                review_inputs["review_scores_rating"] = st.slider(
                    "Valoración media", 0.0, 5.0, 4.8,
                    help="Puntuación media de las reseñas, de 0 a 5.",
                )
                review_inputs["days_since_last_review"] = st.number_input(
                    "Días desde la última reseña", min_value=0, value=30,
                    help="Cuántos días han pasado desde la reseña más reciente.",
                )
                review_inputs["estimated_occupancy_l365d"] = st.number_input(
                    "Noches ocupadas/año (estimado)", min_value=0, max_value=365, value=100,
                    help="Estimación de cuántas noches se ha reservado el alojamiento en el último año.",
                )
            )
        else:
            st.caption("Sin reseñas, disponibilidad ni historial todavía: el modelo lo tiene en cuenta como tal.")

```

```

with st.container(key="form_predict_wrap"):
    clicked = st.button(":material/bolt: Predecir precio", type="primary",
                        use_container_width=True)

host_entire_homes_ratio = 1.0 if room_type == "Entire home/apt" else 0.0

current_inputs = {
    "district": district,
    "neighbourhood": neighbourhood,
    "room_type": room_type,
    "property_type": property_type,
    "accommodates": accommodates,
    "bedrooms": bedrooms,
    "beds": beds,
    "bathrooms": bathrooms,
    "minimum_nights": minimum_nights,
    "maximum_nights": maximum_nights,
    "availability_365": availability_365,
    "has_license": has_license,
    "host_is_superhost": host_is_superhost,
    "host_has_profile_pic": host_has_profile_pic,
    "host_identity_verified": host_identity_verified,
    "calculated_host_listings_count": calculated_host_listings_count,
    "host_tenure_years": host_tenure_years,
    "host_user_tenure_years": host_user_tenure_years,
    "host_entire_homes_ratio": host_entire_homes_ratio,
    "is_new_listing": is_new_listing,
    "latitude": latitude,
    "longitude": longitude,
    "has_ac": has_ac,
    "has_pool": has_pool,
    "has_dishwasher": has_dishwasher,
    "has_sea_view": has_sea_view,
    "n_amenities": n_amenities,
    **review_inputs,
}

st.session_state["live_inputs"] = current_inputs
confirmed_inputs = st.session_state.get("confirmed_inputs", DEFAULT_INPUTS)

if clicked:
    st.session_state["confirmed_inputs"] = current_inputs
    confirmed_inputs = current_inputs
    st.switch_page(page_prediccion_p)
elif current_inputs != confirmed_inputs:
    st.caption(":material/warning: Has cambiado el formulario – pulsa **Predecir precio**
    para actualizar el resultado.")

inputs = st.session_state.get("confirmed_inputs", DEFAULT_INPUTS)

district = inputs["district"]
neighbourhood = inputs["neighbourhood"]
room_type = inputs["room_type"]
accommodates = inputs["accommodates"]
bedrooms = inputs["bedrooms"]
bathrooms = inputs["bathrooms"]
minimum_nights = inputs["minimum_nights"]
latitude = inputs["latitude"]
longitude = inputs["longitude"]

feature_row = build_features(inputs)
X = pd.DataFrame([feature_row])[FEATURE_COLS]
for c in X.select_dtypes(include="bool").columns:
    X[c] = X[c].astype(int)

pred_log = model.predict(X)[0]
pred = float(np.expm1(pred_log))

# Cada señal de confianza pesa distinto según la evidencia real de 05_model_evaluation.ipynb:
# Group Flat es la causa documentada de los peores errores del modelo en las 3 ciudades (cola
# alta de precio subestimada de forma sistemática); un distrito de poca muestra tiene MAE

```

```

# "contenido" según el propio notebook, no un fallo grave; y "sin reseñas todavía" se basa en
# unos pocos anuncios de entrenamiento con 0 reviews y precio disparatado (probablemente
# abandonados o anti-reserva), no en que un anfitrión nuevo con precio honesto prediga peor –
# y es, con diferencia, la señal más común (todo anuncio nuevo la dispara). Se toma el máximo
# entre señales activas, no la suma, para no disparar el rango si coinciden varias.
_CONFIDENCE_MULTIPLIERS = [
    ("Group Flat", 2.2),
    ("tiene poca representación", 1.5),
    ("Al no tener reseñas", 1.3),
]

def _confidence_multiplier(flag_list):
    mult = 1.0
    for f in flag_list:
        for marker, m in _CONFIDENCE_MULTIPLIERS:
            if marker in f:
                mult = max(mult, m)
    return mult

# MAE del quintil de precio al que pertenece esta predicción, no el MAE plano de toda
# la ciudad (ver load_price_segment_errors): un anuncio a 150€ y una villa a 1500€ no
# deberían leer el mismo margen de error en euros.
flags = confidence_flags(inputs)
multiplier = _confidence_multiplier(flags)
segment = mae_for_price(CITY, pred)
segment_mae = float(segment["mae"])
low = max(0.0, pred - segment_mae * multiplier)
high = pred + segment_mae * multiplier

def _empty_state_sin_formulario(mensaje):
    """Estado vacío compartido por Predicción y Comparar cuando el usuario entra sin
    haber confirmado nunca un anuncio (sin 'confirmed_inputs'): ambas páginas, sin esto,
    mostrarían silenciosamente una predicción sobre el anuncio por defecto, como si fuera
    el suyo. Mismo contenedor con borde y el mismo st.page_link que el resto de la app."""
    with st.container(border=True):
        st.markdown(
            f'<p style="font-size:15px;color:#6B6A66;margin:0 0 14px;">{html.escape(mensaje)}</p>',
            unsafe_allow_html=True,
        )
    st.page_link(page_formulario_p, label="Ir al Formulario", icon=":material/edit_note:")

def page_prediccion():
    if "confirmed_inputs" not in st.session_state:
        st.header("Precio recomendado")
        _empty_state_sin_formulario(
            "Todavía no has descrito tu anuncio. Rellena el Formulario y pulsa "
            "“Predecir precio” para ver aquí el precio recomendado."
        )
    return

stale = st.session_state.get("live_inputs", inputs) != inputs
title_col, alert_col = st.columns([1, 1.6], vertical_alignment="center")
with title_col:
    st.header("Precio recomendado")
with alert_col:
    if stale:
        st.info(
            "Has cambiado el formulario desde el último cálculo. Ve a **Formulario** y pulsa "
            " "
            "***Predecir precio** para ver el resultado actualizado."
        )

def fmt_whatif_value(v):
    if isinstance(v, bool):
        return "Sí" if v else "No"
    if isinstance(v, float):
        return f"{v:g}"
    return str(v)

```

```

def whatif_options_and_current(key):
    if key in WHATIF_NUMERIC_VALUES:
        current = inputs.get(key, feature_row.get(key, REVIEW_DEFAULTS.get(key)))
        return sorted(set(WHATIF_NUMERIC_VALUES[key] | {current}), current)
    if key == "district":
        return DISTRICTS, district
    if key == "neighbourhood":
        return sorted(NEIGHBOURHOOD_STATS.keys()), neighbourhood
    if key == "room_type":
        return ROOM_TYPES, room_type
    if key == "property_type":
        return PROPERTY_TYPES, inputs["property_type"]
    return [True, False], inputs.get(key, True)

# Se lee el estado de la simulación "qué pasaría si" (sección de la derecha) antes de
# pintar el precio y el waterfall, para que ambos reflejen el valor simulado si lo hay.
whatif_labels_list = list(WHATIF_LABELS.values())
whatif_choice = st.session_state.get("whatif_choice", whatif_labels_list[0])
whatif_key = {v: k for k, v in WHATIF_LABELS.items()}[whatif_choice]
whatif_all_options, whatif_real_val = whatif_options_and_current(whatif_key)
sim_val_key = f"whatif_val_{whatif_key}"
reset_flag_key = f"whatif_reset_flag_{whatif_key}"
if st.session_state.get(reset_flag_key):
    # El botón "Restablecer" solo puede marcar esta bandera (no puede tocar
    # sim_val_key directamente: ese widget ya se instanció en el run anterior).
    # Aquí, al principio del run siguiente y antes de crear el widget, sí se puede.
    st.session_state[sim_val_key] = whatif_real_val
    st.session_state[reset_flag_key] = False
sim_val = st.session_state.get(sim_val_key, whatif_real_val)
simulating = sim_val != whatif_real_val

if simulating:
    sim_inputs = whatif_apply(inputs, whatif_key, sim_val)
    sim_row = build_features(sim_inputs)
    X_disp = pd.DataFrame([sim_row])[FEATURE_COLS]
    for c in X_disp.select_dtypes(include="bool").columns:
        X_disp[c] = X_disp[c].astype(int)
    pred_disp = float(np.exp1(model.predict(X_disp)[0]))
    flags_disp = confidence_flags(sim_inputs)
else:
    X_disp, pred_disp, flags_disp = X, pred, flags
    multiplier_disp = _confidence_multiplier(flags_disp)
    segment_disp = mae_for_price(CITY, pred_disp)
    segment_mae_disp = float(segment_disp["mae"])
    low_disp = max(0.0, pred_disp - segment_mae_disp * multiplier_disp)
    high_disp = pred_disp + segment_mae_disp * multiplier_disp

# Frases cortas para la señal de confianza en una sola línea: el texto completo
# de confidence_flags() (features.py) es una frase larga pensada para leerse
# sola, no para caber junto al precio, así que aquí se resume sin tocar la
# lógica que decide cuándo aparece cada señal ni el multiplicador de rango.
_CONF_SHORT = {
    "Group Flat": "alojamiento muy grande, pocas referencias históricas",
    "Al no tener reseñas": "sin reseñas todavía, menos información",
}
def _short_flag(text):
    for marker, short in _CONF_SHORT.items():
        if marker in text:
            return short
    if "tiene poca representación" in text:
        return f"{text.split(' tiene poca representación')[0]}, pocos anuncios en la zona"
    return text

with st.container(border=True, key="section_prediccion_resumen"):
    st.markdown("#### :material/payments: Precio estimado")
    price_col, conf_col = st.columns([1, 1.6], vertical_alignment="center")
    with price_col:
        _rango_tip = (
            f"El modelo se equivoca en {segment_mae_disp:,.0f} € de media (MAE) para anuncios de "
            f"{CITY_LABEL} en tu mismo rango de precio ({segment_disp['price_min']:,.0f}-"
            f"{segment_disp['price_max']:,.0f}€ en los datos reales), así que el rango normal

```

```

        es tu "
        f"precio ± {segment_mae_disp:,.0f} €."
    )
    if multiplier_disp != 1.0:
        _rango_tip += (
            f" Aquí se amplía a ± {segment_mae_disp * multiplier_disp:,.0f} € porque hay
            señales que "
            "reducen la confianza (ver el aviso de la derecha): con menos información, el
            error típico "
            "también es mayor."
        )
    st.markdown(
        f'<div style="display:flex; align-items:baseline; gap:0.6rem; flex-wrap:wrap;
        line-height:1.2;">'
        f'<span style="font-size:2rem; font-weight:700; color:#2C2C2A;">{pred_disp:,.0f}
        €</span>'
        f'<span class="fn-tooltip" style="font-size:0.85rem; color:#6B6862;">'
        f'<span class="fn-tooltip-box">{html.escape(_rango_tip)}</span>'
        f'</span>'
        f'</div>',
        unsafe_allow_html=True,
    )
    with conf_col:
        if flags_disp:
            n = len(flags_disp)
            conf_line = f"■ {n} señal{'es' if n > 1 else ''}: " + "; ".join(_short_flag(f)
            for f in flags_disp) + "."
            st.markdown(
                f'<div style="background:#F7EFD9; border-radius:8px; padding:0.4rem 0.75rem;
                '
                f'color:#6B4E1D; font-size:0.9rem;">{conf_line}</div>',
                unsafe_allow_html=True,
            )
        else:
            st.markdown(
                '<div style="background:#E7F0E4; border-radius:8px; padding:0.4rem 0.75rem; '
                'color:#2F5233; font-size:0.9rem;">■ Confianza alta</div>',
                unsafe_allow_html=True,
            )

    col_shap, col_whatif = st.columns([1.2, 1])
    with col_shap:
        with st.container(border=True, key="section_prediccion_shap"):
            st.markdown("#### :material/insights: Por qué este precio")
            explainer = get_shap_explainer(model, CITY)
            shap_exp = explain(explainer, X_disp)
            base_value = float(shap_exp.base_values[0])
            contributions = pd.Series(shap_exp.values[0], index=FEATURE_COLS)
            group_contrib =
            contributions.groupby(contributions.index.map(FEATURE_TO_GROUP)).sum()
            group_contrib =
            group_contrib.reindex(group_contrib.abs().sort_values(ascending=False).index)

            display_inputs = sim_inputs if simulating else inputs
            onehot_selected = {
                "Distrito": display_inputs["district"],
                "Tipo de alojamiento": display_inputs["room_type"],
                "Tipo de propiedad": display_inputs["property_type"],
            }
            def display_label(label):
                selected = onehot_selected.get(label)
                return f"{label}: {selected}" if selected else label

            def group_value_text(label):
                if label == "Latitud":
                    return f"{display_inputs['latitude']:.5f}"
                if label == "Longitud":
                    return f"{display_inputs['longitude']:.5f}"
                spec = PDP_GROUPS.get(label)
                if spec is None:
                    return ""
                kind = spec["kind"]

```

```

        if kind == "numeric":
            val = display_inputs.get(spec["sample_key"],
                REVIEW_DEFAULTS.get(spec["sample_key"]))
            return "" if val is None or pd.isna(val) else f"{val:g}"
        if kind == "boolean":
            return "Sí" if display_inputs.get(spec["col"]) else "No"
        if kind == "onehot":
            return onehot_selected.get(label, "")
        if kind == "host_scale":
            val = display_inputs.get("calculated_host_listings_count")
            return "" if val is None else f"{val:g}"
        return ""

top_n_wf = 10
chk_col, sim_col = st.columns([1.3, 2], vertical_alignment="center")
with chk_col:
    show_all_wf = st.checkbox(f"Ver las {len(group_contrib)} variables",
        key="show_all_wf")
with sim_col:
    if simulating:
        st.markdown(
            f'<div style="text-align:right; color:#6B6862; font-size:0.85rem;">'
            f'Simulando: {fmt_whatif_value(sim_val)} (real:'
            f'{fmt_whatif_value(whatif_real_val)})</div>',
            unsafe_allow_html=True,
        )
    if show_all_wf or len(group_contrib) <= top_n_wf:
        steps = list(group_contrib.items())
    else:
        steps = list(group_contrib.head(top_n_wf).items())
        other_sum = group_contrib.iloc[top_n_wf:].sum()
        steps.append((f"Otras {len(group_contrib) - top_n_wf} variables", other_sum))

# What-if: contribución SHAP de TODAS las variables realmente afectadas por la
# simulación bajo los datos REALES (sin simular), para poder comparar
# antes/después en su fila. No es solo la variable elegida en el desplegable:
# simular "Distrito", por ejemplo, también recalcula barrio, lat/lon, distancia
# al centro y densidad (ver whatif_apply/build_features), así que se comparan
# las columnas del modelo una a una para detectar TODAS las que de verdad
# cambiaron. Coste despreciable (~1ms una fila más sobre el mismo explainer ya
# cacheado).
changed_groups = set()
group_contrib_before = None
if simulating:
    real_vals, sim_vals = X.iloc[0], X_disp.iloc[0]
    diff_mask = (real_vals != sim_vals) & ~(real_vals.isna() & sim_vals.isna())
    changed_groups = {FEATURE_TO_GROUP.get(c, c) for c in diff_mask[diff_mask].index}
    shap_exp_before = explain(explainer, X)
    contributions_before = pd.Series(shap_exp_before.values[0], index=FEATURE_COLS)
    group_contrib_before = contributions_before.groupby(
        contributions_before.index.map(FEATURE_TO_GROUP)
    ).sum()

cum = base_value
level = np.exp(cum) - 1.0
rows = [{
    "label": "Punto de partida del modelo",
    "start": 0.0, "end": level, "kind": "punto de partida",
    "etiqueta": f"{level:,.0f} €",
    "color_etiqueta": "#6B6862",
    "tip1": f"Punto de partida del modelo: {level:,.0f} €",
    "tip2": (
        f"Precio medio de referencia entre todos los anuncios de {CITY_LABEL} con los
        que se "
        "entrenó el modelo, antes de aplicar ninguna característica de este anuncio
        en concreto."
    ),
}]
for label, val in steps:
    before = level
    cum += val
    level = np.exp(cum) - 1.0
    delta = level - before

```

```

pct = (delta / before * 100) if before else 0.0
value_text = group_value_text(label)
row_label = display_label(label)
# El tooltip parte del label crudo, no de display_label: para las one-hot
# este ya incluye la categoría elegida ("Distrito: Eixample") y volver a
# pegarle el valor la repetiría.
wf_row = {
    "label": row_label,
    "start": min(before, level), "end": max(before, level),
    "kind": "sube" if delta >= 0 else "baja",
    "etiqueta": f"{delta:+,.0f} €".replace("-", "-"),
    "color_etiqueta": GREEN if delta >= 0 else RED,
    "tip1": f"{label}: {value_text}" if value_text else label,
    "tip2": f"{before:,.0f} € → {level:,.0f} €",
    "tip3": f"{pct:+,.0f} %".replace("-", "-"),
}
# Barra "antes" en CADA fila cuya variable realmente cambió al simular (no
# solo la elegida en el desplegable, ver changed_groups arriba): mismo
# punto de partida ("before"), llegando hasta donde habría llegado con el
# valor real de esa variable en vez del simulado. La etiqueta principal
# ("etiqueta", ya fijada arriba) queda tal cual, con el signo y color del
# valor ACTUAL; aquí solo se añade el "antes" en gris, pequeño, al lado.
if group_contrib_before is not None and label in changed_groups:
    old_group_val = group_contrib_before.get(label, 0.0)
    alt_cum = (cum - val) + old_group_val
    alt_level = np.exp(alt_cum) - 1.0
    old_delta = alt_level - before
    # Formato flecha "antes → ahora" en vez del número suelto en la barra:
    # "etiqueta" (la barra) se vacía, y en su sitio van dos textos pegados
    # uno al otro: el "antes" en gris neutro (sin colorear por signo, es
    # solo referencia) y el "ahora" con el color de dirección de siempre
    # (color_etiqueta, ya fijado arriba). Sin signo forzado ("17 €", no
    # "+17 €"), igual que el "Precio final: X € → Y €" de arriba.
    antes_text = f"{old_delta:,.0f} € → ".replace("-", "-")
    wf_row["etiqueta_antes"] = antes_text
    wf_row["dx_antes"] = 5
    wf_row["etiqueta_ahora"] = f"{delta:,.0f} €".replace("-", "-")
    # dx en píxeles: arranca justo donde termina el texto gris "antes → "
    # (estimando su ancho renderizado a partir de su nº de caracteres, a
    # fontSize 11), para que quede pegado sin solaparse.
    wf_row["dx_ahora"] = 5 + len(antes_text) * 6.5 + 2
    wf_row["etiqueta"] = ""
    wf_row["before_start"] = min(before, alt_level)
    wf_row["before_end"] = max(before, alt_level)
    wf_row["before_tip"] = (
        f"Sin simular: {before:,.0f} € → {alt_level:,.0f} € ({old_delta:+,.0f}
        €)"
    ).replace("-", "-")
    rows.append(wf_row)
total_row = {
    "label": "Precio final", "start": 0.0, "end": level, "kind": "total",
    "etiqueta": f"{level:,.0f} €",
    "color_etiqueta": WF_TOTAL,
    "tip1": f"Precio final: {level:,.0f} €",
    "tip2": "",
}
if simulating:
    total_row["before_start"] = 0.0
    total_row["before_end"] = pred
    total_row["before_tip"] = f"Sin simular: {pred:,.0f} €"
    # Mismo formato flecha que las variables que cambiaron: antes en gris,
    # ahora en su color (WF_TOTAL, ya fijado en color_etiqueta arriba).
    antes_text = f"{pred:,.0f} € → ".replace("-", "-")
    total_row["etiqueta_antes"] = antes_text
    total_row["dx_antes"] = 5
    total_row["etiqueta_ahora"] = total_row["etiqueta"]
    total_row["dx_ahora"] = 5 + len(antes_text) * 6.5 + 2
    total_row["etiqueta"] = ""
    rows.append(total_row)

if simulating:
    diff = pred_disp - pred
    diff_color = GREEN if diff >= 0 else RED

```

```

diff_str = f"{diff:+,.0f} €".replace("-", "-")
st.markdown(
    f"""Precio final: {pred:,.0f} € → {pred_disp:,.0f} € "
    f"(<span style='color:{diff_color}'>{diff_str}</span>)*""",
    unsafe_allow_html=True,
)

# Leyenda propia en HTML/CSS para las 5 categorías del waterfall, en vez de la
# de Vega-Lite (desactivada en chart_waterfall): meter "antes (valor anterior)"
# como 5ª categoría de la escala de color de Vega, fusionada entre las 3 capas
# de barras, provocaba un bug real de Vega (categorías que desaparecían sin
# avisar). Aquí se controla el layout entero a mano, así que "antes" sale justo
# al lado de "punto de partida", en la misma fila que el resto.
def _wf_legend_item(color, label):
    return (
        f'<span style="display:inline-flex; align-items:center; gap:5px;">'
        f'<span style="display:inline-block; width:13px; height:13px; '
        f'border-radius:2px; background:{color};"></span>{label}</span>'
    )
legend_items = [_wf_legend_item(WF_START, "punto de partida")]
if simulating:
    antes_swatch = (
        f'<span style="display:inline-block; width:13px; height:13px; '
        f'border-radius:2px; background-image:repeating-linear-gradient('
        f'45deg, {WF_HATCH_LINE} 0, {WF_HATCH_LINE} 2px, {WF_HATCH_BG} 2px, '
        f'{WF_HATCH_BG} 5px);"></span>'
    )
    # Justo al lado de "punto de partida": los dos son grises de referencia,
    # sin color de dirección, así que van agrupados al principio de la leyenda.
    legend_items.append(
        f'<span style="display:inline-flex; align-items:center; gap:5px;">'
        f'{antes_swatch}antes (valor anterior)</span>'
    )
legend_items += [
    _wf_legend_item(GREEN, "sube"),
    _wf_legend_item(RED, "baja"),
    _wf_legend_item(WF_TOTAL, "total"),
]
st.markdown(
    f'<div style="display:flex; flex-wrap:wrap; align-items:center; gap:14px; '
    f'margin:0 0 6px 2px; font-size:12.5px; color:#6B6862;">'
    + "".join(legend_items) + "</div>",
    unsafe_allow_html=True,
)
chart = chart_waterfall(pd.DataFrame(rows), x_title="Precio (€)")
# key dinámica (no fija): cuando se activa/desactiva la simulación, o cambia la
# variable/valor simulado, la capa rayada "antes" aparece/desaparece y el spec de
# Vega cambia de forma (nº de capas y datasets). Con una key fija, Streamlit intenta
# parchear la vista de Vega anterior en vez de recrearla, y eso dispara un error real
# de Vega-Embed ("Unrecognized data set") que deja el gráfico en blanco de forma
# permanente (visible solo en la consola del navegador, no en Python). Cambiar la key
# cuando cambia la forma del spec fuerza a remontar el componente en vez de
# parchearlo.
waterfall_key = f"chart_shap_waterfall_{simulating}_{whatif_key}_{sim_val}"
st.altair_chart(chart, use_container_width=True, key=waterfall_key)

with col_whatif:
    with st.container(border=True, key="section_prediccion_whatif"):
        st.markdown("#### :material/tune: ¿Qué pasaría si cambio...?")

        wcol1, wcol2 = st.columns(2)
        with wcol1:
            whatif_choice = st.selectbox("Variable a explorar", whatif_labels_list,
                                         key="whatif_choice")
            whatif_key = {v: k for k, v in WHATIF_LABELS.items()}[whatif_choice]
            options, current_val = whatif_options_and_current(whatif_key)
            this_sim_key = f"whatif_val_{whatif_key}"
            if this_sim_key not in st.session_state:
                st.session_state[this_sim_key] = current_val
            is_numeric_whatif = whatif_key in WHATIF_NUMERIC_VALUES

        with wcol2:
            if is_numeric_whatif:

```

```

        sim_val = st.select_slider("Simula un valor", options=options,
                                   key=this_sim_key)
    else:
        sim_val = st.selectbox(
            "Simula un valor", options, format_func=fmt_whatif_value,
            key=this_sim_key,
        )

    if is_numeric_whatif:
        prices = [whatif_predict(model, inputs, whatif_key, v) for v in options]
        chart = chart_line_highlight(options, prices, current_val, sim_val,
                                     x_title=whatif_choice)
        st.altair_chart(chart, use_container_width=True)
    else:
        prices_all = [whatif_predict(model, inputs, whatif_key, o) for o in options]
        order = sorted(range(len(options)), key=lambda i: prices_all[i], reverse=True)
        # La ventana se centra en el valor SIMULADO (el del desplegable), no en el
        # guardado en el formulario: si no, al simular un valor lejano del real, la
        # ventana se queda donde estaba y el simulado puede quedar fuera de pantalla.
        sim_rank = order.index(options.index(sim_val))

        top_n_cat = 12
        if len(options) > top_n_cat:
            show_all_cat = st.checkbox(
                f"Ver las {len(options)} opciones", key=f"whatif_expand_{whatif_key}",
            )
        else:
            show_all_cat = True

        if show_all_cat:
            shown_idx = order
        else:
            # Ventana de top_n_cat opciones centrada en el rango (por precio) del
            # valor simulado, para que siempre se vea sin tener que desplegar todo.
            half = top_n_cat // 2
            lo = max(0, min(sim_rank - half, len(order) - top_n_cat))
            shown_idx = order[lo:lo + top_n_cat]

        display_labels = [fmt_whatif_value(options[i]) for i in shown_idx]
        shown_prices = [prices_all[i] for i in shown_idx]
        sim_label = fmt_whatif_value(sim_val)
        real_label = fmt_whatif_value(current_val)

        # Si el valor real guardado cae fuera de la ventana visible de barras, no
        # aparece resaltado ahí: se muestra aparte, como texto, para poder comparar
        # igualmente contra la barra naranja del simulado.
        if sim_val != current_val and real_label not in display_labels:
            real_price = prices_all[options.index(current_val)]
            st.caption(
                f"Tu {whatif_choice.lower()} real ({fmt_whatif_value(current_val)}): "
                f"{real_price:,.0f} €"
            )

        chart = chart_hbar_highlight(
            display_labels, shown_prices, sim_label, real_label, x_title="Precio predicho
            (€)"
        )
        st.altair_chart(chart, use_container_width=True)

    if sim_val != current_val:
        if st.button("Restablecer al valor real", key=f"whatif_reset_btn_{whatif_key}"):
            st.session_state[reset_flag_key] = True
            st.rerun()

ROOM_TYPE_PLURAL = {
    "Entire home/apt": "viviendas enteras",
    "Private room": "habitaciones privadas",
    "Shared room": "habitaciones compartidas",
    "Hotel room": "habitaciones de hotel",
}

```

```

def _inject_comparar_css():
    st.markdown(
        """<style>
        /* Ancho máximo de contenido (1000px, como en la referencia): sin esto el panel se
        estira a todo el ancho de la ventana y el párrafo del veredicto queda demasiado
        largo, empujando la columna de cifras lejos del titular. */
        .st-key-comparar_page_wrap { max-width: 1000px; margin: 0 auto; gap: 14px !important; }

        .st-key-comparar_resumen {
            background: #F5F3EE;
            border-radius: 12px;
            padding: 18px 22px;
            margin-bottom: 14px;
            gap: 8px !important;
        }
        .st-key-comparar_resumen .cmp-frase {
            margin: 0;
            font-size: 15px;
            line-height: 1.6;
            color: #6B6A66;
        }
        .st-key-comparar_resumen .cmp-frase b { color: #2C2C2A; font-weight: 600; }
        .st-key-comparar_resumen [data-testid="stButton"] button {
            border: none;
            background: transparent;
            color: #C42843;
            font-weight: 600;
            font-size: 13.5px;
            padding: 0;
            box-shadow: none;
        }
        .st-key-comparar_resumen [data-testid="stButton"] button:hover {
            color: #FF385C;
            background: transparent;
        }
        .st-key-comparar_resumen .cmp-criterios {
            display: flex;
            gap: 8px;
            flex-wrap: wrap;
        }
        .st-key-comparar_resumen .cmp-chip {
            background: #fff;
            border: 1px solid #E3E0DA;
            border-radius: 8px;
            padding: 5px 11px;
            font-size: 12.5px;
            color: #6B6A66;
        }
        .st-key-comparar_resumen .cmp-chip b { color: #2C2C2A; font-weight: 600; }
        .st-key-comparar_resumen .cmp-aviso-ampliado {
            margin: 14px 0 0;
            padding: 11px 14px;
            background: #FDF6E9;
            border: 1px solid #EDDCB9;
            border-radius: 9px;
            font-size: 13px;
            color: #6B5420;
            line-height: 1.55;
        }
        .st-key-comparar_resumen .cmp-aviso-ampliado b { font-weight: 700; }

        /* Streamlit mete su propio padding/gap por defecto en cualquier contenedor con
        border=True y entre cada widget/markdown suyo: sumado, hacía el panel mucho más
        alto de lo necesario. Se sustituye por el espaciado de la referencia (paneles:
        padding 24px 28px 26px, radio 14px, borde 1px #E3E0DA). */
        .st-key-section_comparar_veredicto, .st-key-section_comparar_mapa {
            gap: 10px !important;
            padding: 24px 28px 26px !important;
            border-radius: 14px !important;
            border-color: #E3E0DA !important;
        }
        .st-key-section_comparar_veredicto .stMarkdown { padding: 0 !important; }
        /* 24px/600: el mismo tamaño que usan los demás títulos de sección "####" en el

```

```

resto de la app (p.ej. "Ubicación" en el Formulario), no el 19px de la maqueta. */
.st-key-section_comparar_veredicto h4 {
  padding: 0 !important; margin: 0 0 4px !important; font-size: 24px !important;
  font-weight: 600 !important;
}

/* Streamlit añade un margen inferior negativo (-16px) por defecto al final de
cada bloque de markdown (pensado para compensar el margen del último <p> que
normalmente cierra un bloque de texto). Nuestro HTML no termina en ese <p>, así
que ese negativo recortaba el alto que Streamlit reservaba para el bloque entero
y la última línea del párrafo del veredicto se solapaba con el gráfico de debajo. */
.st-key-section_comparar_veredicto [data-testid="stMarkdownContainer"],
.st-key-section_comparar_mapa [data-testid="stMarkdownContainer"],
.st-key-comparar_resumen [data-testid="stMarkdownContainer"] {
  margin-bottom: 0 !important;
}

/* Separador antes del gráfico (borde superior + aire), como en la referencia:
se aplica al bloque que sigue inmediatamente a la cabecera del veredicto. */
.st-key-section_comparar_veredicto div:has(> .stMarkdown .cmp-cabecera) + div {
  margin-top: 12px;
  padding-top: 20px;
  border-top: 1px solid #F1EEE9;
}

.st-key-section_comparar_veredicto .cmp-cabecera {
  display: grid;
  grid-template-columns: 1fr auto;
  gap: 28px;
  align-items: start;
}

.st-key-section_comparar_veredicto .cmp-pastilla {
  display: inline-block;
  margin: 0 0 10px;
  padding: 4px 12px;
  border-radius: 7px;
  font-size: 12px;
  font-weight: 700;
  letter-spacing: .03em;
}

.st-key-section_comparar_veredicto .cmp-titular {
  margin: 0 0 10px;
  font-size: 28px;
  font-weight: 700;
  letter-spacing: -.025em;
  line-height: 1.15;
  color: #2C2C2A;
}

.st-key-section_comparar_veredicto .cmp-explica {
  margin: 0;
  font-size: 15px;
  color: #6B6A66;
  line-height: 1.6;
  max-width: 58ch;
}

.st-key-section_comparar_veredicto .cmp-explica b { color: #2C2C2A; font-weight: 600; }
.st-key-section_comparar_veredicto .cmp-par { text-align: right; flex-shrink: 0; }
.st-key-section_comparar_veredicto .cmp-par-rot { margin: 0 0 3px; font-size: 12.5px;
color: #6B6A66; }
.st-key-section_comparar_veredicto .cmp-par-cifra {
  margin: 0 0 14px; font-size: 25px; font-weight: 700; letter-spacing: -.02em; color:
#2C2C2A;
}

.st-key-section_comparar_veredicto .cmp-par-cifra:last-child { margin-bottom: 0; }
.st-key-section_comparar_veredicto .cmp-par-cifra.cmp-tuyo { color: #C42843; }
.st-key-section_comparar_veredicto .cmp-nota-margen {
  margin: 0;
  padding: 10px 14px;
  background: #F5F3EE;
  border-radius: 9px;
  font-size: 13px;
  color: #6B6A66;
  line-height: 1.5;
}

```

```

.st-key-section_comparar_veredicto .cmp-nota-margen b { color: #2C2C2A; font-weight: 600;
}
.cmp-leyenda-hist { display: flex; gap: 16px; flex-wrap: wrap; margin: 4px 0 0;
font-size: 12px; color: #6B6A66; }
.cmp-leyenda-hist span { display: flex; align-items: center; gap: 6px; }
.cmp-leyenda-hist i { width: 10px; height: 10px; border-radius: 3px; display:
inline-block; }

/* El conjunto entero (etiqueta + campo) NO se fija a 200px: a esa anchura la
pregunta no cabe en una línea. Solo el control numérico (campo + botones) se fija
a 200px; la etiqueta queda libre para ocupar el ancho del panel y así entrar en
una sola línea, con el icono de ayuda justo detrás del texto en vez de suelto en
la esquina (comportamiento por defecto de Streamlit: label en flex con el icono
empujado al extremo derecho de su contenedor). */
.st-key-comparar_precio_input [data-testid="stWidgetLabel"] {
display: inline-flex;
align-items: center;
justify-content: flex-start;
gap: 4px;
width: auto;
margin-bottom: 2px !important;
}
.st-key-comparar_precio_input [data-testid="stWidgetLabel"] > div:last-child { flex: 0 0
auto; }
.st-key-comparar_precio_input label p { font-size: 13px !important; white-space: nowrap;
}
.st-key-comparar_precio_input [data-testid="stNumberInputContainer"] { width: 200px; }
/* El botón "Usar precio predicho" vive en la columna de al lado del campo: sin
este margin-top quedaría a la altura de la ETIQUETA del campo (arriba), no de la
fila del propio input (abajo), porque ambas columnas arrancan en el mismo punto. */
.st-key-comparar_precio_input [data-testid="stButton"] { margin-top: 26px; }
.st-key-comparar_precio_input [data-testid="stButton"] button {
border: none; background: transparent; color: #C42843; font-weight: 600; font-size:
13px;
padding: 0; box-shadow: none;
}
.st-key-comparar_precio_input [data-testid="stButton"] button:hover { color: #FF385C;
background: transparent; }

.cmp-anuncios-titulo { margin: 0 0 4px; font-size: 28px; font-weight: 700; color:
#2C2C2A; }
.cmp-anuncios-sub { margin: 0; font-size: 13.5px; color: #6B6A66; }

/* Conmutador Lista/Mapa deliberadamente distinto del selector "Mostrar" del mapa
(que se deja con el estilo por defecto de segmented_control): pastilla sólida,
más grande, para que se lea como el interruptor principal de la sección. */
.st-key-comparar_vista [data-baseweb="button-group"] {
background: #F5F3EE; border-radius: 999px; padding: 4px; gap: 2px !important;
}
.st-key-comparar_vista button[data-testid="stBaseButton-segmented_control"],
.st-key-comparar_vista button[data-testid="stBaseButton-segmented_controlActive"] {
font-size: 15px !important; font-weight: 600 !important; padding: 9px 26px
!important;
border-radius: 999px !important; border: none !important;
}
.st-key-comparar_vista button[data-testid="stBaseButton-segmented_control"] {
background: transparent !important; color: #6B6A66 !important;
}
.st-key-comparar_vista button[data-testid="stBaseButton-segmented_controlActive"] {
background: #FF385C !important; color: #fff !important;
}

.cmp-pagina-info { margin: 0; font-size: 13px; color: #6B6A66; text-align: center; }

.cc-card {
background: #fff; border: 1px solid #E3E0DA; border-radius: 14px; overflow: hidden;
box-shadow: 0 2px 10px rgba(44,44,42,.08);
}
.cc-foto-wrap, .cc-card .cc-sinfoto:first-child { height: 150px; }
.cc-foto-wrap img { width: 100%; height: 150px; object-fit: cover; display: block; }
.cc-card .cc-sinfoto {
height: 64px; background: #F5F3EE; display: flex; align-items: center;
justify-content: center;
}

```

```

    }
    .cc-card .cc-sinfoto span { font-size: 12px; color: #8B8A85; }
    .cc-cuerpo { padding: 13px 15px 15px; }
    .cc-titulo { margin: 0 0 5px; font-size: 14.5px; font-weight: 600; line-height: 1.35;
    color: #2C2C2A; }
    .cc-precio { margin: 0 0 4px; font-size: 14px; color: #2C2C2A; }
    .cc-specs { margin: 0 0 11px; font-size: 12.5px; color: #6B6A66; }
    .cc-comparacion { display: flex; gap: 8px; margin: 0 0 13px; flex-wrap: wrap; }
    .cc-marca { font-size: 11.5px; font-weight: 600; padding: 3px 9px; border-radius: 6px; }
    .cc-mas { background: #E8F0F7; color: #1F4460; }
    .cc-menos, .cc-dist { background: #F5F3EE; color: #6B6A66; }
    .cc-enlace {
        display: block; text-align: center; text-decoration: none; background: #FF385C;
        color: #fff;
        font-size: 13.5px; font-weight: 600; padding: 9px 0; border-radius: 9px;
    }
    .cc-enlace:hover { background: #E82E51; }

    .cc-leyenda { display: flex; gap: 16px; flex-wrap: wrap; margin: 10px 0 0; font-size:
    12px; color: #6B6A66; }
    .cc-leyenda span { display: flex; align-items: center; gap: 6px; }
    .cc-leyenda i { width: 10px; height: 10px; border-radius: 3px; display: inline-block; }
</style>""",
    unsafe_allow_html=True,
)

def _miles(n):
    """Formatea con punto de millar en vez de coma (estilo español): 1523 -> '1.523'.
    Se usa número a número en vez de un .replace(",", ".") sobre la frase entera, para
    no convertir de paso una coma de puntuación real en punto."""
    return f"{n:,.0f}".replace(",", ".")

def _rango_tramo_estancia_texto(tramo):
    if tramo == "corta":
        return f"1 a {comparables.CORTA_MAX} noches"
    if tramo == "media":
        return f"{comparables.CORTA_MAX + 1} a {comparables.MEDIA_MAX} noches"
    return f"{comparables.MEDIA_MAX + 1} noches o más"

def _alcance_opciones():
    """(valor, etiqueta del radio, etiqueta del chip/frase) en orden barrio →
    distrito → ciudad. Sin nivel de barrio propio (Málaga, ver
    HAS_NEIGHBOURHOOD_LEVEL) se omite "barrio": ahí sería idéntico a
    "distrito"; comparables.conjunto_comparables ya hace esa misma caída
    internamente si igualmente se le pidiera."""
    geo_unit = features.geo_unit_word(CITY).lower()
    opciones = []
    if HAS_NEIGHBOURHOOD_LEVEL:
        opciones.append(("barrio", f"Tu {geo_unit}", f"tu {geo_unit}"))
    opciones.append(("distrito", "Tu distrito", "tu distrito"))
    opciones.append(("ciudad", "Toda la ciudad", "toda la ciudad"))
    return opciones

def _nombre_zona_alcance(alcance, neighbourhood, district, city_label):
    if alcance == "barrio":
        return neighbourhood
    if alcance == "distrito":
        return district
    return city_label

def _de_zona_texto(alcance, city):
    """del barrio/"del municipio" (contracción de+el, nunca "de el barrio"),
    "del distrito" o "de la ciudad", para frases tipo "la mediana {de_zona}."""
    if alcance == "barrio":
        return f"del {features.geo_unit_word(city).lower()}"
    if alcance == "distrito":
        return "del distrito"
    return "de la ciudad"

```

```

PASTILLA_VEREDICTO = {
    # (texto, fondo, color de texto). Sin rojo ni verde: caro no es malo ni barato es
    # bueno, depende de si el anfitrión quiere llenar o maximizar (tonos del HTML).
    "barato": ("VENDES BARATO", "#E8F0F7", "#2F6491"),
    "mercado": ("EN PRECIO DE MERCADO", "#F5F3EE", "#6B6A66"),
    "caro": ("VENDES CARO", "#DDE8F1", "#1F4460"),
}

TITULAR_VEREDICTO = {
    "barato": "Tu precio está por debajo del mercado",
    "mercado": "Tu precio está en línea con el mercado",
    "caro": "Tu precio está por encima del mercado",
}

def _estado_veredicto(diferencia, margen):
    if abs(diferencia) <= margen:
        return "mercado"
    return "barato" if diferencia < 0 else "caro"

def _explica_veredicto(estado, n_similar, precio_usuario, mediana, precios_comparables, de_zona,
es_prediccion_modelo):
    """Frase del veredicto: proporción ("8 de cada 10") y euros de diferencia para
    barato/caro (nunca la palabra "percentil"); pertenencia (no posición) más nota
    del margen para "mercado" – tal y como pide la Fase 2.

    es_prediccion_modelo distingue dos framings para barato/caro, a propósito:
    - Si el precio mostrado sigue siendo la recomendación del modelo (no lo ha tocado
    el usuario), defendemos su credibilidad explicando POR QUÉ puede discrepar de la
    mediana (ve detalles que la mediana no ve) – sin esto, un usuario podría leer la
    discrepancia como "la predicción falla".
    - Si el usuario ya escribió su propio precio, esa defensa sería falsa (el modelo
    nunca evaluó ese número) y, más importante, anularía la señal que esta pantalla
    existe para dar: puede que el usuario esté infra/sobrevalorando de verdad, y
    decirle "tranquilo, está justificado" sin poder demostrarlo sería un flaco favor.
    """
    diferencia = precio_usuario - mediana
    dif_abs = abs(diferencia)
    if estado == "mercado":
        p25, p75 = precios_comparables.quantile([0.25, 0.75])
        if dif_abs == 0:
            frase_diferencia = "Tu precio coincide con la mediana, la referencia más central
            posible."
        else:
            lado = "por debajo" if diferencia < 0 else "por encima"
            frase_diferencia = f"Estás {_miles(dif_abs)} € {lado} de la mediana, una diferencia
            pequeña para este mercado."
        return (
            f"Tu precio cae dentro de la <b>mitad central {de_zona}</b>: entre {_miles(p25)} y
            {_miles(p75)} € "
            f"se publica la mitad de los anuncios parecidos al tuyo. {frase_diferencia}"
        )

    n_de_10 = round((precios_comparables > precio_usuario).mean() * 10)
    # ^ proporción de comparables más caros que tu precio, en base 10 (nunca "percentil").
    plural = n_de_10 > 1
    solo = "solo " if n_de_10 <= 2 else ""
    verbo = "se publican" if plural else "se publica"
    sujeto = "más caros" if plural else "más caro"
    lado = "por debajo" if estado == "barato" else "por encima"
    cierre = (
        "el modelo ya tiene en cuenta detalles de tu anuncio que esta comparación no ve, "
        "así que la predicción no está mal: solo mira más allá de la mediana."
        if es_prediccion_modelo else
        "una diferencia real, no un error – la mediana no recoge las particularidades de tu
        anuncio."
    )
    return (
        f"De {'esos' if estado == 'barato' else 'los'} {_miles(n_similar)} anuncios,
        <b>{solo}{n_de_10}</b> de cada 10 "
        f"{verbo} {sujeto}</b>. Estás unos <b>{_miles(dif_abs)} € por noche</b> {lado} de la

```

```

        mediana {de_zona}: "
        f"{cierre}"
    )

def chart_veredicto(precios_comparables, precio_usuario, mediana, margen):
    """Gráfico de dos carriles que comparten escala horizontal: arriba, histograma de
    los comparables en 3 tonos según sus propios cuartiles (P25/P75); abajo, un carril
    con la mediana como marca, el margen del usuario como barra CON TOPES (nunca una
    franja a toda altura sobre el histograma: taparía las barras) y su precio como
    círculo con borde blanco."""
    precios = precios_comparables.dropna()
    p25, p75 = precios.quantile([0.25, 0.75])

    # Rango típico por valla de Tukey (P75 + 1.5×RIC), no min/max literal: unos pocos
    # anuncios de lujo con precios disparados (frecuentes en este dataset) estirarían el
    # eje hasta hacer ilegible el histograma del resto. Se amplía igualmente si hiciera
    # falta para que tu precio, tu margen y la mediana entren siempre en el gráfico.
    ric = p75 - p25
    valla_inf = max(precios.min(), p25 - 1.5 * ric)
    valla_sup = min(precios.max(), p75 + 1.5 * ric)
    lo = min(valla_inf, precio_usuario - margen, mediana)
    hi = max(valla_sup, precio_usuario + margen, mediana)
    colchon = (hi - lo) * 0.05 or 1
    dominio = [max(0, lo - colchon), hi + colchon]
    x_scale = alt.Scale(domain=dominio, nice=False, zero=False)

    counts, edges = np.histogram(precios, bins=20, range=(dominio[0], dominio[1]))
    medios = (edges[:-1] + edges[1:]) / 2
    tono = np.where(medios < p25, "bajo", np.where(medios > p75, "alto", "medio"))
    hist_df = pd.DataFrame({"x0": edges[:-1], "x1": edges[1:], "n": counts, "tono": tono})

    histograma = (
        alt.Chart(hist_df)
        .mark_bar()
        .encode(
            # bin="binned": sin esto Vega-Lite no siempre interpreta x/x2 como el ancho
            # real de cada barra ya pre-calculado y el histograma sale mal orientado.
            # title=None: como en la referencia, el eje no lleva un rótulo "Precio (€)"
            # aparte – el € ya va en cada marca de graduación (labelExpr).
            x=alt.X(
                "x0:Q", bin="binned", scale=x_scale, title=None,
                axis=alt.Axis(grid=False, tickCount=6, labelExpr="datum.value + ' €'"),
            ),
            x2="x1:Q",
            y=alt.Y("n:Q", axis=None),
            color=alt.Color(
                "tono:N",
                scale=alt.Scale(domain=["bajo", "medio", "alto"], range=["#C9D9E7", "#8FB4D2",
                "#5C8CB4"]),
                legend=None,
            ),
        ),
    )
    .properties(height=100)
)

carril_df = pd.DataFrame([
    "margen_x0": precio_usuario - margen, "margen_x1": precio_usuario + margen,
    "mediana": mediana, "precio": precio_usuario,
    "mediana_label": f"mediana {_miles(mediana)} €",
    "dom_x0": dominio[0], "dom_x1": dominio[1],
])

# data= una sola vez a nivel de alt.layer (no un alt.Chart(carril_df) por cada una de
# las 8 marcas): con Chart() separados sobre el mismo df, el spec resultante confundía
# al componente Vega de Streamlit al re-renderizar con datos nuevos ("Unrecognized data
# set", carril en blanco). Un único data compartido por todas las capas lo evita.
# y=16, no 8: en la referencia "Tu posición" queda a la altura de la pista (un rótulo
# de fila, pegado a ella), no como un título aparte flotando muy por encima.
etiqueta_carril = alt.Chart().mark_text(aligned="left", font_size=11, font_weight="bold",
color="#8B8A85").encode(
    x=alt.value(0), y=alt.value(16), text=alt.value("Tu posición"),
)

pista = alt.Chart().mark_rule(color="#EDEAE4", stroke_width=9, stroke_cap="round").encode(

```

```

        x=alt.X("dom_x0:Q", scale=x_scale, axis=None, title=None), x2="dom_x1:Q",
        y=alt.value(34),
    )
    barra_margen = alt.Chart().mark_rule(color="#FF385C", strokeWidth=4, opacity=0.32,
strokeCap="round").encode(
        x=alt.X("margen_x0:Q", scale=x_scale), x2="margen_x1:Q", y=alt.value(34),
    )
    tope_izq = alt.Chart().mark_rule(color="#FF385C", strokeWidth=1.5, opacity=0.55).encode(
        x=alt.X("margen_x0:Q", scale=x_scale), y=alt.value(27), y2=alt.value(41),
    )
    tope_der = alt.Chart().mark_rule(color="#FF385C", strokeWidth=1.5, opacity=0.55).encode(
        x=alt.X("margen_x1:Q", scale=x_scale), y=alt.value(27), y2=alt.value(41),
    )
    marca_mediana = alt.Chart().mark_rule(color="#6B6A66", strokeWidth=2).encode(
        x=alt.X("mediana:Q", scale=x_scale), y=alt.value(25), y2=alt.value(47),
    )
    etiqueta_mediana = alt.Chart().mark_text(fontSize=11.5, fontWeight="bold",
color="#4A4945").encode(
        x=alt.X("mediana:Q", scale=x_scale), y=alt.value(15), text="mediana_label:N",
    )
    punto_precio = alt.Chart().mark_point(
        size=170, filled=True, color="#FF385C", stroke="white", strokeWidth=2,
    ).encode(x=alt.X("precio:Q", scale=x_scale), y=alt.value(34))
    etiqueta_precio = alt.Chart().mark_text(fontSize=12.5, fontWeight="bold",
color="#C42843").encode(
        x=alt.X("precio:Q", scale=x_scale), y=alt.value(62),
        text=alt.value(f"tu precio {_miles(precio_usuario)} € · margen ±{_miles(margen)} €"),
    )
    carril = alt.layer(
        etiqueta_carril, pista, barra_margen, tope_izq, tope_der, marca_mediana,
        etiqueta_mediana,
        punto_precio, etiqueta_precio,
        data=carril_df,
    ).properties(height=78)

    return histograma.configure_view(stroke=None), carril.configure_view(stroke=None)

def _plural(n, singular, plural_):
    return f"{n} {singular}" if abs(n) == 1 else f"{n} {plural_}"

def _formato_tamano(bedrooms, bathrooms):
    hab = int(bedrooms) if pd.notna(bedrooms) else 0
    # "baños sin decimal (2.0 -> '2 baños')": Airbnb permite medios baños (1.5), pero para
    # esta lista se redondea al entero más cercano, igual que pide la Fase 4.
    ban = int(round(bathrooms)) if pd.notna(bathrooms) else 0
    return f"{hab} hab · {_plural(ban, 'baño', 'baños')}"

# Mismos tonos que la leyenda del mapa de referencia: el umbral es el margen del tramo
# de precio del usuario, el mismo que decide el veredicto (EN PRECIO DE MERCADO/BARATO/
# CARO), para que mapa y veredicto nunca se contradigan sobre qué es "parecido".
MAPA_COLOR_BARATO = [201, 217, 231]
MAPA_COLOR_PARECIDO = [127, 169, 203]
MAPA_COLOR_CARO = [47, 100, 145]
MAPA_COLOR_TUYO = [255, 56, 92]

def _color_precio_mapa(diferencia, margen):
    if diferencia < -margen:
        return MAPA_COLOR_BARATO
    if diferencia > margen:
        return MAPA_COLOR_CARO
    return MAPA_COLOR_PARECIDO

@st.cache_data(show_spinner=False)
def _comparables_ordenados(anuncio_usuario, comparables_df):
    """Un único orden de proximidad (comparables.ordenar_por_proximidad) para lista y
    mapa. Cacheado por filtros (el anuncio del usuario y el conjunto ya filtrado por
    comparables.conjunto_comparables), no por cuántas filas se muestren: pasar de 10 a
    50 en el selector de cantidad no debe volver a ordenar ni a filtrar nada."""

```

```

return comparables.ordenar_por_proximidad(anuncio_usuario, comparables_df)

def _tarjeta_anuncio_html(row, precio_usuario):
    """Card de un anuncio comparable: foto (o estado sin foto si no hay picture_url o
    la imagen no carga – las URLs de Inside Airbnb caducan a menudo), specs y las dos
    etiquetas de comparación (precio vs. el tuyo, distancia) que son el motivo de que
    esta card exista en esta pantalla."""
    diferencia = row["price"] - precio_usuario
    if diferencia > 0:
        chip_precio = f'<span class="cc-marca cc-mas">+{_miles(diferencia)} € que tu
        precio</span>'
    elif diferencia < 0:
        chip_precio = f'<span class="cc-marca cc-menos">-{_miles(abs(diferencia))} € que tu
        precio</span>'
    else:
        chip_precio = '<span class="cc-marca cc-menos">mismo precio que tú</span>'
    chip_dist = f'<span class="cc-marca cc-dist">a {row["distancia_m"]:.0f}
    m</span>'.replace(", ", ".")

    nombre = row.get("name")
    if isinstance(nombre, str) and nombre.strip():
        titulo = html.escape(nombre.strip())
    else:
        titulo = f"Alojamiento en {html.escape(str(row['neighbourhood_cleansed']))}"
    tipo_texto = ROOM_TYPE_ICONS[row["room_type"]].split(": ", 1)[1]
    specs = (
        f'{int(row["bedrooms"])} hab · {_plural(int(round(row["bathrooms"])), "baño", "baños")}' ·
        f' {_plural(int(row["accommodates"])), "huésped", "huéspedes"} · '
        f'mín. {_plural(int(row["minimum_nights"])), "noche", "noches"}'
    )

    picture_url = row.get("picture_url")
    if isinstance(picture_url, str) and picture_url.startswith("http"):
        foto_html = (
            '<div class="cc-foto-wrap">'
            f''
            '<div class="cc-sinfoto" style="display:none;"><span>Sin foto
            disponible</span></div>'
            "</div>"
        )
    else:
        foto_html = '<div class="cc-sinfoto"><span>Sin foto disponible</span></div>'

    enlace_html = ""
    listing_url = row.get("listing_url")
    if isinstance(listing_url, str) and listing_url.startswith("http"):
        enlace_html = (
            f'<a class="cc-enlace" href="{html.escape(listing_url, quote=True)}" '
            'target="_blank" rel="noopener">Ver en Airbnb →</a>'
        )

    return (
        f'<div class="cc-card">{foto_html}<div class="cc-cuerpo">'
        f'<p class="cc-titulo">{titulo}</p>'
        f'<p class="cc-precio">{_miles(row["price"])} € por noche · {tipo_texto}</p>'
        f'<p class="cc-specs">{specs}</p>'
        f'<div class="cc-comparacion">{chip_precio}{chip_dist}</div>'
        f'{enlace_html}'
        "</div></div>"
    )

def _cerrar_dialog_anuncio():
    # No borra comparar_selected_id (la selección de fila/punto es "pegajosa" a
    # propósito, para que sobreviva a cambios de vista): en su lugar recuerda para QUÉ id
    # ya se cerró el popup, así no se reabre solo en el siguiente rerun porque el propio
    # st.dataframe/pydeck sigue reportando esa misma selección como activa.
    st.session_state["comparar_dialog_dismissed_id"] =
    st.session_state.get("comparar_selected_id")

```

```

@st.dialog("Detalle del anuncio", on_dismiss=_cerrar_dialog_anuncio)
def _dialog_anuncio(row, precio_usuario):
    st.markdown(_tarjeta_anuncio_html(row, precio_usuario), unsafe_allow_html=True)

def _resumen_comparables(anuncio_usuario, comparables_df):
    """Bloque "con quién te comparo": una frase que afirma el contexto (nunca
    un panel de controles), un botón Ajustar que despliega el único filtro
    real de comparables.conjunto_comparables (el alcance geográfico – tipo,
    capacidad y estancia mínima son invariantes duros del propio anuncio, sin
    mecanismo de override), y el aviso ámbar si hizo falta ampliar el alcance
    para llegar a comparables.MIN_COMPARABLES. Devuelve el ResultadoComparables
    ya calculado para que el resto de la pantalla (veredicto, lista, mapa)
    consuma exactamente el mismo conjunto."""
    alcance_key = "comparar_alcance"
    opciones = _alcance_opciones()
    valores_validos = [v for v, _, _ in opciones]
    if st.session_state.get(alcance_key) not in valores_validos:
        st.session_state[alcance_key] = "distrito"

    resultado = comparables.conjunto_comparables(anuncio_usuario, comparables_df,
    st.session_state[alcance_key])

    etiqueta_control = {v: label for v, label, _ in opciones}
    etiqueta_chip = {v: chip for v, _, chip in opciones}

    zona = _nombre_zona_alcance(
        resultado.alcance_final, anuncio_usuario["neighbourhood"], anuncio_usuario["district"],
        CITY_LABEL,
    )
    tramo = comparables.tramo_estancia(anuncio_usuario["minimum_nights"])
    tipo_plural = ROOM_TYPE_PLURAL[anuncio_usuario["room_type"]]
    n_texto = _miles(len(resultado.df))

    frase = (
        f"Te comparo con <b>{n_texto}</b> anuncios reales</b> de {html.escape(str(zona))}: "
        f"{tipo_plural}, de capacidad parecida a la tuya y con estancia mínima {tramo}, como el "
        f"tuyo."
    )

    acc_lo_texto = max(1, anuncio_usuario["accommodates"] - 1)
    acc_hi_texto = min(16, anuncio_usuario["accommodates"] + 1)
    chips_datos = [
        ("Tipo", ROOM_TYPE_ICONS[anuncio_usuario["room_type"]].split(": ", 1)[1].lower()),
        ("Huéspedes", f"{acc_lo_texto} a {acc_hi_texto}"),
        ("Estancia mínima", _rango_tramo_estancia_texto(tramo)),
        ("Alcance", etiqueta_chip[resultado.alcance_final]),
    ]
    chips_html = "".join(f'<span class="cmp-chip">{etiqueta} · <b>{valor}</b></span>' for
    etiqueta, valor in chips_datos)

    with st.container(key="comparar_resumen"):
        col_frase, col_boton = st.columns([6, 1])
        with col_frase:
            st.markdown(f'<p class="cmp-frase">{frase}</p>', unsafe_allow_html=True)
        with col_boton:
            abierto = st.session_state.get("comparar_ajustar_abierto", False)
            if st.button("Ajustar ■" if abierto else "Ajustar ■", key="comparar_ajustar_btn"):
                st.session_state["comparar_ajustar_abierto"] = not abierto
                abierto = not abierto

            st.markdown(f'<div class="cmp-criterios">{chips_html}</div>', unsafe_allow_html=True)

    if abierto:
        # index= explícito además de key=: sin él, un radio que se monta por primera
        # vez en un rerun posterior (aquí, solo al abrir Ajustar) ignora el valor ya
        # presente en session_state y arranca en la primera opción (bug/particularidad
        # de Streamlit con widgets condicionales, no específico de esta pantalla).
        st.radio(
            "Alcance", valores_validos,
            index=valores_validos.index(st.session_state[alcance_key]),

```

```

        format_func=lambda v: etiqueta_control[v], horizontal=True, key=alcance_key,
        help="Tipo de alojamiento, capacidad y estancia mínima salen siempre de tu propio
        "
        "anuncio: el único ajuste real aquí es cuánto abrir la zona de comparación.",
    )

    if resultado.se_amplio:
        st.markdown(
            f'<div class="cmp-aviso-ampliado"><b>He ampliado el alcance.</b> En '
            f'{etiqueta_chip[resultado.alcance_solicitado]} solo había
            {resultado.n_en_alcance_original} "
            "anuncios parecidos al tuyo, muy pocos para calcular un rango fiable, así que he
            pasado a "
            f'{etiqueta_chip[resultado.alcance_final]}. Puedes volver a un alcance más
            estrecho en "
            "Ajustar, pero el resultado será menos sólido.</div>',
            unsafe_allow_html=True,
        )

    return resultado

def page_comparar():
    st.header("Comparar")

    if "confirmed_inputs" not in st.session_state:
        _empty_state_sin_formulario(
            "Todavía no has descrito tu anuncio. Rellena el Formulario y pulsa "
            "'Predecir precio' para poder compararlo con anuncios reales."
        )
    return

    _inject_comparar_css()
    with st.container(key="comparar_page_wrap"):
        _page_comparar_body()

def _usar_precio_recomendado():
    st.session_state["comparar_precio_usuario"] = int(round(pred))

def _page_comparar_body():
    anuncio_usuario = {
        "room_type": room_type,
        "accommodates": accommodates,
        "bedrooms": bedrooms,
        "bathrooms": bathrooms,
        "minimum_nights": minimum_nights,
        "latitude": latitude,
        "longitude": longitude,
        "neighbourhood": neighbourhood,
        "district": district,
    }
    # Un único conjunto de comparables (comparables.conjunto_comparables) alimenta el
    # resumen de arriba, el veredicto y el mapa de abajo: los tres miran lo mismo, en vez
    # de que cada bloque calcule "sus" comparables con un filtro ligeramente distinto.
    resultado = _resumen_comparables(anuncio_usuario, comparables_all)
    similar = resultado.df
    n_similar = len(similar)
    zona_veredicto = _nombre_zona_alcance(resultado.alcance_final, neighbourhood, district,
    CITY_LABEL)
    # Valor de partida para el mapa de más abajo si no hay comparables con los que armar
    # el veredicto (n_similar == 0): se sobrescribe con el precio real del usuario dentro
    # del bloque de veredicto en cuanto hay comparables sobre los que mostrarlo.
    precio_usuario = pred

    veredicto_container = st.container(border=True, key="section_comparar_veredicto")
    mapa_container = st.container(border=True, key="section_comparar_mapa")

    with veredicto_container:
        st.markdown("#### :material/fact_check: Tu precio frente al mercado")

        if n_similar < 10:

```

```

st.warning(
    f"Solo {n_similar} anuncios similares – abre el alcance en Ajustar para una
    comparación más fiable."
)

if n_similar == 0:
    st.caption("No hay anuncios similares todavía para comparar tu precio con el
    mercado.")
else:
    # El precio que se juzga es el TUYO, nunca la predicción del modelo (que aquí
    # solo sirve de valor de partida editable): juzgar la predicción sería evaluar
    # nuestra propia recomendación en vez del precio real que vas a publicar.
    if st.session_state.get("comparar_precio_base") != pred:
        st.session_state["comparar_precio_usuario"] = int(round(pred))
        st.session_state["comparar_precio_base"] = pred

    with st.container(key="comparar_precio_input"):
        col_precio, col_usar = st.columns([1, 1.3])
        with col_precio:
            precio_usuario = st.number_input(
                "¿Qué precio cobras (o piensas cobrar) por noche?",
                min_value=1, step=5,
                key="comparar_precio_usuario",
                help=f"Por defecto, la predicción del modelo para tu anuncio:
                {_miles(pred)} €. "
                "Cámbialo si vas a publicar (o ya publicas) a otro precio.",
            )
        with col_usar:
            if int(round(precio_usuario)) != int(round(pred)):
                # on_click, no un "if st.button(...)" con la asignación dentro: el
                # número ya se instanció más arriba en esta misma ejecución, así que
                # tocar session_state[su key] aquí lanzaría StreamlitAPIException
                # ("cannot be modified after the widget... is instantiated"). Un
                # callback on_click corre antes del siguiente rerun, no en este.
                st.button(
                    "Usar precio predicho", key="comparar_precio_reset_btn",
                    on_click=_usar_precio_recomendado,
                )

    mediana = similar["price"].median()
    margen = mae_for_price(CITY, precio_usuario)["mae"]
    diferencia = precio_usuario - mediana
    estado = _estado_veredicto(diferencia, margen)
    texto_pastilla, fondo_pastilla, color_pastilla = PASTILLA_VEREDICTO[estado]
    de_zona = _de_zona_texto(resultado.alcance_final, CITY)
    explica = _explica_veredicto(
        estado, n_similar, precio_usuario, mediana, similar["price"], de_zona,
        es_prediccion_modelo=(int(round(precio_usuario)) == int(round(pred))),
    )

    st.markdown(
        (
            f'<div class="cmp-cabecera"><div>'
            f'<span class="cmp-pastilla"'
            f'style="background:{fondo_pastilla};color:{color_pastilla};">'
            f'{{texto_pastilla}}</span>'
            f'<p class="cmp-titular">{{TITULAR_VEREDICTO[estado]}}</p>'
            f'<p class="cmp-explica">{{explica}}</p>'
            f'</div>'
            f'<div class="cmp-par">'
            f'<p class="cmp-par-rot">Tu precio</p>'
            f'<p class="cmp-par-cifra cmp-tuyo">{{_miles(precio_usuario)}} €</p>'
            f'<p class="cmp-par-rot">Mediana {{de_zona}}</p>'
            f'<p class="cmp-par-cifra">{{_miles(mediana)}} €</p>'
            f'</div></div>'
        ),
        unsafe_allow_html=True,
    )

    histograma, carril = chart_veredicto(similar["price"], precio_usuario, mediana,
    margen)
    st.altair_chart(histograma, use_container_width=True, theme=None)
    st.altair_chart(carril, use_container_width=True, theme=None)

```

```

# Leyenda de los 3 tonos del histograma: cuartiles de TODO el mercado (P25/P75),
# no comparación con tu precio – por eso el texto no dice "que tú", a diferencia
# de la leyenda de precios del mapa de más abajo, que sí compara contigo.
st.markdown(
    '<div class="cmp-leyenda-hist">'
    '<span><i style="background:#C9D9E7"></i> más económicos</span>'
    '<span><i style="background:#8FB4D2"></i> precio típico</span>'
    '<span><i style="background:#5C8CB4"></i> más caros</span>'
    "</div>",
    unsafe_allow_html=True,
)

if estado == "mercado":
    dif_abs = abs(diferencia)
    if dif_abs == 0:
        nota = "Tu precio coincide con la mediana: no hay margen que discutir."
    else:
        lado = "barato" if diferencia < 0 else "caro"
        nota = (
            f"En el gráfico de arriba se ve por qué no decimos que vendas {lado}: "
            "<b>la marca de la mediana cae dentro de tu barra de margen</b>. Con la "
            "precisión "
            f"que tiene el modelo en tu tramo, esos {_miles(dif_abs)} € no se "
            "distinguen del ruido."
        )
    st.markdown(f'<div class="cmp-nota-margen">{nota}</div>', unsafe_allow_html=True)

if n_similar == 0:
    with mapa_container:
        st.markdown("#### :material/list: Los anuncios uno a uno")
        st.caption("Todavía no hay anuncios similares que listar o situar en el mapa.")
    return

with mapa_container:
    col_titulo, col_vista = st.columns([3, 1.4])
    with col_titulo:
        st.markdown(
            '<p class="cmp-anuncios-titulo">Los anuncios uno a uno</p>'
            '<p class="cmp-anuncios-sub">Ordenados por parecido a tu anuncio</p>',
            unsafe_allow_html=True,
        )
    with col_vista:
        vista = st.segmented_control(
            "Vista", ["Lista", "Mapa"], default="Lista", key="comparar_vista",
            label_visibility="collapsed",
        )
    vista = vista or "Lista"

ordenados = _comparables_ordenados(anuncio_usuario, similar)
seleccionado_id = st.session_state.get("comparar_selected_id")

if vista == "Lista":
    PAGINA_SIZE = 10
    # Tope duro de 200: un listado de miles de filas no aporta nada y sí que pesa
    # (igual criterionos.head(min(len(ordenados), 200))
    total_paginas = max(1, math.ceil(len(fuente) / PAGINA_SIZE))

    # Si cambia el conjunto de comparables (alcance, o un nuevo anuncio predicho),
    # la página vuelve a la 1: seguir en la página 7 de un listado distinto no
    # significa nada para el usuario.
    huella_pagina = (resultado.alcance_final, n_similar)
    if st.session_state.get("comparar_pagina_huella") != huella_pagina:
        st.session_state["comparar_pagina"] = 0
        st.session_state["comparar_pagina_huella"] = huella_pagina
    pagina = min(st.session_state.get("comparar_pagina", 0), total_paginas - 1)

    mostrados = fuente.iloc[pagina * PAGINA_SIZE:(pagina + 1) *
PAGINA_SIZE].reset_index(drop=True)

    tabla = pd.DataFrame({
        "Barrio": mostrados["neighbourhood_cleansed"].values,
        "Tamaño": [

```

```

        _formato_tamano(b, ba) for b, ba in zip(mostrados["bedrooms"],
        mostrados["bathrooms"])
    ],
    "Distancia": mostrados["distancia_m"].round().astype(int),
    "Precio": mostrados["price"].round().astype(int),
    "vs. el tuyo": (mostrados["price"] - precio_usuario).round().astype(int),
})

def _color_diferencia(v):
    return f"color: {'#2F6491' if v > 0 else '#8B8A85'}; font-weight: 600;"

estilo = (
    tabla.style
    .format({
        "Distancia": lambda v: f"{v:,.0f} m".replace(",", "."),
        "Precio": lambda v: f"{v:,.0f} €".replace(",", "."),
        "vs. el tuyo": lambda v: (f"+{v:,.0f} €" if v > 0 else f"{v:,.0f} €").replace(",", "."),
    })
    .map(_color_diferencia, subset=["vs. el tuyo"])
    .set_properties(subset=["Precio"], **{"font-weight": "700"})
)

# La fila seleccionada (si está en esta página) se resalta entera, no solo la
# celda en la que se hizo clic: st.dataframe solo dibuja su propio borde de
# selección alrededor de la celda, así que el resto de la fila lo pintamos
# nosotros con la Styler, en el mismo rosa suave que "activa" en la referencia.
filas_resaltadas = mostrados.index[mostrados["id"] == seleccionado_id].tolist()
if filas_resaltadas:
    estilo = estilo.apply(
        lambda fila: ["background-color: #FFF1F4"] * len(fila)
        if fila.name == filas_resaltadas[0] else [""] * len(fila),
        axis=1,
    )

# selection_mode="single-cell", no "single-row": una selección por fila
# dibuja una columna de checkboxes que no queríamos. Con celdas, clicar
# cualquier celda de la fila selecciona (sin checkbox visible) y la posición
# de fila se lee igual en selection.cells (fila, columna).
seleccion = st.dataframe(
    estilo, hide_index=True, use_container_width=True,
    on_select="rerun", selection_mode="single-cell", key="comparar_tabla_sel",
)

celdas_sel = seleccion.selection.cells if seleccion and seleccion.selection else []
if celdas_sel:
    fila_idx = celdas_sel[0][0]
    nuevo_id = mostrados.iloc[fila_idx]["id"]
    if nuevo_id != seleccionado_id:
        # Un rerun explícito para que el resaltado de fila completa (calculado
        # arriba, antes de dibujar la tabla) ya salga correcto en el clic actual,
        # en vez de ir un clic por detrás. No entra en bucle: en el siguiente
        # paso nuevo_id ya coincide con seleccionado_id y no se vuelve a llamar.
        st.session_state["comparar_selected_id"] = nuevo_id
        st.rerun()
    seleccionado_id = nuevo_id

col_prev, col_info, col_next = st.columns([1, 2, 1])
with col_prev:
    if st.button(
        "← Anterior", key="comparar_pagina_prev", disabled=pagina == 0,
        use_container_width=True,
    ):
        st.session_state["comparar_pagina"] = pagina - 1
        st.rerun()
with col_info:
    st.markdown(
        f'<p class="cmp-pagina-info">Página {pagina + 1} de {total_paginas}</p>',
        unsafe_allow_html=True,
    )
with col_next:
    if st.button(
        "Siguiente →", key="comparar_pagina_next",
        disabled=pagina >= total_paginas - 1, use_container_width=True,
    ):
        st.session_state["comparar_pagina"] = pagina + 1

```

```

st.rerun()

else: # Mapa: layers y geojson solo se calculan si esta vista está seleccionada.
    # Tope duro de 200 en "Todos": un scatter de miles de puntos no aporta nada y
    # sí que pesa. Este selector es propio del mapa (en Lista se pagina en su
    # lugar) y no afecta al veredicto (mediana/cuartiles ya se calcularon arriba
    # sobre `similar` completo, antes de este punto).
    tope_todos = min(n_similar, 200)
    etiqueta_todos = (
        f"Todos ({_miles(tope_todos)} de {_miles(n_similar)})"
        if n_similar > 200 else f"Todos ({_miles(n_similar)})"
    )
    opciones_cantidad = [("10 más parecidos", 10), ("25", 25), ("50", 50),
        (etiqueta_todos, tope_todos)]
    etiquetas_cantidad = [et for et, _ in opciones_cantidad]
    valores_cantidad = [v for _, v in opciones_cantidad]
    idx_sel = st.segmented_control(
        "Mostrar", list(range(len(opciones_cantidad))), format_func=lambda i:
            etiquetas_cantidad[i],
            default=0, key="comparar_cantidad_idx", label_visibility="collapsed",
    )
    n_mostrar = valores_cantidad[idx_sel if idx_sel is not None else 0]
    mostrados = ordenados.head(n_mostrar).reset_index(drop=True)

    listing_point = pd.DataFrame([
        {"id": -1, "latitude": latitude, "longitude": longitude, "price": precio_usuario,
        "name": "Tu anuncio", "picture_url": "", "room_type": room_type,
        "bedrooms": bedrooms, "bathrooms": bathrooms, "minimum_nights": minimum_nights,
        "accommodates": accommodates, "listing_url": ""},
    ])

    hover_card_html = """
<div style="max-width:220px">
  
  <b>{name}</b><br/>
  {price:.0f} € / noche · {room_type}<br/>
  {bedrooms} hab · {bathrooms} baños · min. {minimum_nights} noches
</div>
"""
    # "name" y "picture_url" vienen de datos reales de Airbnb (título que puso el
    # anfitrión), no de nada que controlemos nosotros: sin escapar, un título con
    # caracteres HTML se interpreta tal cual en el tooltip de pydeck (renderiza HTML
    # en crudo, sin sanitizar). Se escapan antes de montar la plantilla.
    def _safe_tooltip_fields(row):
        row = dict(row)
        row["name"] = html.escape(str(row.get("name", "")))
        row["picture_url"] = html.escape(str(row.get("picture_url", "")), quote=True)
        for field in ("bedrooms", "minimum_nights"):
            if field in row and pd.notna(row[field]):
                row[field] = int(row[field])
        return row

    listing_point["tooltip_html"] =
    [hover_card_html.format(**_safe_tooltip_fields(listing_point.iloc[0]))]
    mostrados_mapa = mostrados.copy()
    mostrados_mapa["tooltip_html"] = [
        hover_card_html.format(**_safe_tooltip_fields(row._asdict())) for row in
        mostrados_mapa.itertuples()
    ]
    # Color por precio respecto al TUYO, con el margen del tramo como umbral: el
    # mismo umbral que decide el veredicto, para que mapa y veredicto no puedan
    # contradecirse sobre qué cuenta como "parecido".
    mostrados_mapa["color"] = [
        _color_precio_mapa(p - precio_usuario, margen) for p in mostrados_mapa["price"]
    ]

    # Contorno de la zona activa (barrio o distrito), solo como contexto: gris medio,
    # relleno casi transparente para poder hacer hover en toda el área, sin competir
    # visualmente con los puntos ni con el rojo del usuario. En "Toda la ciudad" no
    # hay una zona única que dibujar, así que no se añade ninguna capa.
    zone_layer = None
    if resultado.alcance_final == "barrio":

```

```

        zone_name = neighbourhood
        zone_features = [
            f for f in load_neighbourhood_geojson(CITY)["features"]
            if f["properties"]["neighbourhood"] == neighbourhood
        ]
    elif resultado.alcance_final == "distrito":
        zone_name = district
        zone_features = [
            f for f in load_district_geojson(CITY)["features"] if
            f["properties"]["district"] == district
        ]
    else:
        zone_name = None
        zone_features = []

    if zone_features:
        zone_geojson = {
            "type": "FeatureCollection",
            "features": [
                {**f, "properties": {**f["properties"], "tooltip_html":
                    f"<b>{zone_name}</b>"}},
                for f in zone_features
            ],
        }
        zone_layer = pdk.Layer(
            "GeoJsonLayer",
            id="zone-out line",
            data=zone_geojson,
            stroked=True,
            filled=True,
            get_fill_color=[154, 160, 166, 20],
            get_line_color=[110, 115, 120, 220],
            get_line_width=2,
            line_width_min_pixels=2,
            line_width_units="'pixels'",
            pickable=True,
        )

    layers = []
    if zone_layer is not None:
        layers.append(zone_layer)
    layers.append(
        pdk.Layer(
            "ScatterplotLayer",
            id="comparables",
            data=mostrados_mapa,
            get_position="[longitude, latitude]",
            get_radius=8,
            radius_units="'pixels'",
            radius_min_pixels=6,
            radius_max_pixels=14,
            get_fill_color="color",
            get_line_color=[255, 255, 255],
            line_width_min_pixels=1,
            stroked=True,
            pickable=True,
            auto_highlight=True,
            highlight_color=[255, 215, 0, 230],
        )
    )
    layers.append(
        pdk.Layer(
            "ScatterplotLayer",
            id="your-listing",
            data=listing_point,
            get_position="[longitude, latitude]",
            get_radius=10,
            radius_units="'pixels'",
            radius_min_pixels=8,
            radius_max_pixels=16,
            get_fill_color=MAPA_COLOR_TUYO + [230],
            get_line_color=[255, 255, 255],
            line_width_min_pixels=2,

```

```

        stroked=True,
    )
)
view_state = pdk.ViewState(latitude=latitude, longitude=longitude, zoom=13)
tooltip = {
    "html": "{tooltip_html}",
    "style": {
        "backgroundColor": "white", "color": "black", "fontSize": "12px",
        "borderRadius": "8px", "padding": "8px", "boxShadow": "0 2px 8px
        rgba(0,0,0,0.25)",
    },
}
map_state = st.pydeck_chart(
    pdk.Deck(layers=layers, initial_view_state=view_state, tooltip=tooltip,
    map_style="light"),
    on_select="rerun",
    selection_mode="single-object",
    height=420,
    key="comparar_map_sel",
)
if map_state and map_state.selection:
    objetos = map_state.selection.get("objects", {}).get("comparables", [])
    if objetos:
        seleccionado_id = objetos[0]["id"]
        st.session_state["comparar_selected_id"] = seleccionado_id

st.markdown(
    '<div class="cc-leyenda">'
    f'<span><i style="background:rgb({",".join(map(str, MAPA_COLOR_BARATO))})"></i>'
    'más baratos que tú</span>'
    f'<span><i style="background:rgb({",".join(map(str, MAPA_COLOR_PARECIDO))})"></i>'
    'precio parecido</span>'
    f'<span><i style="background:rgb({",".join(map(str, MAPA_COLOR_CARO))})"></i>'
    'más caros que tú</span>'
    f'<span><i style="background:rgb({",".join(map(str, MAPA_COLOR_TUYO))})"></i>'
    'tu anuncio</span>'
    "</div>",
    unsafe_allow_html=True,
)

# Popup encima del contenido (st.dialog), no una tarjeta empujando el resto de la
# página hacia abajo. comparar_dialog_dismissed_id evita que reaparezca solo en el
# siguiente rerun: la selección de fila/punto es "pegajosa" a propósito (sobrevive
# a cambios de vista), así que sin ese guardado el popup se reabría cada vez.
if (
    seleccionado_id is not None and seleccionado_id != -1
    and seleccionado_id != st.session_state.get("comparar_dialog_dismissed_id")
):
    fila = ordenados[ordenados["id"] == seleccionado_id]
    if not fila.empty:
        _dialog_anuncio(fila.iloc[0], precio_usuario)

MERCADO_METRICS = {
    "Precio medio": {
        "col": "avg_price", "kind": "sequential", "legend_title": "Precio medio (€)",
        "value_fmt": lambda v: f"{v:,.0f} €", "tooltip_fmt": lambda v: f"precio medio {v:,.0f}
        €",
    },
    "Ocupación media anual": {
        "col": "avg_occupancy", "kind": "sequential", "legend_title": "Ocupación media
        (noches/año)",
        "value_fmt": lambda v: f"{v:,.0f} noches", "tooltip_fmt": lambda v: f"ocupación media
        {v:,.0f} noches",
    },
    "Densidad de anuncios": {
        "col": "n_listings", "kind": "sequential", "legend_title": "Nº de anuncios",
        "value_fmt": lambda v: f"{v:,.0f}", "tooltip_fmt": lambda v: f"{v:,.0f} anuncio" + (" " if
        v == 1 else "s"),
    },
    "Valoración media": {
        "col": "avg_rating", "kind": "sequential", "legend_title": "Valoración media (1-5)",
        "value_fmt": lambda v: f"{v:.2f}", "tooltip_fmt": lambda v: f"valoración media {v:.2f}",
    },
}

```

```

},
# Divergente (centro fijo en 0.5) SOLO para variables genuinamente binarias de dos
# polos. "% de corta estancia" lo es (turístico <=30 noches vs temporada >30). Las
# demás proporciones (vivienda entera, licencia) resumen una sola categoría sobre el
# total y no tienen un "polo opuesto" real (vivienda entera tiene 3 alternativas
# distintas, no 1), así que van con el mismo secuencial que el resto de la página.
"% de corta estancia (turístico vs temporada)": {
    "col": "pct_corta_estancia", "kind": "diverging", "legend_title": "% de corta estancia",
    "colors": (MARKET_COLOR, GRAY_MUTED, BLUE), "labels": ("Temporada", "Turístico"),
    "tooltip_fmt": lambda v: (
        lambda pct: f"{pct:.0f}% corta estancia · {100 - pct:.0f}% temporada"
    )(round(v * 100)),
    "footnote": "Los dos mercados (turístico vs temporada) se explican en **Modelo**.",
},
"% de vivienda entera": {
    "col": "pct_entire_home", "kind": "sequential", "legend_title": "% de vivienda entera",
    "value_fmt": lambda v: f"{v * 100:,.0f} %", "tooltip_fmt": lambda v: f"{v * 100:.0f}%
vivienda entera",
},
"% con licencia": {
    "col": "pct_license", "kind": "sequential", "legend_title": "% con licencia turística",
    "value_fmt": lambda v: f"{v * 100:,.0f} %", "tooltip_fmt": lambda v: f"{v * 100:.0f}% con
licencia",
},
}

def page_mercado():
    st.header("Mapa del mercado")

    with st.container(border=True, key="section_mercado_mapa"):
        st.markdown("#### :material/public: Mapa por barrio")

        col_map, col_controls = st.columns([2.2, 1.3])

        with col_controls:
            metric_choice = st.selectbox("Ver por", list(MERCADO_METRICS.keys()),
            key="mercado_metric")
            metric = MERCADO_METRICS[metric_choice]

            st.markdown("***Filtrar**")
            room_filter_choice = st.pills(
                "Filtrar por tipo de alojamiento",
                ["Todos"] + list(ROOM_TYPE_ICONS.values()),
                default="Todos",
                key="mercado_room_filter",
            )
            room_type_filter = (
                "Todos" if room_filter_choice in (None, "Todos") else
                ROOM_TYPE_LABELS[room_filter_choice]
            )

            accommodates_filter = st.slider(
                "Filtrar por nº de huéspedes",
                1, 16, (1, 16),
                key="mercado_accommodates_filter",
            )

            bedrooms_filter = st.slider(
                "Filtrar por nº de dormitorios",
                0, BEDROOMS_SLIDER_MAX, (0, BEDROOMS_SLIDER_MAX),
                key="mercado_bedrooms_filter",
                help=f"{BEDROOMS_SLIDER_MAX} = {BEDROOMS_SLIDER_MAX} o más dormitorios.",
            )

            col = metric["col"]
            agg = neighbourhood_market_agg(
                CITY, room_type_filter, accommodates_filter, bedrooms_filter
            ).set_index("neighbourhood")
            geojson = load_neighbourhood_geojson(CITY)

            if metric["kind"] == "sequential":
                vmin, vmax = float(agg[col].min()), float(agg[col].max())

```

```

        colormap = cm.LinearColormap(colors=[WHATIF_ALT, RED], vmin=vmin, vmax=vmax)
    else:
        # Divergente con el neutro fijado en 0.5, no en la media de los datos: el
        # gris debe significar siempre "repartido 50/50", no "valor típico". Cada
        # métrica divergente trae sus propios colores de polo (metric["colors"]),
        # para no confundir "turístico/temporada" con "habitación/vivienda entera".
        vmin, vmax = 0.0, 1.0
        low_color, mid_color, high_color = metric["colors"]
        colormap = cm.LinearColormap(colors=[low_color, mid_color, high_color], index=[0,
        0.5, 1], vmin=0, vmax=1)

features = []
for feat in geojson["features"]:
    name = feat["properties"]["neighbourhood"]
    props = dict(feat["properties"])
    # Un barrio puede tener anuncios pero ningún valor válido para esta métrica
    # en concreto (p.ej. "Valoración media" si ninguno tiene reseñas todavía), o
    # quedarse con muy pocos anuncios tras los filtros: en ambos casos se trata
    # como "sin datos", en vez de un 0/NaN que rompa el colormap o una media poco
    # fiable calculada sobre 2-3 casos.
    has_enough = name in agg.index and agg.loc[name, "n_listings"] >=
    MIN_LISTINGS_PER_BARRIO
    value = agg.loc[name, col] if has_enough else None
    if value is not None and pd.notna(value):
        props["fill_color"] = colormap(float(value))
        props["tooltip_text"] = f"{name} · {metric['tooltip_fmt'](value)}"
    else:
        props["fill_color"] = "#D8D5CC"
        props["tooltip_text"] = f"{name} · sin datos"
    features.append(**feat, "properties": props)
geojson_colored = {"type": "FeatureCollection", "features": features}

_map_zoom = {"barcelona": 12, "madrid": 11, "malaga": 11, "sevilla": 12, "mallorca": 9,
"euskadi": 8}.get(CITY, 12)
fmap = folium.Map(location=list(CITY_CENTER), zoom_start=_map_zoom,
tiles="OpenStreetMap")
folium.GeoJson(
    geojson_colored,
    style_function=lambda feat: {
        "fillColor": feat["properties"]["fill_color"],
        "color": "#FFFFFF",
        "weight": 1,
        "fillOpacity": 0.85,
    },
    highlight_function=lambda feat: {"weight": 2, "color": "#3A3A37"},
    tooltip=folium.GeoJsonTooltip(fields=["tooltip_text"], aliases=[""], labels=False,
    sticky=True),
).add_to(fmap)

with col_map:
    st_folium(fmap, use_container_width=True, height=480, returned_objects=[],
    key="mercado_map")

with col_controls:
    # Leyenda propia en vez de la de branca (su ancho se estira con el mapa y el
    # texto acaba solapado con la barra de color): control total del layout aquí.
    if metric["kind"] == "sequential":
        st.markdown(
            f"""
            <div style="margin-top:4px;">
            <div style="font-size:12px;color:#6B6862;margin-bottom:4px;">{metric['legende_title']}

```

```

    )
    else:
        low_color, mid_color, high_color = metric["colors"]
        label_low, label_high = metric["labels"]
        st.markdown(
            f"""
            <div style="margin-top:4px;">
            <div style="font-size:12px;color:#6B6862;margin-bottom:4px;">{metric['legend_title']}

```

app/features.py

Reconstruye el vector de features que espera price_model.pkl a partir de las respuestas del formulario.

```
"""Reconstruye el vector de features que espera price_model.pkl (uno por ciudad,
ver set_city) a partir de un formulario simple, sin obligar al usuario a conocer
columnas derivadas como neighbourhood_price_encoded o distance_to_center_km.
"""

import json
import math
from pathlib import Path

import numpy as np
import pandas as pd
import streamlit as st
from sklearn.neighbors import BallTree

APP_DIR = Path(__file__).resolve().parent
PROJECT_DIR = APP_DIR.parent

# Ciudades soportadas por la app. Añadir una nueva ciudad implica: datos en
# data/raw/<city>/, notebooks en notebooks/<city>/, modelo en models/<city>/ y
# assets en app/assets/<city>/ (mismo esquema para las tres), más una entrada
# en cada uno de estos diccionarios.
CITIES = ["barcelona", "madrid", "valencia", "malaga", "sevilla", "mallorca", "euskadi"]
CITY_LABELS = {
    "barcelona": "Barcelona", "madrid": "Madrid", "valencia": "Valencia", "malaga": "Málaga",
    "sevilla": "Sevilla", "mallorca": "Mallorca", "euskadi": "Euskadi",
}
CITY_CENTERS = {
    "barcelona": (41.3874, 2.1686), # Plaça Catalunya
    "madrid": (40.4168, -3.7038), # Puerta del Sol
    "valencia": (39.4699, -0.3763), # Plaza del Ayuntamiento
    "malaga": (36.7213, -4.4214), # Plaza de la Constitución
    "sevilla": (37.3826, -5.9963), # Catedral de Sevilla
    "mallorca": (39.5673, 2.6474), # Catedral de Palma (La Seu)
    # Centro geográfico del bounding box de anuncios (no un hito real): Euskadi no
    # tiene un único centro (ver distance_to_center_km más abajo) y este punto solo
    # se usa para centrar el mapa de "Explora el mercado" de forma que las 3
    # provincias queden visibles a la vez.
    "euskadi": (42.9646, -2.5991),
}
CITY_CENTER_NAMES = {
    "barcelona": "Plaça Catalunya", "madrid": "Puerta del Sol", "valencia": "Plaza del
Ayuntamiento",
    "malaga": "Plaza de la Constitución", "sevilla": "Catedral de Sevilla",
    "mallorca": "Catedral de Palma (La Seu)",
    "euskadi": "la capital de provincia más cercana (Bilbao, Donostia-San Sebastián o
Vitoria-Gasteiz)",
}
DEFAULT_DISTRICT = {
    "barcelona": "Eixample", "madrid": "Centro", "valencia": "Ciutat Vella", "malaga": "Centro",
    "sevilla": "Casco Antiguo", "mallorca": "Palma de Mallorca", "euskadi": "Vizcaya",
}
# Distritos con muestra pequeña en el test de 05_model_evaluation.ipynb de cada
# ciudad, para el aviso de confianza en confidence_flags().
LOW_SAMPLE_DISTRICTS = {
    "barcelona": ("Horta-Guinardó", "Nou Barris"),
    "madrid": ("Vicálvaro", "Moratalaz", "Villa de Vallecas"),
    "valencia": ("Poblats del Nord", "Benimaclet", "Poblats de l'Oest"),
    "malaga": ("Puerto de la Torre", "Campanillas", "Teatinos-Universidad"),
    "sevilla": ("Macarena - Norte", "Este - Alcosa - Torreblanca", "Cerro - Amate"),
    "mallorca": ("Santa Eugènia", "Estellencs", "Consell"),
    # Las 3 provincias tienen muestra grande de sobra en test (89-605 filas, ver
    # 05_model_evaluation.ipynb de Euskadi, sección 4.2): ninguna necesita este aviso.
    "euskadi": (),
}

# Palabra para la unidad geográfica más fina que se pide en el Formulario, usada en
```

```

# los textos de la home: "Barrio" en las ciudades, "Municipio" en Euskadi (que no es
# una sola ciudad con barrios, sino una región de varios municipios independientes).
GEO_UNIT_WORD = {
    "euskadi": "Municipio",
}

def geo_unit_word(city):
    return GEO_UNIT_WORD.get(city, "Barrio")

ROOM_TYPES = ["Entire home/apt", "Private room", "Shared room", "Hotel room"]

# Euskadi no tiene un único centro (ver 02_feature_engineering.ipynb de Euskadi,
# sección 8.1): distance_to_center_km ahí se calcula como la distancia a la capital
# de provincia más cercana de las tres, no a un único punto. Se replica aquí para
# que la predicción en vivo use la misma definición que el modelo entrenado.
_EUSKADI_CAPITALS = {
    "Bilbao": (43.2630, -2.9350),
    "Donostia-San Sebastián": (43.3183, -1.9812),
    "Vitoria-Gasteiz": (42.8467, -2.6716),
}

_EARTH_R_KM = 6371.0
_NEARBY_RADIUS_M = 150

@st.cache_resource(show_spinner=False)
def _load_city_bundle(city: str):
    """Carga todo lo que depende de la ciudad activa: metadatos del modelo, columnas
    de features, estadísticas de barrio y el árbol espacial para la densidad de
    anuncios cercanos. Cacheado por ciudad: cambiar de ciudad en el selector solo
    recalcula la primera vez, las siguientes veces reutiliza lo ya cargado."""
    with open(PROJECT_DIR / "models" / city / "model_metadata.json", encoding="utf-8") as f:
        metadata = json.load(f)
        feature_cols = metadata["feature_cols"]

    with open(APP_DIR / "assets" / city / "neighbourhood_stats.json", encoding="utf-8") as f:
        neighbourhood_stats = json.load(f)
        districts = sorted({v["district"] for v in neighbourhood_stats.values()})
        neighbourhoods_by_district = {
            d: sorted(n for n, v in neighbourhood_stats.items() if v["district"] == d) for d in
            districts
        }
    # Málaga no tiene nivel de barrio por debajo del distrito (ver 02_feature_engineering.ipynb
    # de Málaga, sección 5.2): cada distrito mapea a un único "barrio" (él mismo). En ese caso
    # la app no debe mostrar distrito y barrio como si fueran dos cosas distintas.
    has_neighbourhood_level = any(len(v) > 1 for v in neighbourhoods_by_district.values())

    # Las categorías de property_type que sobrevivieron al agrupado por frecuencia
    # en 02_feature_engineering.ipynb (>=30 apariciones) varían de una ciudad a
    # otra: se leen directamente de feature_cols en vez de mantener una lista
    # aparte, para que nunca se puedan desincronizar del modelo real.
    property_types = sorted(
        c[len("property_type_"):] for c in feature_cols if c.startswith("property_type_") and c
        != "property_type_Other"
    )
    if any(c == "property_type_Other" for c in feature_cols):
        property_types.append("Other")

    # Árbol espacial sobre los anuncios reales de esta ciudad (mismo universo y
    # radio que n_nearby_150m en 02_feature_engineering.ipynb), para poder calcular
    # la densidad de un anuncio nuevo/simulado sin recorrer todos los anuncios.
    nearby_coords = pd.read_csv(
        PROJECT_DIR / "data" / "processed" / city / "listings_full_clean.csv",
        usecols=["latitude", "longitude"]
    )
    nearby_tree = BallTree(np.radians(nearby_coords[["latitude", "longitude"]].to_numpy()),
        metric="haversine")

    return {
        "metadata": metadata,
        "feature_cols": feature_cols,
        "neighbourhood_stats": neighbourhood_stats,
    }

```

```

        "districts": districts,
        "neighbourhoods_by_district": neighbourhoods_by_district,
        "has_neighbourhood_level": has_neighbourhood_level,
        "property_types": property_types,
        "nearby_tree": nearby_tree,
    }

def set_city(city: str):
    """Activa `city` como ciudad actual: recarga (o reutiliza de caché) todo lo que
    depende de ella y lo deja en variables de módulo, para que el resto de este
    fichero y de app.py sigan usando los mismos nombres de siempre (METADATA,
    DISTRICTS...) sin tener que pasar la ciudad de función en función."""
    global CITY, METADATA, FEATURE_COLS, NEIGHBOURHOOD_STATS, DISTRICTS
    global NEIGHBOURHOODS_BY_DISTRICT, HAS_NEIGHBOURHOOD_LEVEL, PROPERTY_TYPES, CITY_CENTER,
    _NEARBY_TREE

    bundle = _load_city_bundle(city)
    CITY = city
    METADATA = bundle["metadata"]
    FEATURE_COLS = bundle["feature_cols"]
    NEIGHBOURHOOD_STATS = bundle["neighbourhood_stats"]
    DISTRICTS = bundle["districts"]
    NEIGHBOURHOODS_BY_DISTRICT = bundle["neighbourhoods_by_district"]
    HAS_NEIGHBOURHOOD_LEVEL = bundle["has_neighbourhood_level"]
    PROPERTY_TYPES = bundle["property_types"]
    CITY_CENTER = CITY_CENTERS[city]
    _NEARBY_TREE = bundle["nearby_tree"]

def count_nearby(lat, lon, radius_m=_NEARBY_RADIUS_M):
    radius_rad = (radius_m / 1000) / _EARTH_R_KM
    count = _NEARBY_TREE.query_radius(np.radians([[lat, lon]]), r=radius_rad, count_only=True)
    return int(count[0])

set_city("barcelona")

FEATURE_LABELS = {
    "minimum_nights": "Estancia mínima (noches)",
    "maximum_nights": "Estancia máxima (noches)",
    "accommodates": "Huéspedes",
    "bedrooms": "Dormitorios",
    "beds": "Camas",
    "bathrooms": "Baños",
    "availability_365": "Días disponibles al año",
    "has_license": "Tiene licencia turística",
    "host_is_superhost": "Es superhost",
    "host_has_profile_pic": "Tiene foto de perfil",
    "host_identity_verified": "Identidad verificada",
    "host_entire_homes_ratio": "% de sus anuncios que son vivienda entera",
    "host_tenure_years": "Años como anfitrión",
    "host_user_tenure_years": "Años con cuenta en la plataforma",
    "calculated_host_listings_count": "Anuncios que gestiona el anfitrión",
    "neighbourhood_price_encoded": "Nivel de precios del barrio",
    "distance_to_center_km": "Distancia al centro (km)",
    "latitude": "Latitud",
    "longitude": "Longitud",
    "has_reviews": "Tiene reseñas",
    "number_of_reviews": "Número de reseñas",
    "number_of_reviews_ltm": "Reseñas último año",
    "reviews_per_month": "Reseñas al mes",
    "review_scores_rating": "Valoración media",
    "listing_age_days": "Antigüedad del anuncio (días)",
    "days_since_last_review": "Días desde la última reseña",
    "estimated_occupancy_l365d": "Noches ocupadas último año",
    "has_ac": "Aire acondicionado",
    "has_pool": "Piscina",
    "has_dishwasher": "Lavavajillas",
    "has_sea_view": "Vistas al mar",
    "n_amenities": "Nº de amenities",
    "n_nearby_150m": "Anuncios cercanos (150m)",
}

```

```

def pretty_feature_name(feature):
    if feature in FEATURE_LABELS:
        return FEATURE_LABELS[feature]
    if feature.endswith("_log"):
        base = feature[: -len("_log")]
        return f"{pretty_feature_name(base)} (log)"
    for prefix, category in [
        ("room_type_", "Tipo"),
        ("district_", "Distrito"),
        ("property_type_", "Propiedad"),
        ("host_size_", "Tamaño anfitrión"),
    ]:
        if feature.startswith(prefix):
            return f"{category}: {feature[len(prefix):]}"
    return feature.replace("_", " ").capitalize()

def haversine_km(lat1, lon1, lat2, lon2):
    r = 6371.0
    p1, p2 = math.radians(lat1), math.radians(lat2)
    dphi = math.radians(lat2 - lat1)
    dlmb = math.radians(lon2 - lon1)
    a = math.sin(dphi / 2) ** 2 + math.cos(p1) * math.cos(p2) * math.sin(dlmb / 2) ** 2
    return 2 * r * math.asin(math.sqrt(a))

def host_size_tier(listings_count):
    if listings_count <= 1:
        return "individual"
    if listings_count <= 10:
        return "pequeno"
    if listings_count <= 50:
        return "mediano"
    return "grande"

def build_features(inputs: dict):
    """inputs: dict con las claves definidas en app.py (ver sidebar).
    Devuelve un dict alineado con FEATURE_COLS, listo para pasar a pandas.DataFrame.
    """
    neigh_stats = NEIGHBOURHOOD_STATS[inputs["neighbourhood"]]
    lat = inputs.get("latitude") or neigh_stats["lat"]
    lon = inputs.get("longitude") or neigh_stats["lon"]
    if CITY == "euskadi":
        distance_to_center_km = min(haversine_km(lat, lon, *c) for c in
            _EUSKADI_CAPITALS.values())
    else:
        distance_to_center_km = haversine_km(lat, lon, *CITY_CENTER)
    # Se puede forzar con inputs["n_nearby_150m"] (usado por el simulador "qué pasaría
    # si..." de Predicción); si no viene, se calcula a partir de las coordenadas.
    n_nearby_150m = inputs.get("n_nearby_150m", count_nearby(lat, lon))

    listings_count = inputs["calculated_host_listings_count"]
    tier = host_size_tier(listings_count)

    is_new_listing = inputs["is_new_listing"]
    if is_new_listing:
        number_of_reviews = 0
        number_of_reviews_ltm = 0
        reviews_per_month = 0.0
        review_scores_rating = float("nan")
        has_reviews = False
        listing_age_days = float("nan")
        days_since_last_review = float("nan")
        estimated_occupancy_l365d = float("nan")
    else:
        number_of_reviews = inputs["number_of_reviews"]
        number_of_reviews_ltm = inputs["number_of_reviews_ltm"]
        reviews_per_month = inputs["reviews_per_month"]
        review_scores_rating = inputs["review_scores_rating"]
        has_reviews = number_of_reviews > 0

```

```

listing_age_days = inputs["listing_age_days"]
days_since_last_review = inputs["days_since_last_review"]
estimated_occupancy_l365d = inputs["estimated_occupancy_l365d"]

row = {
    "host_is_superhost": inputs["host_is_superhost"],
    "host_has_profile_pic": inputs["host_has_profile_pic"],
    "host_identity_verified": inputs["host_identity_verified"],
    "latitude": lat,
    "longitude": lon,
    "accommodates": inputs["accommodates"],
    "bathrooms": inputs["bathrooms"],
    "bedrooms": inputs["bedrooms"],
    "beds": inputs["beds"],
    "minimum_nights": inputs["minimum_nights"],
    "maximum_nights": inputs["maximum_nights"],
    "availability_365": inputs["availability_365"],
    "number_of_reviews": number_of_reviews,
    "number_of_reviews_ltm": number_of_reviews_ltm,
    "estimated_occupancy_l365d": estimated_occupancy_l365d,
    "review_scores_rating": review_scores_rating,
    "calculated_host_listings_count": listings_count,
    "reviews_per_month": reviews_per_month,
    "has_license": inputs["has_license"],
    "number_of_reviews_log": math.log1p(number_of_reviews),
    "number_of_reviews_ltm_log": math.log1p(number_of_reviews_ltm),
    "reviews_per_month_log": math.log1p(reviews_per_month),
    "bedrooms_log": math.log1p(inputs["bedrooms"]),
    "beds_log": math.log1p(inputs["beds"]),
    "bathrooms_log": math.log1p(inputs["bathrooms"]),
    "calculated_host_listings_count_log": math.log1p(listings_count),
    "neighbourhood_price_encoded": neigh_stats["price_encoded"],
    "host_tenure_years": inputs["host_tenure_years"],
    "host_entire_homes_ratio": inputs["host_entire_homes_ratio"],
    "has_reviews": has_reviews,
    "listing_age_days": listing_age_days,
    "days_since_last_review": days_since_last_review,
    "distance_to_center_km": distance_to_center_km,
    "host_user_tenure_years": inputs["host_user_tenure_years"],
    "has_ac": inputs["has_ac"],
    "has_pool": inputs["has_pool"],
    "has_dishwasher": inputs.get("has_dishwasher", False),
    "has_sea_view": inputs.get("has_sea_view", False),
    "n_amenities": inputs["n_amenities"],
    "n_nearby_150m": n_nearby_150m,
}

for rt in ROOM_TYPES:
    row[f"room_type_{rt}"] = inputs["room_type"] == rt

for d in DISTRICTS:
    row[f"district_{d}"] = neigh_stats["district"] == d

for pt in PROPERTY_TYPES:
    row[f"property_type_{pt}"] = inputs["property_type"] == pt

for t in ["individual", "pequeno", "mediano", "grande"]:
    row[f"host_size_{t}"] = tier == t

return {col: row[col] for col in FEATURE_COLS}

def confidence_flags(inputs: dict):
    """Señales de baja confianza documentadas en 05_model_evaluation.ipynb:
    anuncios tipo Group Flat (cola alta de precio) y distritos con poca muestra.
    """
    flags = []
    if inputs["bedrooms"] >= 7 or inputs["accommodates"] >= 10:
        flags.append(
            "Este tamaño de alojamiento (Group Flat) tiene pocos anuncios de referencia "
            "en los datos históricos: trata el resultado como orientativo."
        )
    neigh_stats = NEIGHBOURHOOD_STATS[inputs["neighbourhood"]]

```

```

if neigh_stats["district"] in LOW_SAMPLE_DISTRICTS.get(CITY, ()):
    flags.append(
        f"{neigh_stats['district']} tiene poca representación en los datos de entrenamiento,
        "
        "así que la predicción es menos fiable que en distritos con más anuncios."
    )
if inputs["is_new_listing"]:
    flags.append(
        "Al no tener reseñas todavía, el modelo estima con menos información de la habitual."
    )
return flags

```

app/comparables.py

Selección de un conjunto único de anuncios reales comparables, consumido por las pantallas de veredicto, lista y mapa.

```

"""Conjunto único de comparables para la pantalla Comparar: un anuncio del
usuario entra, un conjunto de anuncios reales ordenados por parecido sale.
Veredicto, lista y mapa consumen el resultado de este módulo para no volver
a calcular "los mismos" comparables de formas distintas y contradecirse
entre sí. Sin Streamlit dentro: solo pandas/numpy, para poder testear y
razonar sobre esto sin levantar la app.
"""

from __future__ import annotations

from dataclasses import dataclass

import numpy as np
import pandas as pd

# Si un alcance geográfico se queda con menos comparables que esto, se amplía
# al siguiente (barrio -> distrito -> ciudad) antes de darlo por bueno.
MIN_COMPARABLES = 30

# Como mucho 2 anuncios del mismo anfitrión sobreviven al ordenar por
# proximidad: los datos de Airbnb están llenos de hosts con varias
# propiedades casi idénticas, y dejarlas todas sesga el conjunto hacia ese
# host en vez de reflejar variedad real del mercado.
MAX_POR_ANFITRION = 2

ALCANCES = ("barrio", "distrito", "ciudad")

COL_BARRIO = "neighbourhood_cleansed"
COL_DISTRITO = "neighbourhood_group_cleansed"

# Cajones de estancia mínima: no es un rango libre alrededor del valor del
# usuario porque los anuncios de estancia larga tienen otra lógica de precio
# y contaminarían el rango típico de uno de estancia corta con un valor de
# minimum_nights parecido en número pero de mercado distinto (ver
# MARKET_CUTOFF en app.py para la frontera turístico/temporada, un corte
# distinto y más grueso que este).
CORTA_MAX = 3
MEDIA_MAX = 7

def tramo_estancia(minimum_nights) -> str:
    """'corta' (1-3 noches), 'media' (4-7) o 'larga' (8+). Un valor por
    debajo de 1 (no debería darse en datos reales) cae en 'corta'."""
    if minimum_nights <= CORTA_MAX:
        return "corta"
    if minimum_nights <= MEDIA_MAX:
        return "media"
    return "larga"

def _tramo_estancia_vectorizado(minimum_nights: pd.Series) -> pd.Series:
    condiciones = [minimum_nights <= CORTA_MAX, minimum_nights <= MEDIA_MAX]

```

```

opciones = ["corta", "media"]
return pd.Series(np.select(condiciones, opciones, default="larga"),
index=minimum_nights.index)

@dataclass(frozen=True)
class ResultadoComparables:
    df: pd.DataFrame
    alcance_solicitado: str
    alcance_final: str
    se_amplio: bool
    n_en_alcance_original: int
    insuficiente: bool # True si ni siquiera a nivel ciudad se llega a MIN_COMPARABLES

def conjunto_comparables(anuncio_usuario: dict, df_zona: pd.DataFrame, alcance: str) ->
ResultadoComparables:
    """Filtra df_zona (todos los anuncios de la ciudad activa, ver
    load_comparables en app.py) a los comparables de anuncio_usuario.

    Filtros duros, nunca relajados: mismo room_type, capacidad  $\pm 1$ , mismo
    número de bedrooms y mismo tramo de estancia mínima (ver
    tramo_estancia). El alcance geográfico (barrio/distrito/ciudad) sí se
    amplía como respaldo si el original no llega a MIN_COMPARABLES.

    bedrooms entra como filtro exacto (no  $\pm 1$ , a diferencia de accommodates)
    porque, a igual capacidad, el número de habitaciones por sí solo ya
    explica una diferencia de precio real y grande entre anuncios (datos de
    Barcelona: 234€/mediana con 1 hab. frente a 297€ con 3 hab., para la
    misma capacidad 3-5) – mezclarlos en un mismo grupo inflaba la dispersión
    de la mediana sin motivo. Reduce el tamaño típico del conjunto (~20%) y
    sube algo la tasa de "alcance ampliado"/"insuficiente", un precio que
    merece la pena por un peer-group más comparable de verdad.

    anuncio_usuario necesita: room_type, accommodates, bedrooms,
    minimum_nights, neighbourhood, district. df_zona necesita: room_type,
    accommodates, bedrooms, minimum_nights, neighbourhood_cleansed y, si la
    ciudad tiene nivel de distrito propio, neighbourhood_group_cleansed (si
    no está la columna, "distrito" cae sobre la misma columna que "barrio",
    igual que ya hace find_similar_comparables en app.py para Málaga).
    """
    if alcance not in ALCANCES:
        raise ValueError(f"alcance debe ser uno de {ALCANCES}, recibido: {alcance!r}")

    tramo = tramo_estancia(anuncio_usuario["minimum_nights"])
    tiene_distrito = COL_DISTRITO in df_zona.columns

    base = df_zona[
        (df_zona["room_type"] == anuncio_usuario["room_type"])
        & df_zona["accommodates"].between(anuncio_usuario["accommodates"] - 1,
        anuncio_usuario["accommodates"] + 1)
        & (df_zona["bedrooms"] == anuncio_usuario["bedrooms"])
        & (_tramo_estancia_vectorizado(df_zona["minimum_nights"]) == tramo)
    ]

    def _en_alcance(alc: str) -> pd.DataFrame:
        if alc == "ciudad":
            return base
        if alc == "distrito" and tiene_distrito:
            return base[base[COL_DISTRITO] == anuncio_usuario["district"]]
        # "barrio", o "distrito" en una ciudad sin nivel de distrito propio.
        return base[base[COL_BARRIO] == anuncio_usuario["neighbourhood"]]

    n_en_alcance_original = len(_en_alcance(alcance))

    idx_inicio = ALCANCES.index(alcance)
    alcance_final = alcance
    resultado = _en_alcance(alcance)
    for alc in ALCANCES[idx_inicio:]:
        resultado = _en_alcance(alc)
        alcance_final = alc
        if len(resultado) >= MIN_COMPARABLES:
            break

```

```

return ResultadoComparables(
    df=resultado,
    alcance_solicitado=alcance,
    alcance_final=alcance_final,
    se_amplio=(alcance_final != alcance),
    n_en_alcance_original=n_en_alcance_original,
    insuficiente=len(resultado) < MIN_COMPARABLES,
)

# Pesos de cada eje en la distancia de proximidad. Capacidad y distancia
# geográfica pesan más (son lo primero que un anfitrión mira para decidir si
# dos anuncios "compiten" de verdad); baños pesa menos (con la misma
# capacidad y habitaciones, el número de baños distingue poco el precio).
# Suman 1.0.
PESOS_PROXIMIDAD = {
    "distancia": 0.30,
    "capacidad": 0.30,
    "habitaciones": 0.15,
    "estancia_minima": 0.15,
    "banos": 0.10,
}

_RADIO_TIERRA_M = 6_371_000.0

def _distancia_haversine_m(lat1, lon1, lat2, lon2):
    """Distancia haversine en metros. Acepta escalares o pd.Series/np.array."""
    lat1_r, lon1_r, lat2_r, lon2_r = np.radians(lat1), np.radians(lon1), np.radians(lat2),
    np.radians(lon2)
    dlat = lat2_r - lat1_r
    dlon = lon2_r - lon1_r
    a = np.sin(dlat / 2) ** 2 + np.cos(lat1_r) * np.cos(lat2_r) * np.sin(dlon / 2) ** 2
    return 2 * _RADIO_TIERRA_M * np.arcsin(np.sqrt(a))

def _normalizar(valores: pd.Series) -> pd.Series:
    """Divide por la desviación típica de `valores` (z-score sin restar la
    media: aquí valores son diferencias absolutas, ya positivas, y lo único
    que importa es la escala). Con desviación 0 (todas las filas iguales en
    ese eje) no hay nada que distinga a las filas: devuelve 0 en vez de
    dividir por cero."""
    desviacion = valores.std(ddof=0)
    if not desviacion or pd.isna(desviacion):
        return pd.Series(0.0, index=valores.index)
    return valores / desviacion

def ordenar_por_proximidad(anuncio_usuario: dict, df_comparables: pd.DataFrame) -> pd.DataFrame:
    """Reordena df_comparables (normalmente el .df de un ResultadoComparables)
    de más a menos parecido a anuncio_usuario, con como mucho
    MAX_POR_ANFITRION filas por host_id.

    anuncio_usuario necesita: latitude, longitude, accommodates, bedrooms,
    bathrooms, minimum_nights. df_comparables necesita esas mismas columnas
    más host_id (no la carga hoy load_comparables en app.py: hace falta
    añadirla ahí para conectar esto a la interfaz).

    Añade una columna "distancia_m" (metros al anuncio del usuario) al
    resultado; el resto de columnas de df_comparables se conservan tal cual.
    No excluye reseñas ni historial del anfitrión de la distancia: explican
    el precio, no el parecido a ojos de un anfitrión.
    """
    if "host_id" not in df_comparables.columns:
        raise ValueError(
            "df_comparables necesita la columna 'host_id' para aplicar el tope de "
            f"{MAX_POR_ANFITRION} anuncios por anfitrión."
        )
    if df_comparables.empty:
        return df_comparables.assign(distancia_m=pd.Series(dtype=float))

    df = df_comparables.copy()

```

```

df["distancia_m"] = _distancia_haversine_m(
    anuncio_usuario["latitude"], anuncio_usuario["longitude"], df["latitude"],
    df["longitude"],
)

diferencias = {
    "distancia": df["distancia_m"],
    "capacidad": (df["accommodates"] - anuncio_usuario["accommodates"]).abs(),
    "habitaciones": (df["bedrooms"] - anuncio_usuario["bedrooms"]).abs(),
    "banos": (df["bathrooms"] - anuncio_usuario["bathrooms"]).abs(),
    "estancia_minima": (df["minimum_nights"] - anuncio_usuario["minimum_nights"]).abs(),
}

puntuacion = sum(PESOS_PROXIMIDAD[eje] * _normalizar(valores) for eje, valores in
diferencias.items())

# mergesort es estable: a igualdad de puntuación, se conserva el orden de
# entrada en vez de uno arbitrario, para que el resultado sea determinista.
df = df.assign(_puntuacion=puntuacion).sort_values("_puntuacion", kind="mergesort")

vistos: dict = {}
conservar = []
for host_id, idx in zip(df["host_id"], df.index):
    n_visto = vistos.get(host_id, 0)
    if n_visto < MAX_POR_ANFITRION:
        conservar.append(idx)
        vistos[host_id] = n_visto + 1

return df.loc[conservar].drop(columns="_puntuacion")

```

Pipeline de modelización (Barcelona)

notebooks/barcelona/01_eda.ipynb

Carga, tipado, nulos, duplicados, análisis univariante/bivariante, correlaciones, geografía y amenities. Detalle narrativo completo en el Anexo II.

```

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv("/Users/yagocoll/Documents/Master/Airbnb/data/raw/barcelona/listings.csv.gz")
df.head()

print(df.shape)
df.dtypes.value_counts()

null_pct = (df.isnull().mean() * 100).round(2)
null_pct[null_pct == 100].sort_index()

df.describe().T

df["price"] = (
    df["price"]
    .str.replace("$", "", regex=False)
    .str.replace(",", "", regex=False)
    .astype(float)
)
df["price"].dtype

date_cols = ["last_scraped", "first_review", "last_review", "calendar_last_scraped"]
df[date_cols] = df[date_cols].apply(pd.to_datetime)
df[date_cols].dtypes

bool_cols = ["host_is_superhost", "host_has_profile_pic", "host_identity_verified",
"has_availability"]
df[bool_cols] = df[bool_cols].apply(lambda s: s.map({"t": True, "f": False}).astype("boolean"))

```

```

df[bool_cols].dtypes

df["host_profile_id"] = df["host_profile_id"].astype("Int64")
df["host_profile_id"].dtype

null_pct = (df.isnull().mean() * 100).round(2).sort_values(ascending=False)
null_pct[null_pct > 0]

empty_cols = [
    "neighbourhood", "host_acceptance_rate", "host_response_time", "host_thumbnail_url",
    "host_neighbourhood", "host_total_listings_count", "host_verifications", "host_since",
    "host_response_rate", "instant_bookable", "neighborhood_overview", "calendar_updated",
]
df = df.drop(columns=empty_cols)
df.shape

bathrooms_extracted = df["bathrooms_text"].str.extract(r"([\d.]+)")[0].astype(float)
half_mask = df["bathrooms_text"].str.contains("half-bath", case=False, na=False) &
bathrooms_extracted.isnull()
bathrooms_extracted[half_mask] = 0.5

recovered = df["bathrooms"].isnull() & bathrooms_extracted.notnull()
print("bathrooms recuperados desde bathrooms_text:", recovered.sum())

df["bathrooms"] = df["bathrooms"].fillna(bathrooms_extracted)
df["bathrooms"].isnull().sum()

mask = df["review_scores_rating"].isnull()
print("coincide con number_of_reviews == 0:", (df.loc[mask, "number_of_reviews"] == 0).all())
print("reviews_per_month coincide:", (df["reviews_per_month"].isnull() == mask).all())

df["reviews_per_month"] = df["reviews_per_month"].fillna(0)

df["has_license"] = df["license"].notna()
df["license"] = df["license"].fillna("Sin licencia")

df = df.dropna(subset=["price"])
df.shape

low_vol_cols = [
    "host_profile_id", "host_is_superhost", "host_name", "hosts_time_as_user_years",
    "host_identity_verified", "host_has_profile_pic", "hosts_time_as_user_months",
    "hosts_time_as_host_years", "host_listings_count", "hosts_time_as_host_months",
    "host_picture_url", "host_profile_url", "has_availability", "bathrooms_text",
    "minimum_nights", "maximum_nights", "minimum_minimum_nights", "maximum_minimum_nights",
    "minimum_maximum_nights", "maximum_maximum_nights",
]
df = df.dropna(subset=low_vol_cols)
df.shape

df["bedrooms"] = df.groupby("room_type")["bedrooms"].transform(lambda s: s.fillna(s.median()))
df["beds"] = df.groupby("room_type")["beds"].transform(lambda s: s.fillna(s.median()))

df[["bedrooms", "beds"]].isnull().sum()

null_pct = (df.isnull().mean() * 100).round(2).sort_values(ascending=False)
null_pct[null_pct > 0]

df.duplicated().sum()

df["id"].duplicated().sum()

dup_cols = ["name", "host_name", "latitude", "longitude"]
logical_dup_mask = df.duplicated(subset=dup_cols, keep=False)

print("filas implicadas:", logical_dup_mask.sum())
print("grupos:", df[logical_dup_mask].groupby(dup_cols).ngroups)

cols_except_id = [c for c in df.columns if c != "id"]
is_real_duplicate = df.duplicated(subset=cols_except_id, keep="first")
is_real_duplicate.sum()

cat_cols = df.select_dtypes(include=["object", "boolean"]).columns

```

```

df[cat_cols].nunique().sort_values(ascending=False)

df["has_availability"].value_counts(dropna=False)

df = df.drop(columns=["has_availability"])

df["source"].value_counts(normalize=True).mul(100).round(1)

df["property_type"].value_counts().head(15)

df["room_type"].value_counts()

df["neighbourhood_group_cleansed"].value_counts()

for c in ["host_is_superhost", "host_has_profile_pic", "host_identity_verified", "has_license"]:
    print(c)
    print(df[c].value_counts(normalize=True).mul(100).round(1))
    print()

num_cols = df.select_dtypes(include="number").columns
print(num_cols.tolist())
len(num_cols)

nights_family = [
    "minimum_nights", "maximum_nights", "minimum_minimum_nights", "maximum_minimum_nights",
    "minimum_maximum_nights", "maximum_maximum_nights", "minimum_nights_avg_ntm",
    "maximum_nights_avg_ntm",
]
df[nights_family].corr().round(2)

avail_family = ["availability_30", "availability_60", "availability_90", "availability_365",
"availability_eoy"]
df[avail_family].corr().round(2)

hosts_time_cols = ["hosts_time_as_user_years", "hosts_time_as_user_months",
"hosts_time_as_host_years", "hosts_time_as_host_months"]
df[hosts_time_cols].describe()

df[["host_listings_count", "calculated_host_listings_count"]].corr()

num_vars = [
    "price", "accommodates", "bathrooms", "bedrooms", "beds",
    "minimum_nights", "maximum_nights", "availability_365",
    "number_of_reviews", "number_of_reviews_ltm", "reviews_per_month",
    "review_scores_rating", "calculated_host_listings_count", "estimated_revenue_l365d",
]
df[num_vars].describe().T

df[num_vars].skew().sort_values(ascending=False)

df[num_vars].hist(bins=30, figsize=(16, 12))
plt.tight_layout()

outlier_summary = []
for c in num_vars:
    q1, q3 = df[c].quantile([0.25, 0.75])
    iqr = q3 - q1
    low, high = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    n_outliers = ((df[c] < low) | (df[c] > high)).sum()
    outlier_summary.append({
        "variable": c, "low": low, "high": high,
        "n_outliers": n_outliers, "pct_outliers": round(n_outliers / len(df) * 100, 2),
    })

outlier_summary = pd.DataFrame(outlier_summary).sort_values("pct_outliers", ascending=False)
outlier_summary

cols = ["name", "room_type", "accommodates", "bedrooms", "beds", "price"]
df[df["beds"] > 30][cols].sort_values("beds", ascending=False)

error_mask = (df["beds"] > 30) & (df["bedrooms"] < 5)
df.loc[error_mask, ["name", "bedrooms", "beds", "accommodates"]]

```

```

df.loc[error_mask, "beds"] = df.groupby("room_type")["beds"].transform("median")[error_mask]
df.loc[error_mask, ["name", "bedrooms", "beds", "accommodates"]]

df[num_vars].plot(kind="box", subplots=True, layout=(4, 4), figsize=(16, 12))
plt.tight_layout()

print(df.groupby("neighbourhood_group_cleansed")["price"].agg(["count", "mean",
"median"]).sort_values("mean", ascending=False))
print()
print(df.groupby("room_type")["price"].agg(["count", "mean", "median"]))

size_vars = ["accommodates", "bedrooms", "bathrooms", "beds"]
for c in size_vars:
    print(f"{c}: pearson={df['price'].corr(df[c]):.2f} spearman={df['price'].corr(df[c],
method='spearman'):.2f}")

df.groupby("bedrooms")["price"].agg(["count", "median"])

plt.figure(figsize=(10, 5))
sns.boxplot(data=df[df["bedrooms"] <= 6], x="bedrooms", y="price")

df["price"].corr(df["review_scores_rating"]), df["price"].corr(df["review_scores_rating"],
method="spearman")

df.groupby("has_license")["price"].agg(["count", "mean", "median"])

df.groupby(["room_type", "has_license"])["price"].median()

df.groupby("host_is_superhost")["price"].agg(["count", "mean", "median"])

top_property_types = df["property_type"].value_counts().head(10).index
df[df["property_type"].isin(top_property_types)].groupby("property_type")["price"].agg(["count",
"mean", "median"]).sort_values("median", ascending=False)

corr_vars = num_vars + ["has_license", "host_is_superhost"]
df[corr_vars].corr(numeric_only=True)["price"].sort_values(ascending=False)

df[corr_vars].corr(method="spearman", numeric_only=True)["price"].sort_values(ascending=False)

plt.figure(figsize=(10, 8))
sns.heatmap(
    df[corr_vars].corr(method="spearman"),
    annot=True, fmt=".2f", center=0, cmap="coolwarm",
)

pairplot_data = df[["price", "bedrooms", "accommodates"]].copy()
pairplot_data["price"] = np.log1p(pairplot_data["price"])
pairplot_data = pairplot_data.rename(columns={"price": "price_log1p"})

sns.pairplot(pairplot_data, plot_kws={"alpha": 0.3, "s": 10})

df[["latitude", "longitude"]].describe().loc[["min", "max"]]

plt.figure(figsize=(8, 8))
sns.scatterplot(
    data=df, x="longitude", y="latitude", hue="neighbourhood_group_cleansed",
    s=8, alpha=0.5, linewidth=0,
)
plt.gca().set_aspect("equal")
plt.legend(bbox_to_anchor=(1.02, 1), loc="upper left")

plt.figure(figsize=(8, 8))
sc = plt.scatter(
    df["longitude"], df["latitude"],
    c=np.log1p(df["price"]), cmap="viridis",
    s=8, alpha=0.6, linewidth=0,
)
plt.gca().set_aspect("equal")
plt.colorbar(sc, label="log1p(price)")
plt.xlabel("longitude")
plt.ylabel("latitude")

df.loc[df["bedrooms"] >= 10, "neighbourhood_group_cleansed"].value_counts()

```

```

from sklearn.neighbors import BallTree

coords_rad = np.radians(df[["latitude", "longitude"]].to_numpy())
tree = BallTree(coords_rad, metric="haversine")
EARTH_R_KM = 6371
radius_m = 150
counts = tree.query_radius(coords_rad, r=(radius_m / 1000) / EARTH_R_KM, count_only=True)
n_nearby_150m = counts - 1 # excluir el propio anuncio

pd.Series(n_nearby_150m, name="n_nearby_150m").describe()

import ast
from collections import Counter

amenities_parsed = df["amenities"].apply(ast.literal_eval)
n_amenities = amenities_parsed.apply(len)

print("nulos:", df["amenities"].isna().sum())
n_amenities.describe()

amenity_counter = Counter()
for lst in amenities_parsed:
    amenity_counter.update(lst)

n_once = sum(1 for c in amenity_counter.values() if c == 1)
print("valores distintos de amenity:", len(amenity_counter))
print(f"aparecen una sola vez: {n_once} de {len(amenity_counter)} ({n_once / len(amenity_counter):.0%})")

pd.Series(dict(amenity_counter.most_common(12)), name="n_anuncios")

ac_variants = [k for k in amenity_counter if "air condition" in k.lower() or
k.lower().startswith("ac ")]
{v: amenity_counter[v] for v in sorted(ac_variants, key=lambda k: -amenity_counter[k])}

def has_any(lst, keywords, exclude=None):
    for item in lst:
        low = item.lower()
        if exclude and any(ex in low for ex in exclude):
            continue
        if any(kw in low for kw in keywords):
            return True
    return False

candidates = pd.DataFrame({
    "n_amenities": n_amenities,
    "has_ac": amenities_parsed.apply(lambda l: has_any(l, ["air condition", "ac - "]]),
    "has_pool": amenities_parsed.apply(lambda l: has_any(l, ["pool"], exclude=["pool table"])),
    "has_dishwasher": amenities_parsed.apply(lambda l: has_any(l, ["dishwasher"])),
})
candidates["price"] = df["price"]

for col in ["n_amenities", "has_ac", "has_pool", "has_dishwasher"]:
    prevalencia = candidates[col].mean() if col == "n_amenities" else candidates[col].mean() *
    100
    corr = candidates[col].astype(float).corr(candidates["price"])
    label = f"media={prevalencia:.1f}" if col == "n_amenities" else
    f"prevalencia={prevalencia:.1f}%"
    print(f"{col:16s} {label:18s} corr con price: {corr:.3f}")

df.to_csv("/Users/yagocoll/Documents/Master/Airbnb/data/processed/barcelona/listings_full_clean.c
sv", index=False)

```

notebooks/barcelona/02_feature_engineering.ipynb

Construcción de las 71 columnas de features del modelo final a partir de las 43 columnas de partida (Sección 3 de la memoria).

```

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv("/Users/yagocoll/Documents/Master/Airbnb/data/processed/barcelona/listings_full_clean.csv")
df.head()

(df["price"] == df["price_quote_price_per_night"]).mean()

(df["price"] * df["estimated_occupancy_l365d"]).corr(df["estimated_revenue_l365d"])

drop_cols = [
    # URLs y metadatos de scraping
    "listing_url", "picture_url", "host_url", "host_profile_url", "host_picture_url",
    "scrape_id", "last_scraped", "source", "calendar_last_scraped",
    # texto libre sin procesar (amenities NO va aquí: se procesa en la sección 9)
    "name", "description", "host_about", "host_location",
    # columnas que ya cumplieron su función
    "bathrooms_text", "license", "host_name", "host_listings_count",
    # price_quote_* (redundante con price)
    "price_quote_checkin_date", "price_quote_checkout_date", "price_quote_total_price",
    "price_quote_price_per_night", "price_quote_raw",
    # familias redundantes de estancia
    "minimum_minimum_nights", "maximum_minimum_nights", "minimum_maximum_nights",
    "maximum_maximum_nights", "minimum_nights_avg_ntm", "maximum_nights_avg_ntm",
    # familia redundante de disponibilidad
    "availability_30", "availability_60", "availability_90", "availability_eoy",
    # ventanas de reviews más finas
    "number_of_reviews_l30d", "number_of_reviews_ly",
    # fuga de información
    "estimated_revenue_l365d",
]

df = df.drop(columns=drop_cols)
df.shape

df.columns.tolist()

log_vars = [
    "price", "number_of_reviews", "number_of_reviews_ltm", "reviews_per_month",
    "bedrooms", "beds", "bathrooms", "calculated_host_listings_count",
]

for c in log_vars:
    df[f"{c}_log"] = np.log1p(df[c])

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
sns.histplot(df, x="price", ax=axes[0])
axes[0].set_title("price")
sns.histplot(df, x="price_log", ax=axes[1])
axes[1].set_title("price_log")

df["price_per_accommodate"] = df["price"] / df["accommodates"]
df["price_per_min_night"] = df["price"] / df["minimum_nights"]

df[["price_per_accommodate", "price_per_min_night"]].describe()

sns.histplot(df, x="price_per_accommodate", hue="room_type")

size_vars = ["accommodates", "bedrooms", "beds", "bathrooms"]

corr_matrix = df[size_vars].corr().values
vif = np.diag(np.linalg.inv(corr_matrix))

pd.Series(vif, index=size_vars, name="VIF")

df = pd.get_dummies(df, columns=["room_type"], prefix="room_type")

df = pd.get_dummies(df, columns=["neighbourhood_group_cleansed"], prefix="district")

global_mean_price = df["price"].mean()

```

```

neigh_stats = df.groupby("neighbourhood_cleansed")["price"].agg(["mean", "count"])

smoothing = 10
smoothed_mean = (neigh_stats["count"] * neigh_stats["mean"] + smoothing * global_mean_price) /
(neigh_stats["count"] + smoothing)

df["neighbourhood_price_encoded"] = df["neighbourhood_cleansed"].map(smoothed_mean)

df[["neighbourhood_cleansed",
"neighbourhood_price_encoded"]].drop_duplicates().sort_values("neighbourhood_price_encoded",
ascending=False).head(10)

property_type_counts = df["property_type"].value_counts()
rare_property_types = property_type_counts[property_type_counts < 30].index

df["property_type_grouped"] = df["property_type"].where(
~df["property_type"].isin(rare_property_types), "Other"
)
df["property_type_grouped"].nunique()

df = pd.get_dummies(df, columns=["property_type_grouped"], prefix="property_type")

df["host_tenure_years"] = df["hosts_time_as_host_years"] + df["hosts_time_as_host_months"] / 12
df["host_tenure_years"].describe()

for threshold in [1, 2, 5, 10]:
    pct = (df["calculated_host_listings_count"] > threshold).mean() * 100
    print(f"más de {threshold} anuncios: {pct:.1f}%")

bins = [0, 1, 10, 50, df["calculated_host_listings_count"].max()]
labels = ["individual", "pequeno", "mediano", "grande"]
df["host_size_tier"] = pd.cut(df["calculated_host_listings_count"], bins=bins, labels=labels)

df.groupby("host_size_tier", observed=True)["price"].agg(["count", "median"])

df = pd.get_dummies(df, columns=["host_size_tier"], prefix="host_size")

df["host_entire_homes_ratio"] = (
    df["calculated_host_listings_count_entire_homes"] / df["calculated_host_listings_count"]
)
df["host_entire_homes_ratio"].corr(df["price"])

df["has_reviews"] = df["number_of_reviews"] > 0
df["has_reviews"].value_counts()

df["first_review"] = pd.to_datetime(df["first_review"])
df["last_review"] = pd.to_datetime(df["last_review"])

reference_date = df["last_review"].max()
df["listing_age_days"] = (reference_date - df["first_review"]).dt.days
df["days_since_last_review"] = (reference_date - df["last_review"]).dt.days

df[["listing_age_days", "days_since_last_review"]].describe()

df = df.drop(columns=[
    "review_scores_accuracy", "review_scores_cleanliness", "review_scores_checkin",
    "review_scores_communication", "review_scores_location", "review_scores_value",
])

def haversine(lat1, lon1, lat2, lon2):
    R = 6371 # radio de la Tierra en km
    lat1, lon1, lat2, lon2 = map(np.radians, [lat1, lon1, lat2, lon2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat / 2) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2) ** 2
    return 2 * R * np.arcsin(np.sqrt(a))

center_lat, center_lon = 41.3874, 2.1686 # Plaça Catalunya
df["distance_to_center_km"] = haversine(df["latitude"], df["longitude"], center_lat, center_lon)

df["distance_to_center_km"].describe()

df["distance_to_center_km"].corr(df["price"]),

```

```

df["distance_to_center_km"].corr(df["neighbourhood_price_encoded"])

plt.figure(figsize=(8, 5))
plt.scatter(df["distance_to_center_km"], np.log1p(df["price"]), s=5, alpha=0.2)
plt.xlabel("distance_to_center_km")
plt.ylabel("log1p(price)")

from sklearn.neighbors import BallTree

coords_rad = np.radians(df[["latitude", "longitude"]].to_numpy())
tree = BallTree(coords_rad, metric="haversine")
EARTH_R_KM = 6371
radius_m = 150
counts = tree.query_radius(coords_rad, r=(radius_m / 1000) / EARTH_R_KM, count_only=True)
df["n_nearby_150m"] = counts - 1 # excluir el propio anuncio

df["n_nearby_150m"].describe()

import ast

amenities_parsed = df["amenities"].apply(ast.literal_eval)

def has_any(lst, keywords, exclude=None):
    for item in lst:
        low = item.lower()
        if exclude and any(ex in low for ex in exclude):
            continue
        if any(kw in low for kw in keywords):
            return True
    return False

df["has_ac"] = amenities_parsed.apply(lambda l: has_any(l, ["air condition", "ac - "]))
df["has_pool"] = amenities_parsed.apply(lambda l: has_any(l, ["pool"], exclude=["pool table"]))
df["has_dishwasher"] = amenities_parsed.apply(lambda l: has_any(l, ["dishwasher"]))
df["n_amenities"] = amenities_parsed.apply(len)

df[["has_ac", "has_pool", "has_dishwasher", "n_amenities"]].describe()

df = df.drop(columns=["amenities"])
df.shape

df["host_user_tenure_years"] = df["hosts_time_as_user_years"] + df["hosts_time_as_user_months"] /
12

final_drop = [
    "property_type",
    "hosts_time_as_host_years", "hosts_time_as_host_months",
    "hosts_time_as_user_years", "hosts_time_as_user_months",
    "first_review", "last_review",
    "calculated_host_listings_count_entire_homes",
    "calculated_host_listings_count_private_rooms",
    "calculated_host_listings_count_shared_rooms",
]

df = df.drop(columns=final_drop)
df.shape

null_counts = df.isnull().sum()
null_counts[null_counts > 0]

id_cols = ["id", "host_id", "host_profile_id"]
target_col = "price"
pending_cols = ["neighbourhood_cleaned"]
feature_cols = [c for c in df.columns if c not in id_cols + [target_col] + pending_cols]

print("identificadores:", len(id_cols))
print("objetivo:", target_col)
print("pendiente de la corrección de fuga (03_model_baseline.ipynb):", len(pending_cols))
print("features:", len(feature_cols))

df.to_csv("/Users/yagocoll/Documents/Master/Airbnb/data/processed/barcelona/listings_full_feature
s.csv", index=False)

```

Baselines (media, mediana, mediana por bedrooms, regresión lineal) y detección/corrección de la fuga en `neighbourhood_price_encoded` (Sección 3.4).

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

df = pd.read_csv("/Users/yagocoll/Documents/Master/Airbnb/data/processed/barcelona/listings_full_features.csv")
df.shape

id_cols = ["id", "host_id", "host_profile_id"]
target_col = "price"
leakage_cols = ["price_log", "price_per_accommodate", "price_per_min_night"]
# neighbourhood_cleansed todavía no es una feature (viene de 02_feature_engineering.ipynb)
# pendiente de la corrección de la sección 3.1bis): se excluye de X igual que los
# identificadores, pero se mantiene en df para poder usarla justo después del split.
pending_cols = ["neighbourhood_cleansed"]
feature_cols = [c for c in df.columns if c not in id_cols + [target_col] + leakage_cols +
pending_cols]

X = df[feature_cols].copy()
y = df[target_col].copy()

X.shape, y.shape

bool_cols = X.select_dtypes(include="bool").columns
X[bool_cols] = X[bool_cols].astype(int)
len(bool_cols)

null_cols = X.columns[X.isnull().any()].tolist()
print(null_cols)

X[null_cols] = X[null_cols].fillna(X[null_cols].median())
X.isnull().sum().sum()

room_type_cols = [c for c in df.columns if c.startswith("room_type-")]
room_type_for_stratify = df[room_type_cols].idxmax(axis=1)

X_train, X_test, y_train, y_test, df_train, df_test = train_test_split(
    X, y, df, test_size=0.2, random_state=42, stratify=room_type_for_stratify
)

X_train.shape, X_test.shape

smoothing = 10
global_mean_price_train = y_train.mean()
neigh_stats_train = df_train.groupby("neighbourhood_cleansed")["price"].agg(["mean", "count"])
smoothed_mean_train = (
    neigh_stats_train["count"] * neigh_stats_train["mean"] + smoothing * global_mean_price_train
) / (neigh_stats_train["count"] + smoothing)

df_train["neighbourhood_price_encoded"] =
df_train["neighbourhood_cleansed"].map(smoothed_mean_train)
df_test["neighbourhood_price_encoded"] = (
    df_test["neighbourhood_cleansed"].map(smoothed_mean_train).fillna(global_mean_price_train)
)

# X_train/X_test ya tenían la versión con fuga (calculada en la sección 2.1 antes del
# split): se sincronizan con el valor corregido de df_train/df_test.
X_train["neighbourhood_price_encoded"] = df_train["neighbourhood_price_encoded"]
X_test["neighbourhood_price_encoded"] = df_test["neighbourhood_price_encoded"]

df_train = df_train.drop(columns=["neighbourhood_cleansed"])
df_test = df_test.drop(columns=["neighbourhood_cleansed"])
```

```

df_train[["neighbourhood_price_encoded"]].describe()

df_train.to_csv("/Users/yagocoll/Documents/Master/Airbnb/data/processed/barcelona/listings_train.csv", index=False)
df_test.to_csv("/Users/yagocoll/Documents/Master/Airbnb/data/processed/barcelona/listings_test.csv", index=False)

global_mean = y_train.mean()
pred_mean = np.full(len(y_test), global_mean)

print("RMSE:", np.sqrt(mean_squared_error(y_test, pred_mean)))
print("MAE:", mean_absolute_error(y_test, pred_mean))
print("R2:", r2_score(y_test, pred_mean))

global_median = y_train.median()
pred_median = np.full(len(y_test), global_median)

print("RMSE:", np.sqrt(mean_squared_error(y_test, pred_median)))
print("MAE:", mean_absolute_error(y_test, pred_median))
print("R2:", r2_score(y_test, pred_median))

train_medians_by_bedrooms = X_train.assign(price=y_train).groupby("bedrooms")["price"].median()
pred_bedrooms = X_test["bedrooms"].map(train_medians_by_bedrooms).fillna(global_median)

print("RMSE:", np.sqrt(mean_squared_error(y_test, pred_bedrooms)))
print("MAE:", mean_absolute_error(y_test, pred_bedrooms))
print("R2:", r2_score(y_test, pred_bedrooms))

def evaluate(y_true, y_pred, name):
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)
    mape = np.mean(np.abs((y_true - y_pred) / y_true)) * 100
    return {"modelo": name, "RMSE": rmse, "MAE": mae, "R2": r2, "MAPE": mape}

rows = [
    evaluate(y_test, pred_mean, "Media"),
    evaluate(y_test, pred_median, "Mediana"),
    evaluate(y_test, pred_bedrooms, "Mediana por bedrooms"),
]

results = pd.DataFrame(rows).set_index("modelo")
results.round(2)

y_train_log = np.log1p(y_train)

model = LinearRegression()
model.fit(X_train, y_train_log)

pred_log = model.predict(X_test)
pred_lr = np.expm1(pred_log)

results.loc["Regresión lineal (log)"] = evaluate(y_test, pred_lr, "Regresión lineal (log)")
results.round(2)

import numpy.linalg as la

la.cond(X_train.values)

```

notebooks/barcelona/04_model_training.ipynb

Comparativa de las cuatro familias de modelos y ajuste de hiperparámetros con RandomizedSearchCV (Sección 4.2-4.3).

```

import pandas as pd
import numpy as np
import seaborn as sns

```

```

import matplotlib.pyplot as plt
import shap
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
from sklearn.ensemble import RandomForestRegressor, ExtraTreesRegressor,
HistGradientBoostingRegressor
from sklearn.model_selection import cross_val_score, RandomizedSearchCV
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

train_df = pd.read_csv("/Users/yagocoll/Documents/Master/Airbnb/data/processed/barcelona/listings_
_train.csv")
test_df =
pd.read_csv("/Users/yagocoll/Documents/Master/Airbnb/data/processed/barcelona/listings_test.csv")
train_df.shape, test_df.shape

id_cols = ["id", "host_id", "host_profile_id"]
target_col = "price"
leakage_cols = ["price_log", "price_per_accommodate", "price_per_min_night"]
feature_cols = [c for c in train_df.columns if c not in id_cols + [target_col] + leakage_cols]

len(feature_cols)

X_train, y_train = train_df[feature_cols].copy(), train_df[target_col].copy()
X_test, y_test = test_df[feature_cols].copy(), test_df[target_col].copy()

X_train.shape, X_test.shape

bool_cols = X_train.select_dtypes(include="bool").columns
X_train[bool_cols] = X_train[bool_cols].astype(int)
X_test[bool_cols] = X_test[bool_cols].astype(int)

len(bool_cols)

X_train.isnull().sum()[X_train.isnull().sum() > 0]

y_train_log = np.log1p(y_train)

pipelines = {
    "Ridge": Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
        ("model", Ridge(random_state=42)),
    ]),
    "Random Forest": Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("model", RandomForestRegressor(random_state=42, n_jobs=-1)),
    ]),
    "Extra Trees": Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("model", ExtraTreesRegressor(random_state=42, n_jobs=-1)),
    ]),
    "Hist Gradient Boosting": Pipeline([
        ("model", HistGradientBoostingRegressor(random_state=42)),
    ]),
}

list(pipelines.keys())

def evaluate(y_true, y_pred, name):
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)
    mape = np.mean(np.abs((y_true - y_pred) / y_true)) * 100
    return {"modelo": name, "RMSE": rmse, "MAE": mae, "R2": r2, "MAPE": mape}

from sklearn.model_selection import KFold

def cv_evaluate(pipeline, X, y_log, name, n_splits=5):
    kf = KFold(n_splits=n_splits, shuffle=True, random_state=42)
    scores = []
    for train_idx, val_idx in kf.split(X):

```

```

X_tr, X_val = X.iloc[train_idx], X.iloc[val_idx]
y_tr_log, y_val_log = y_log.iloc[train_idx], y_log.iloc[val_idx]
pipeline.fit(X_tr, y_tr_log)
pred = np.expml(pipeline.predict(X_val))
y_val = np.expml(y_val_log)
scores.append(evaluate(y_val, pred, name))
return pd.DataFrame(scores).drop(columns="modelo").mean()

cv_rows = []
for name, pipe in pipelines.items():
    scores = cv_evaluate(pipe, X_train, y_train_log, name)
    scores["modelo"] = name
    cv_rows.append(scores)

cv_results = pd.DataFrame(cv_rows).set_index("modelo")
cv_results.round(2)

rf_param_dist = {
    "model__n_estimators": [100, 200, 300, 500],
    "model__max_depth": [None, 10, 20, 30],
    "model__min_samples_split": [2, 5, 10],
    "model__min_samples_leaf": [1, 2, 4],
    "model__max_features": ["sqrt", "log2", 0.5],
}

rf_search = RandomizedSearchCV(
    pipelines["Random Forest"], rf_param_dist,
    n_iter=15, cv=3, scoring="r2", random_state=42, n_jobs=-1,
)
rf_search.fit(X_train, y_train_log)

print(rf_search.best_params_)
print(rf_search.best_score_)

et_param_dist = {
    "model__n_estimators": [100, 200, 300, 500],
    "model__max_depth": [None, 10, 20, 30],
    "model__min_samples_split": [2, 5, 10],
    "model__min_samples_leaf": [1, 2, 4],
    "model__max_features": ["sqrt", "log2", 0.5],
}

et_search = RandomizedSearchCV(
    pipelines["Extra Trees"], et_param_dist,
    n_iter=15, cv=3, scoring="r2", random_state=42, n_jobs=-1,
)
et_search.fit(X_train, y_train_log)

print(et_search.best_params_)
print(et_search.best_score_)

hgb_param_dist = {
    "model__max_iter": [100, 200, 300],
    "model__max_depth": [None, 5, 10, 15],
    "model__learning_rate": [0.01, 0.05, 0.1, 0.2],
    "model__max_leaf_nodes": [15, 31, 63, 127],
    "model__l2_regularization": [0, 0.1, 1.0],
}

hgb_search = RandomizedSearchCV(
    pipelines["Hist Gradient Boosting"], hgb_param_dist,
    n_iter=15, cv=3, scoring="r2", random_state=42, n_jobs=-1,
)
hgb_search.fit(X_train, y_train_log)

print(hgb_search.best_params_)
print(hgb_search.best_score_)

from sklearn.base import clone

tuned_pipelines = {
    "Random Forest (tuned)": clone(rf_search.best_estimator_),
    "Extra Trees (tuned)": clone(et_search.best_estimator_),

```

```

    "Hist Gradient Boosting (tuned)": clone(hgb_search.best_estimator_),
}

tuned_rows = []
for name, pipe in tuned_pipelines.items():
    scores = cv_evaluate(pipe, X_train, y_train_log, name)
    scores["modelo"] = name
    tuned_rows.append(scores)

tuned_results = pd.DataFrame(tuned_rows).set_index("modelo")
tuned_results.round(2)

final_model_name = "Hist Gradient Boosting"
final_pipeline = hgb_search.best_estimator_
final_pipeline

pred_final = np.expm1(final_pipeline.predict(X_test))

evaluate(y_test, pred_final, "Hist Gradient Boosting (final)")

pred_train = np.expm1(final_pipeline.predict(X_train))

pd.DataFrame([
    evaluate(y_train, pred_train, "Train"),
    evaluate(y_test, pred_final, "Test"),
]).set_index("modelo")

{
    "Regresión lineal (baseline)": {"RMSE": 172.37, "MAE": 62.83, "R2": 0.69, "MAPE": 25.77},
    "Hist Gradient Boosting (final)": evaluate(y_test, pred_final, "Hist Gradient Boosting (final)"),
}

from sklearn.inspection import permutation_importance

y_test_log = np.log1p(y_test)

perm_result = permutation_importance(
    final_pipeline, X_test, y_test_log,
    scoring="r2", n_repeats=10, random_state=42, n_jobs=-1,
)

importances = pd.Series(perm_result.importances_mean,
    index=X_test.columns).sort_values(ascending=False)
importances.head(15)

plt.figure(figsize=(8, 6))
importances.head(15).sort_values().plot(kind="barh")
plt.xlabel("Caída de R² al desordenar la variable")

explainer = shap.TreeExplainer(final_pipeline.named_steps["model"])
shap_values = explainer(X_test)

shap.plots.bar(shap_values, max_display=15)

shap.plots.beeswarm(shap_values, max_display=15)

pred_test_all = np.expm1(final_pipeline.predict(X_test))
error_test = pred_test_all - y_test.values
median_price = y_test.median()

candidate_mask = (np.abs(y_test.values - median_price) < 20) & (np.abs(error_test) < 10)
example_pos = np.where(candidate_mask)[0][0]

print(f"Precio real: {y_test.iloc[example_pos]:.2f}€, predicho: {pred_test_all[example_pos]:.2f}€")
shap.plots.waterfall(shap_values[example_pos], max_display=12)

import joblib

joblib.dump(final_pipeline,
    "/Users/yagocoll/Documents/Master/Airbnb/models/barcelona/price_model.pkl")

```

```

import json
from datetime import date

metadata = {
    "model": "HistGradientBoostingRegressor",
    "date": str(date.today()),
    "target": target_col,
    "id_cols": id_cols,
    "leakage_cols_excluded": leakage_cols,
    "feature_cols": feature_cols,
    "best_params": {k.replace("model_", ""): v for k, v in hgb_search.best_params_.items()},
    "test_metrics": evaluate(y_test, pred_final, "Hist Gradient Boosting (final)",
    "baseline_metrics": {"model": "Regresión lineal", "RMSE": 172.37, "MAE": 62.83, "R2": 0.69,
    "MAPE": 25.77},
}

metadata["test_metrics"]["R2_log"] = float(r2_score(y_test_log, final_pipeline.predict(X_test)))
with open("/Users/yagocoll/Documents/Master/Airbnb/models/barcelona/model_metadata.json", "w") as f:
    json.dump(metadata, f, indent=2, default=str)

metadata

```

notebooks/barcelona/05_model_evaluation.ipynb

Evaluación final en test, importancia de variables y SHAP (Secciones 4.4 y 5 de la memoria).

```

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import joblib
import json
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

final_pipeline =
joblib.load("/Users/yagocoll/Documents/Master/Airbnb/models/barcelona/price_model.pkl")

with open("/Users/yagocoll/Documents/Master/Airbnb/models/barcelona/model_metadata.json") as f:
    metadata = json.load(f)

test_df =
pd.read_csv("/Users/yagocoll/Documents/Master/Airbnb/data/processed/barcelona/listings_test.csv")
metadata["test_metrics"]

feature_cols = metadata["feature_cols"]
target_col = metadata["target"]

X_test = test_df[feature_cols].copy()
y_test = test_df[target_col].copy()

bool_cols = X_test.select_dtypes(include="bool").columns
X_test[bool_cols] = X_test[bool_cols].astype(int)

X_test.shape

def evaluate(y_true, y_pred, name):
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)
    mape = np.mean(np.abs((y_true - y_pred) / y_true)) * 100
    return {"modelo": name, "RMSE": rmse, "MAE": mae, "R2": r2, "MAPE": mape}

pred = np.expml(final_pipeline.predict(X_test))
residuals = pred - y_test.values

evaluate(y_test, pred, "Hist Gradient Boosting")

plt.figure(figsize=(6, 6))

```

```

plt.scatter(y_test, pred, s=8, alpha=0.3)
lims = [0, 2000]
plt.plot(lims, lims, "r--", linewidth=1)
plt.xlabel("Precio real (€)")
plt.ylabel("Precio predicho (€)")
plt.xlim(lims)
plt.ylim(lims)

print("Media:", residuals.mean())
print("Mediana:", np.median(residuals))

plt.figure(figsize=(8, 5))
sns.histplot(residuals, bins=60)
plt.axvline(0, color="r", linestyle="--")
plt.xlabel("Residuo (predicho - real), €")
plt.xlim(-500, 500)

plt.figure(figsize=(8, 5))
plt.scatter(y_test, residuals, s=8, alpha=0.3)
plt.axhline(0, color="r", linestyle="--")
plt.xlabel("Precio real (€)")
plt.ylabel("Residuo (predicho - real), €")
plt.xlim(0, 2000)

price_quintile = pd.qcut(y_test, 5)
pd.DataFrame({"residual": residuals}).groupby(price_quintile,
observed=True)["residual"].agg(["mean", "median", "count"])

room_type_cols = [c for c in feature_cols if c.startswith("room_type_")]
room_type_series = X_test[room_type_cols].idxmax(axis=1).str.replace("room_type_", "",
regex=False)

rows = []
for rt in room_type_series.unique():
    mask = (room_type_series == rt).values
    scores = evaluate(y_test[mask], pred[mask], rt)
    scores["n"] = mask.sum()
    rows.append(scores)

pd.DataFrame(rows).set_index("modelo").round(2)

district_cols = [c for c in feature_cols if c.startswith("district_")]
district_series = X_test[district_cols].idxmax(axis=1).str.replace("district_", "", regex=False)

rows = []
for d in district_series.unique():
    mask = (district_series == d).values
    scores = evaluate(y_test[mask], pred[mask], d)
    scores["n"] = mask.sum()
    rows.append(scores)

pd.DataFrame(rows).set_index("modelo").round(2).sort_values("n", ascending=False)

price_quintile = pd.qcut(y_test, 5)

rows = []
for q in price_quintile.cat.categories:
    mask = (price_quintile == q).values
    scores = evaluate(y_test[mask].values, pred[mask], str(q))
    scores["n"] = mask.sum()
    rows.append(scores)

pd.DataFrame(rows).set_index("modelo").round(2)

worst_idx = np.argsort(-np.abs(residuals))[:15]

cols_show = ["id", "price", "bedrooms", "accommodates", "minimum_nights", "bathrooms"]
worst = test_df.iloc[worst_idx][cols_show].copy()
worst["pred"] = pred[worst_idx]
worst["residual"] = residuals[worst_idx]
worst

anomaly_ids = [1183146635229347122, 1675517721211179327, 909087702399642381]

```

```
cols_anomaly = [  
    "id", "price", "bedrooms", "accommodates", "bathrooms", "minimum_nights",  
    "has_license", "host_is_superhost", "review_scores_rating", "number_of_reviews",  
]  
test_df[test_df["id"].isin(anomaly_ids)][cols_anomaly]
```